Onlinevarsity
your e-way to learning

AGILE - Paradigm Shift in SDLC

Are you registered with **Onlinevarsity.com**?	Yes	No
Did you download this book from **Onlinevarsity.com**?	Yes	No

Scores

For each **YES** you score **50**
For each **NO** you score **0**
If you score less than 100 this book is illegal.
Register on **www.onlinevarsity.com**

AGILE - Paradigm Shift in SDLC

© 2014 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED
Contact E-mail: ov-support@onlinevarsity.com

Edition 1 - 2014





Dear Learner,

We congratulate you on your decision to pursue an Aptech course.

Aptech Ltd. designs its courses using a sound instructional design model – from conceptualization to execution, incorporating the following key aspects:

- Scanning the user system and needs assessment

Needs assessment is carried out to find the educational and training needs of the learner

Technology trends are regularly scanned and tracked by core teams at Aptech Ltd. TAG* analyzes these on a monthly basis to understand the emerging technology training needs for the Industry.

An annual Industry Recruitment Profile Survey is conducted during August - October to understand the technologies that Industries would be adapting in the next 2 to 3 years. An analysis of these trends & recruitment needs is then carried out to understand the skill requirements for different roles & career opportunities.

The skill requirements are then mapped with the learner profile (user system) to derive the Learning objectives for the different roles.

- Needs analysis and design of curriculum

The Learning objectives are then analyzed and translated into learning tasks. Each learning task or activity is analyzed in terms of knowledge, skills and attitudes that are required to perform that task. Teachers and domain experts do this jointly. These are then grouped in clusters to form the subjects to be covered by the curriculum.

In addition, the society, the teachers, and the industry expect certain knowledge and skills that are related to abilities such as *learning-to-learn, thinking, adaptability, problem solving, positive attitude etc*. These competencies would cover both cognitive and affective domains.

A precedence diagram for the subjects is drawn where the prerequisites for each subject are graphically illustrated. The number of levels in this diagram is determined by the duration of the course in terms of number of semesters etc. Using the precedence diagram and the time duration for each subject, the curriculum is organized.

- Design & development of instructional materials

The content outlines are developed by including additional topics that are required for the completion of the domain and for the logical development of the competencies identified. Evaluation strategy and scheme is developed for the subject. The topics are arranged/organized in a meaningful sequence.

The detailed instructional material – Training aids, Learner material, reference material, project guidelines, etc.- are then developed. Rigorous quality checks are conducted at every stage.

- Strategies for delivery of instruction

Careful consideration is given for the integral development of abilities like thinking, problem solving, learning-to-learn etc. by selecting appropriate instructional strategies (training methodology), instructional activities and instructional materials.

The area of IT is fast changing and nebulous. Hence considerable flexibility is provided in the instructional process by specially including creative activities with group interaction between the students and the trainer. The positive aspects of Web based learning –acquiring information, organizing information and acting on the basis of insufficient information are some of the aspects, which are incorporated, in the instructional process.

- Assessment of learning

The learning is assessed through different modes – tests, assignments & projects. The assessment system is designed to evaluate the level of knowledge & skills as defined by the learning objectives.

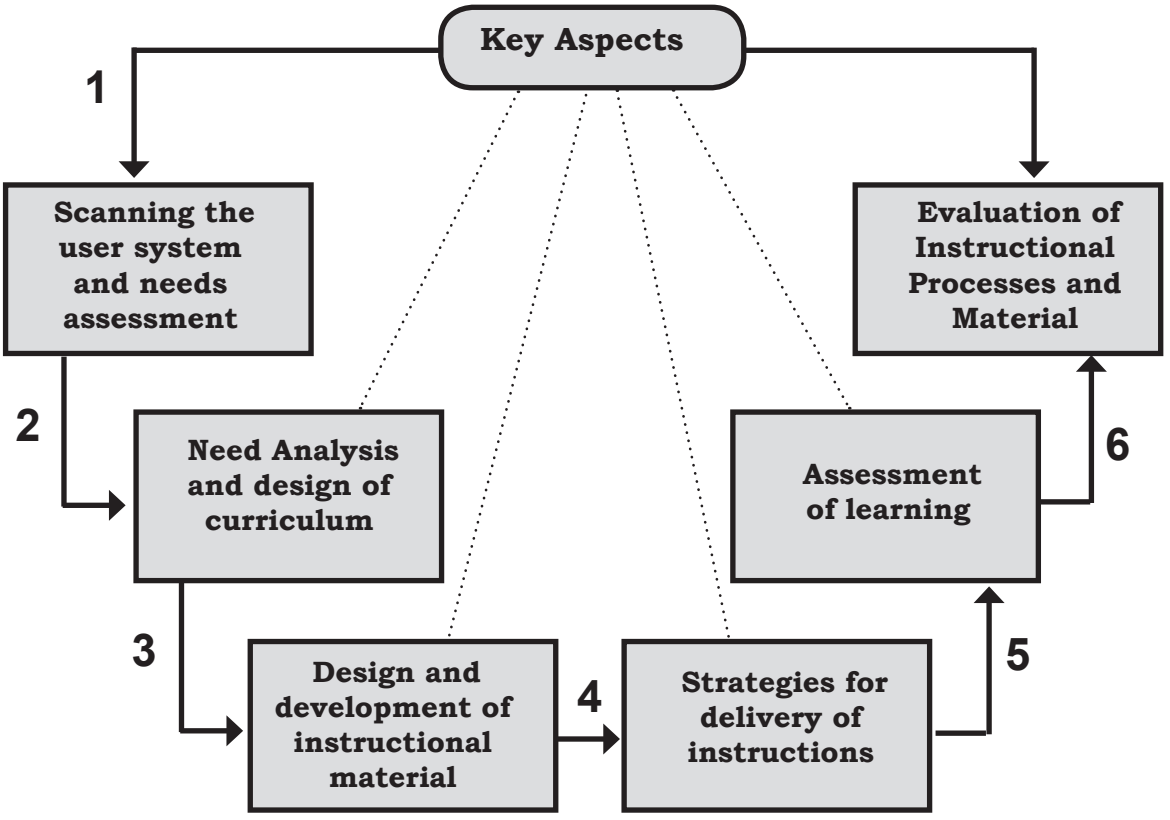
- Evaluation of instructional process and instructional materials

The instructional process is backed by an elaborate monitoring system to evaluate - on-time delivery, understanding of a subject module, ability of the instructor to impart learning. As an integral part of this process, we request you to kindly send us your feedback in the reply pre-paid form appended at the end of each module.

*TAG – Technology & Academics Group comprises of members from Aptech Ltd., professors from reputed Academic Institutions, Senior Managers from Industry, Technical gurus from Software Majors & representatives from regulatory organizations/forums.

Technology heads of Aptech Ltd. meet on a monthly basis to share and evaluate the technology trends. The group interfaces with the representatives of the TAG thrice a year to review and validate the technology and academic directions and endeavors of Aptech Ltd.

Aptech New Products Design Model



Onlinevarsity
your e-way to learning



To enhance your knowledge,

visit the **REFERENCES** *page*

@

Preface

The book, AGILE - Paradigm Shift in SDLC, enables the students to understand the Agile methodology in software development. It begins with an introduction to software and its characteristics as a process and a product. Further, it explains about Software Development Life Cycle (SDLC) concepts, phases, and methodologies that enable students to develop timely and high-quality software products.

The book explores the concept, principles, phases, and values of the Agile SDLC adopted in the software development environment. It also discusses the various Agile process models, such as Extreme Programming, Scrum, Adaptive Software Development (ASD), Dynamic System Development Method (DSDM), Feature Driven Development (FDD), and so on that encourage incremental delivery of software with improved customer satisfaction. Finally, the book provides a comparison of various SDLC process models used by organizations in software development.

This book is the result of a concentrated effort of the Design Team, which is continuously striving to bring you the best and the latest in Information Technology. The process of design has been a part of the ISO 9001 certification for Aptech-IT Division, Education Support Services. As part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends.

We will be glad to receive your suggestions.

Design Team

Table of Contents

Sessions

1. Software Characteristics
2. Software Development Life Cycle
3. Linear Process Models
4. Iterative Process Models
5. Specialized Process Models
6. Agile Software Development - I
7. Agile Software Development - II
8. Extreme Programming
9. Scrum
10. Agile Process Models - I
11. Agile Process Models - II
12. Comparing Agile with Traditional Models

Session - 1

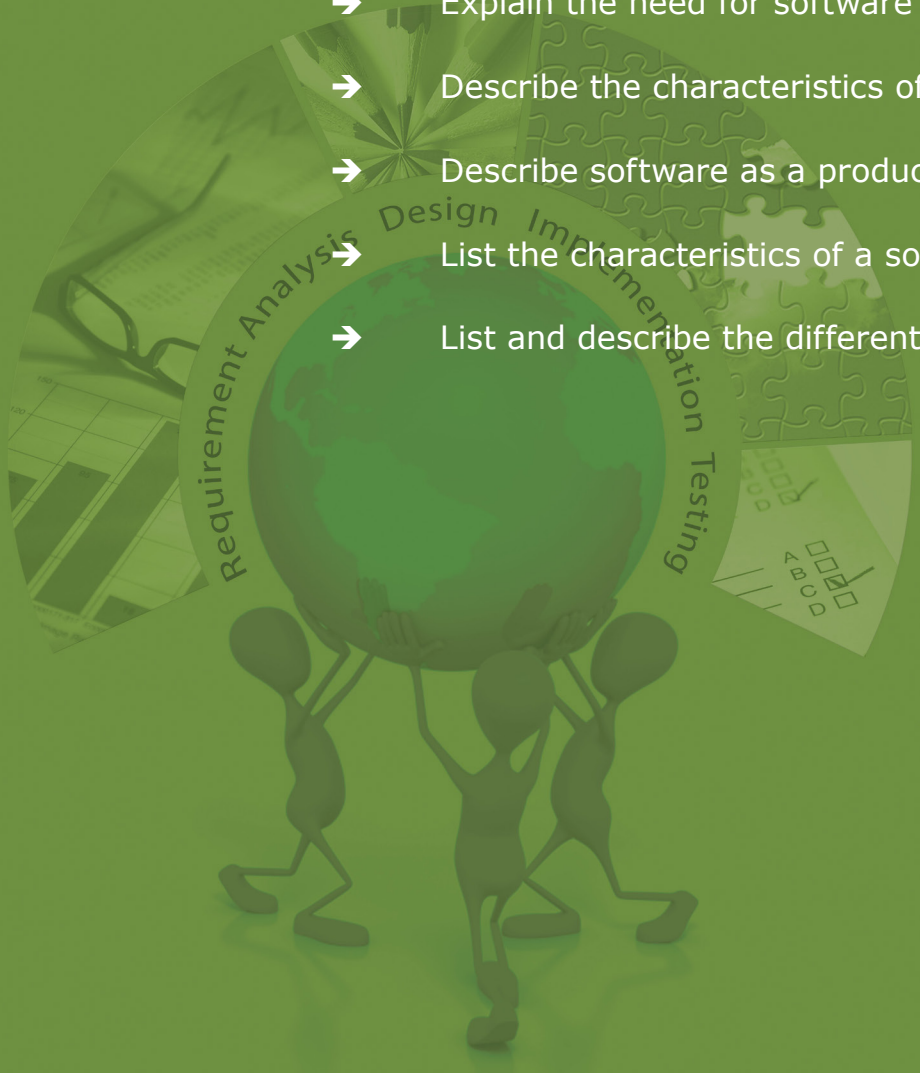
Software Characteristics

Welcome to the Session, **Software Characteristics**.

This session will introduce about software and its need. The session defines software and describe its characteristics as a process. Further, the session also describes software as a product and its characteristics. Finally, the session explains different types of software and their usage.

In this Session, you will learn to:

- Define software
- Explain the need for software
- Describe the characteristics of software as a process
- Describe software as a product
- List the characteristics of a software
- List and describe the different types of software



1.1 Introduction

The electronic computers evolved in the 1940s. Hardware posed a major challenge at that time, and all the early efforts were focused on designing the hardware. Hardware was where most technical difficulties existed. However, with the advent of new techniques, the problem subsided. With the availability of cheaper and powerful machines, higher-level languages, and more user-friendly operating systems, the applications of computers grew rapidly. In addition, the nature of software engineering evolved from simple programming exercises to developing software systems, which were much larger in scope, and required great effort by many people.

The techniques for writing simple programs could not be scaled up for developing software systems, and the computing world found itself in the midst of a 'software crisis'. At that time, the term **software engineering** was coined in the conferences sponsored by NATO Science Committee in Europe in 1960. The IEEE Glossary of Software Engineering defines software engineering as: "The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software."

Currently, computer systems are used in such diverse areas as business applications, scientific work, video games, air traffic control, and so on. This increase in the use of computers in every field has led to a dramatic increase in the need for software. Furthermore, the complexity of these systems is also increasing – imagine the complexity of the software for aircraft control or a telephone network monitoring system. The complexity has grown at such a pace that one is unable to deal with it. Consequently, many years after the software crisis was first declared, one finds that it has not yet ended. Software engineering is the discipline whose goal is to deal with this problem.

Today, software takes on a dual role. It is a product, and at the same time, the vehicle to deliver a product. As a product, it delivers the computing potential embodied by computer hardware. Whether it resides within a cellular phone or operates inside a mainframe computer, software is an information transformer – producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as a multimedia simulation. As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).

1.2 Software

Software is not merely a collection of computer programs. IEEE defines software as a collection of computer programs, procedures, rules, and associated documentation and data. Unlike a program, the programming system product is generally not used only by the author of the program, but is used largely by people other than the developers of the system.

The different components of a software are as follows:

- **Instructions or computer programs** – The execution of these provides desired features, functions, and performance.
- **Data structures** – These are used to store information required by programs to work upon or process.

Concepts

➔ **Documentation** – These describe the operation and use of programs.

Software falls under the category of programming systems product. The user may be of different backgrounds and so it is generally provided with a proper user interface. The programs are thoroughly tested before operational use so that there are no bugs left behind. Also, as diverse people having diverse hardware environment use the product.

A software process may be defined as "a framework for the tasks that are required to build high-quality software".

1.2.1 Need for Software

Software is developed to either automate or speed up an existing manual task with increased accuracy. It executes complex tasks that cannot be performed by an individual alone. The large amount of data or information can be managed efficiently and retrieved through software.

Some of the reasons that need a software development are as follows:

➔ A market trend or a demand

➔ A business requirement

➔ A client requirement

➔ A legal requirement Need to upgrade from manual to automated processes

➔ Systems such as transportation, medical, telecommunications, military, industrial, entertainment, and so on

1.3 Development of Software

Development of software is an ongoing process. As the business needs change, the software may become less useful with the new changes in the business requirement. Hence, it constantly needs to be updated or replaced.

1.3.1 Software as a Process

A software process is a set of activities, together with ordering constraints among them, such that if the activities are performed properly and in accordance with the ordering constraints, the desired result is produced. The desired result is high-quality software at low cost. Clearly, a process that does not scale up, that is, cannot handle large software projects or cannot produce good-quality software is not a suitable process.

The fundamental objectives of a process are optimality and scalability. Optimality means that the process should be able to produce high-quality software at low cost, and scalability means that it should also be applicable for large software projects. To achieve these objectives, a process should have some properties.

A few important properties are discussed as follows:

➔ **Predictability**

Predictability of a process determines how accurately the outcome of following process in a project can be predicted before the project is completed. If it is not predictable, the process is of limited use. If the past experiences to control costs and ensure quality are used, a predictable process must be used. With low predictability, the experience gained through projects is of little value. A predictable process is also said to be under statistical control.

➔ **Maintainability**

Maintenance costs generally exceed the development costs during the life of the software. Thus, to reduce the overall cost of software, the goal of development should be to reduce the maintenance effort.

According to a survey done by Bell Labs, the effort distribution within phases of a software process is shown in table 1.1.

Activity	Percentage
Requirements	10%
Design	20%
Coding	20%
Testing	50%

Table 1.1: Effort Distribution with Different Phases

Similarly, the distribution of how programmers spend their time on different activities is shown in table 1.2.

Programmer Activity	Time Percentage
Writing Programs	13%
Reading Programs and manuals	16%
Job communication	32%
Others (including personal)	39%

Table 1.2: Programmers Activity and Time Spent

So, the goal of the process should not be to reduce the effort of design and coding, but to reduce the cost of testing and maintenance.

➔ **Defect Removal and Prevention**

Errors can occur at any stage of the development cycle. However, the cost of correcting the errors of different phases is not the same, and depends on when the error is detected and corrected.

V 1.0 © Aptech Limited

If the error were corrected after coding, both design and code would have to be changed, thereby increasing the cost of correction. So, one can deduce that errors should be detected in the same phase itself in which it has originated, and not wait until testing.

Figure 1.1 shows the error detection in every phase.

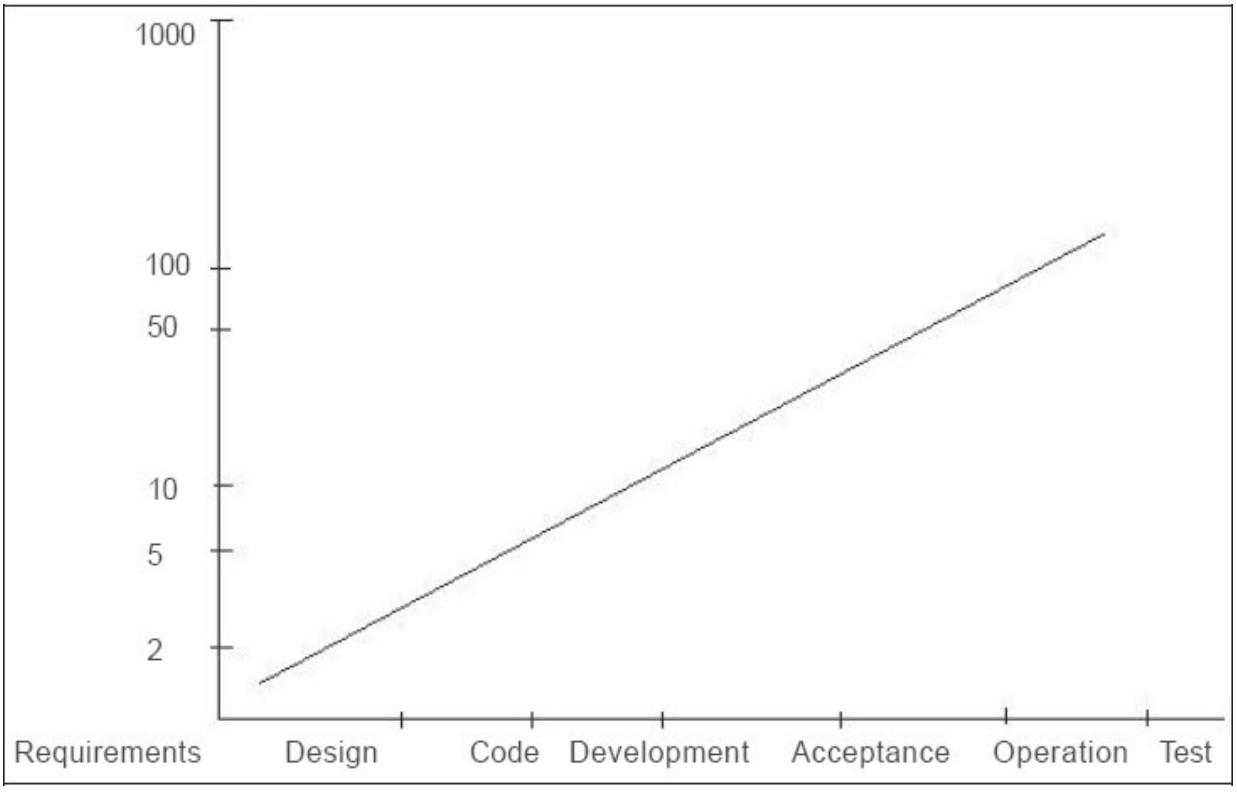


Figure 1.1: Error Detection in Phases

➔ **Process Improvement**

A process is not a static entity. As the cost and quality of the software are dependent on the software process, it should be improved to satisfy goals as quality improvement and cost reduction. The process must learn from the previous experiences. Each project done using the existing process must feed information back to the process itself, which can then use this information for self-improvement.

1.4 Software as a Product

The goal of any engineering activity is to build something- a product. The civil engineer builds a dam, the aerospace engineer makes a plane, and the electrical engineer makes a circuit. The product of software engineering is a 'software system'. It is not as tangible as the other products, but it is a product nonetheless. It serves a function.

Software unlike other products is not a physical entity. One cannot touch or feel it to get an idea about its quality. Software is a logical entity, and therefore, it is different from other engineered products.

To gain an understanding of the software, let us examine the characteristics of software that makes it different from other things that human beings build.

➔ **Software is developed or engineered, it is not manufactured**

Software does not automatically roll out of an assembly line. There exist a number of tools for automating the process, especially for the generation of code; but the development depends on the individual skills and creative abilities of developers. This ability is difficult to specify, difficult to quantify, and even more difficult to standardize.

In most engineering disciplines, the manufacturing process is considered carefully, because it determines the final cost of the product. Also, the process has to be managed closely to ensure that defects are not introduced. The same considerations apply to computer hardware products. For software, on the other hand, manufacturing is a trivial process of duplication. The software production process deals with design and implementation, rather than manufacturing.

➔ **Software is malleable**

The characteristic that sets software apart from other engineering products is that software is malleable. The product itself can be modified (as opposed to its design) rather easily. This makes it quite different from other products as cars and ovens.

The malleability of software is often misused. While it is certainly possible to modify a plane or a bridge to satisfy some new needs – so as to make it support more traffic- but this is not easy to implement. This modification is not taken lightly and it is not directly attempted on the product itself; the design is modified and the impact of change is verified extensively. Software engineers are also often asked to make modifications in their system due to the malleability property. In practice it is not easy.

The code may be changed easily, but meeting the need for which the change was intended is not necessarily to done so easily. One should indeed treat software similar to other engineering products in this regard: a change in software must be viewed as a change in design rather than in code, which is just an instance of the product.

The property of malleability of software can be used to advantage – provided it is done with a lot of discipline; this is where procedures and quality standards for making modifications become important.

➔ **Software does not 'wear out'**

The relationship between failure rate and time for hardware is shown in figure 1.2.

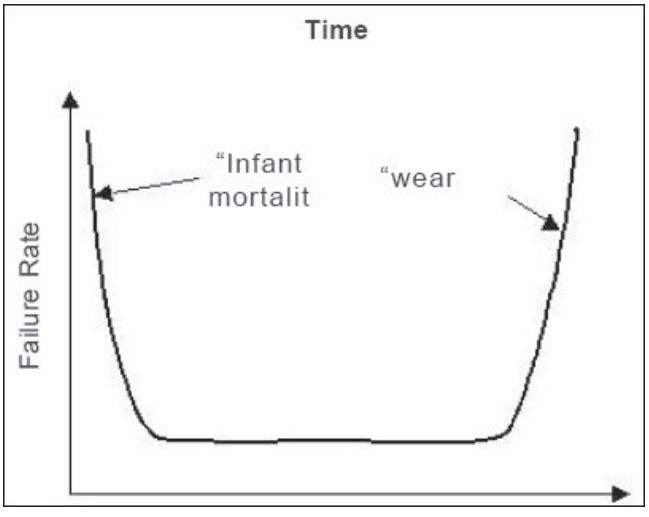


Figure 1.2: Hardware Failure Curve

As can be seen, the figure is shaped as a bathtub and therefore often called as 'bathtub curve'. The relationship depicts that the hardware shows high failure rate in its early life cycle (these failures are due to manufacturing or design defects); defects are corrected, and the failure rate falls to a steady state level for some period of time.

As time passes by, the failure rate rises again as hardware begins to wear out due to cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies.

The curve for software is not the same as shown in figure 1.3, because software is not susceptible to environmental maladies. In theory, the curve for software failure against time should be as shown in figure 1.3.

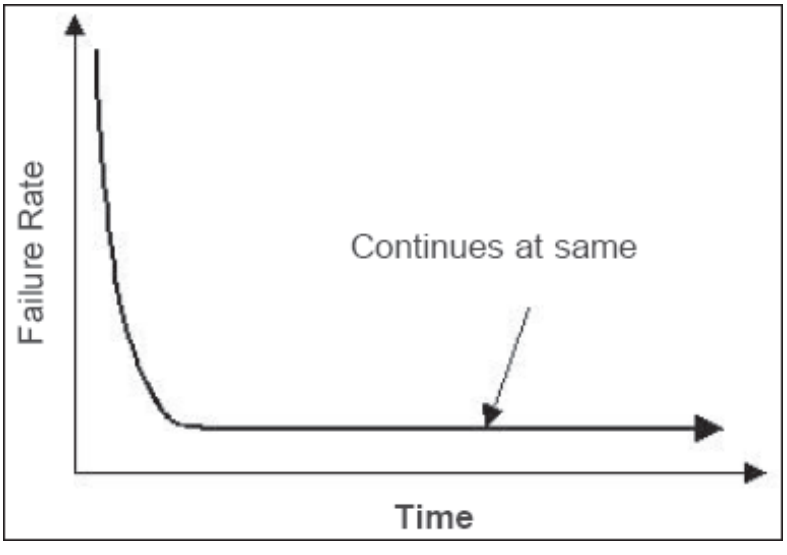


Figure 1.3: Idealized Software Failure Curves

The defects in the early stages will cause high failure rates in the early life cycle but as the defects are corrected, the failure rate drops, and the curve flatten as shown in figure 1.3.

The failure rate for software is shown in figure 1.4.

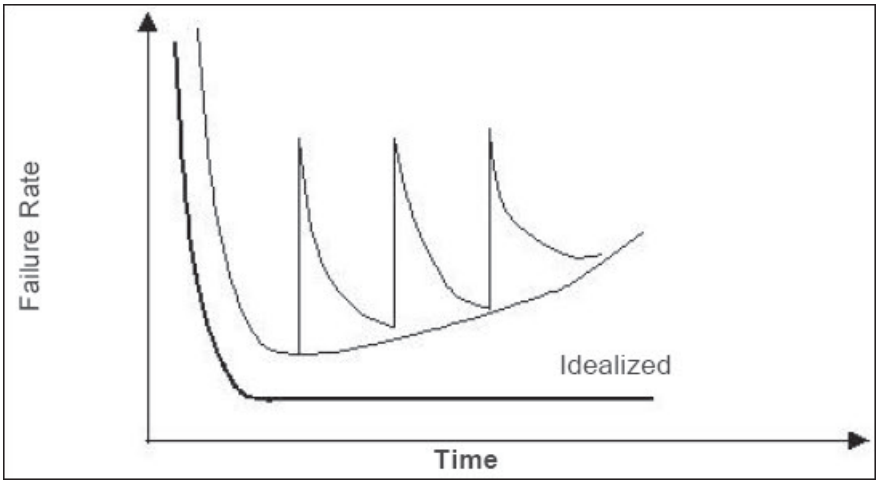


Figure 1.4: Software Failure Curve

This contradicts the earlier curve because software undergoes maintenance during its life. As changes are made, new defects are introduced, causing the failure rate to spike. As the defects are corrected, the failure rate drops again. However, before the curve can return to the original steady state, a new change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise; thus software deteriorates due to continuous change requests.

Another difference between hardware and software is that when a hardware component wears out, the part is replaced by a spare part. However, no such spare parts exist for software. Therefore, the maintenance of software is much more difficult than hardware.

➔ **Most software are custom-built, rather than being assembled from existing components**

In manufacturing engineered products, a design is first made identifying the various components that go into each product. These are then put together as per the original design. This approach affords an enormous amount of flexibility. Any number of people can work independently to produce different components so that the manufacture of one assembly can be entirely independent of another.

Now, if the organization wants to increase production quantities or decrease the production cycle time, it can easily subcontract some of the jobs. The final product merely involves putting together several independently manufactured components. For example, for building hardware, the digital components are assembled to achieve proper functioning. These digital components can be ordered 'off the shelf'.

Till now, software engineers do not have this luxury. Software can be ordered off the shelf, but as a complete unit, not as a software component that can be assembled into software programs. Fortunately, this situation is changing rapidly. The widespread use of object-oriented technology has resulted in the creation of software engineering components.

Though the scenario is changing but this is only the beginning of this concept. This concept is usually known as 'software reusability'.

1.5 Types of Software

Software can be classified into two categories:

- Application software
- System software

1.5.1 Application Software

Application software (also known as 'apps') are designed to allow the user of the system to complete a specific task or set of tasks. They include programs such as Web browsers, office software, games, and so on.

Application software can be packaged software (copyrighted software that meets the needs of a variety of users), custom software (tailor-made software developed at a user's request), freeware (copyrighted software provided at no cost), public-domain software (software donated for public use with no copyright restrictions), or shareware (copyrighted software distributed free for a trial period).

1.5.2 System Software

System software sits directly on top of computer's hardware components. It includes the range of software you would install to your system that enables it to function. This includes the operating system, drivers for hardware devices, linkers, and debuggers.

Operating System is a type of software that can be used for managing computer resources. It serves as the interface between a user, the application software, and the computer's hardware

Utility software is also a type of system software designed to help analyze, configure, optimize, or maintain a computer. Utility software includes anti-virus software, firewalls, disk defragmenters, and so on which helps to maintain and protect the computer system but does not directly interface with the hardware.

Figure 1.5 shows the system software.

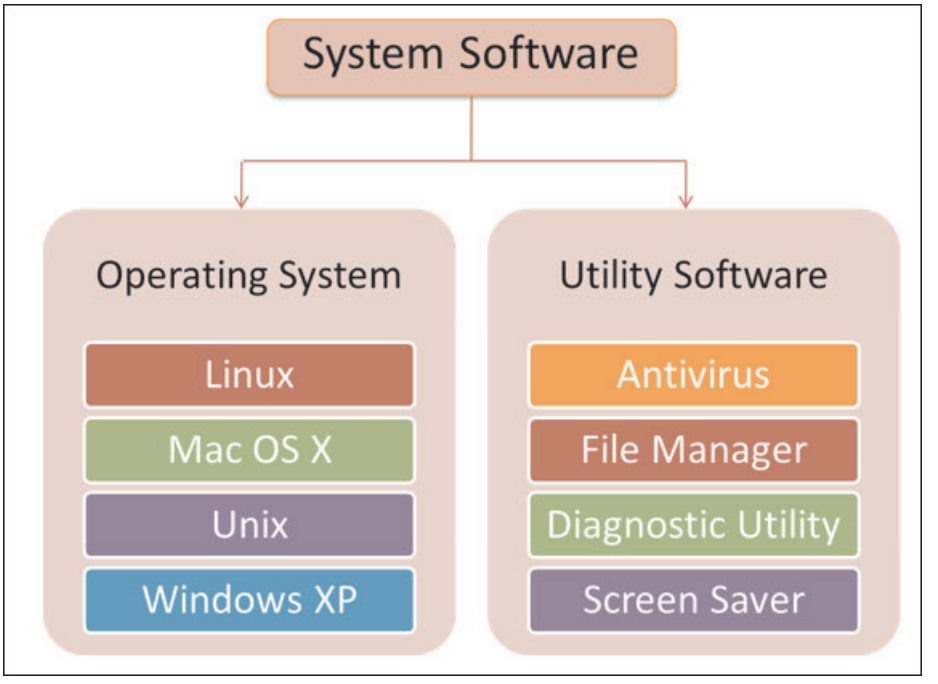


Figure 1.5: System Software

1.6 Check Your Progress

1. Which of the following software directly interacts with the end user?

(A)	System software	(C)	Utility software
(B)	Application software	(D)	Freeware software

2. Which of the following property generally exceed the development costs during the life of the software?

(A)	Maintainability	(C)	Requirement
(B)	Predictability	(D)	Defects

3. Which of the following statements are true regarding software?

(A)	Software products is a physical entity.	(C)	Software is developed or engineered, it is not manufactured.
(B)	Software is a logical entity, and therefore, it is different from other engineered products.	(D)	Software is not malleable.

(A)	A, B	(C)	C, D
(B)	B, C	(D)	A, D

4. Match the following.

Type of Software		Description	
(A)	Packaged	(1)	Copyrighted software distributed free for a trial period
(B)	Custom	(2)	Copyrighted software that meets the needs of a variety of users
(D)	Shareware	(3)	Copyrighted software provided at no cost
(E)	Freeware	(4)	Tailor-made software developed at a user's request

(A)	A-1, B-2, C-3, D-4	(C)	A-3, B-2, C-1, D-4
(B)	A-2, B-4, C-1, D-3	(D)	A-4, B-3, C-2, D-1

5. ____ means that it should also be applicable for large software projects.

(A)	Optimality	(C)	Flexibility
(B)	Scalability	(D)	Extendibility

1.6.1 Answers

1.	B
2.	A
3.	B
4.	B
5.	B



Summary

- Software is a product, and at the same time, the vehicle to deliver a product.
- IEEE defines software as a collection of computer programs, procedures, rules, and associated documentation and data.
- Software is developed to either automate or speed up an existing manual task with increased accuracy.
- A software process may be defined as “a framework for the tasks that are required to build high-quality software.”
- The fundamental objectives of a process are optimality and scalability.
- The product of software engineering is a ’software system’. It is not as tangible as the other products, but it is a product nonetheless.
- Software can be classified into two categories namely, Application software and System software.

Session - 2

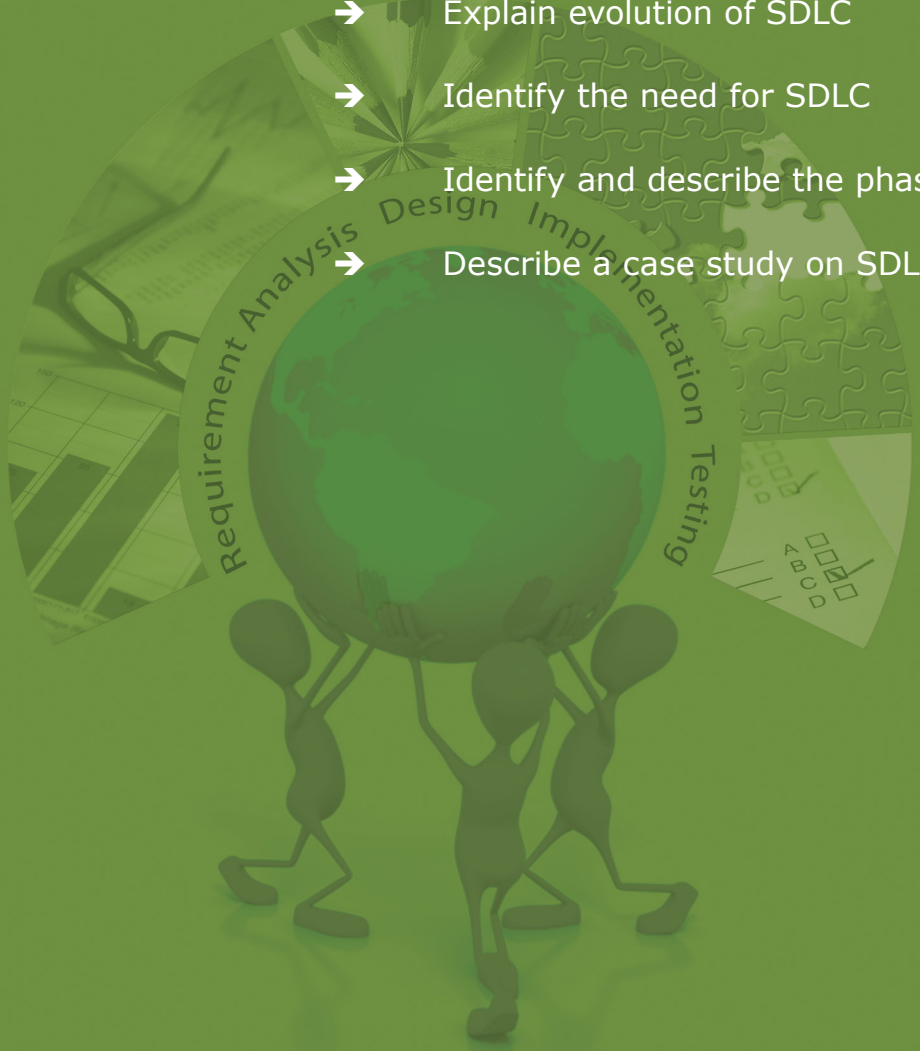
Software Development Life Cycle

Welcome to the Session, **Software Development Life Cycle**.

This session provides an introduction to Software Development Life Cycle (SDLC). It explains the and the evolution of SDLC and its need in software development. Further, the session explains various activities or phases of SDLC followed by organization to develop a software product. Finally, the session presents a case study on SDLC activities.

In this Session, you will learn to:

- Explain SDLC
- Explain evolution of SDLC
- Identify the need for SDLC
- Identify and describe the phases of SDLC
- Describe a case study on SDLC



2.1 Introduction

Software Development Life Cycle (SDLC) is a process applied to the development of a software product. This process consists of a sequence of activities or phases that govern the entire life cycle of a software development. Although the phases may vary from project to project, however, the basic concept of SDLC remains the same.

SDLC process help organizations to plan their software development projects efficiently and allocate the right resources to the project.

SDLC is applied to build new software projects and also to enhance existing software. SDLC is applied right from when the project is planned and extends throughout the development phase until the software is installed or ready for deployment.

SDLC has to meet a company's business requirements which are as follows:

- **High-quality software systems**: The customer must get value for money. Companies should consider spending money on getting new software or updating existing software worthwhile. The benefits of a new system have to be quantified and compared with the cost setting it up. This cost vs. benefit ratio is also a measure of a project's success. However, developing high-quality software is costly. Developers and customers have to strike a balance between desired level of quality and cost.

- **Strong management control**: Developers have to accurately estimate the time and resources needed to execute each project. Software projects may take years to complete, and the original requirements should still be applicable at the end of the development. Throughout a project, regular checks are made to see if it is on track. If not, then corrective measures need to be taken.

- **Increased productivity**: There may be multiple ways of approaching a software solution to a particular business challenge. Businesses and software developers have to choose the approach that yields maximum results for a given cost. If not, there may not be any actual advantage of implementing the solution.

2.2 Evolution of SDLC

The SDLC methodology did not come into existence directly. In the 1940s and 50s, there was a shift from purely mathematical or scientific applications of coding to business applications. This meant that computers and programming were open to more general uses than just technical.

In the 1960s, there was a need for an analysis and design of the project in addition to the implementation. Some of the early analysis techniques were Accurately Defined Systems (ADS) and Time Automated Grid (TAG). However, these techniques were very complex and therefore not used widely. Later, some of the new principles were incorporated such as:

- Working backwards from the desired output to determine the necessary input.

- Represent software structures graphically
- Capture requirements in a formal record.

Note - Capturing requirements refers to the process of identifying and understanding the exact purpose and scope of the project, including information gathered from customers or end-users, and using that information to build a specific product.

A new top-down approach then emerged which helped eliminate premature coding and focused on first the analysis, then the design, and finally on coding. This is how SDLC first made its appearance and was eventually made a practice in software development.

2.3 Need for SDLC

From the developer's perspective, SDLC ensures a quality software product. Using SDLC allows software developers to organize and streamline their tasks. SDLC also enables developers to estimate the time and cost of their projects accurately.

SDLC has become increasingly important today due to the following reasons:

- There is an increasing need for automation of different everyday services such as processing of bills, personal banking, ticket booking, and so on.
- As businesses evolve, their services also need to evolve, which in turn means better and more complex software.
- Software processes have to be designed to handle a wide variety of inputs at the interface level. For example, personal details entered while booking tickets. The software should be able to process input and give expected output. A software application should work similar for all users.
- There should be no errors in a software, especially while used for sensitive purposes, such as updating medical records, no matter how often or number of times it is run.
- Integrating software component with other parts of the application should be easy.

2.4 SDLC Phases

A typical SDLC consists of the phases or key activities that have to be part of any software development activity, as depicted in figure 2.1.

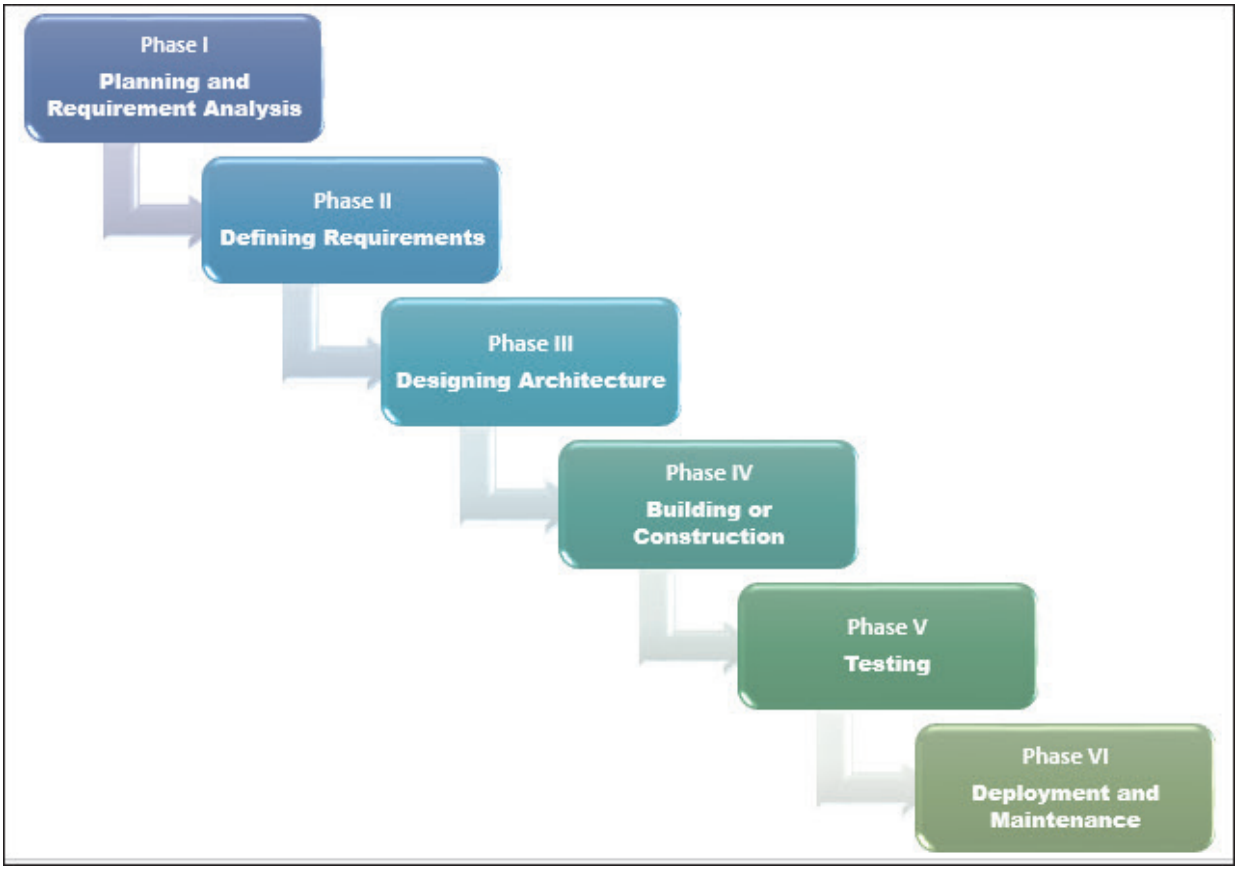


Figure 2.1: Phases of SDLC

2.4.1 Planning and Requirement Analysis

The SDLC starts with the Problem Definition phase wherein the client's problem is identified and understood.

The basic goal of requirement analysis is to gather requirements, that is, obtain a clear idea of the client requirements about project to be developed. Requirement gathering involves conducting a preliminary meeting with the client to understand and analyze the client's needs. The different techniques used here are:

- **Interviewing** - It is an oft-used technique when an analyst tries to get information regarding a problem from managers, department heads, and functional heads. The analyst will ask the customer a set of context free questions, which will lead to a basic understanding of their problem and nature of the solution that is desired. The context free questions can be:

Concepts

- "Whom will the requirements cater to?"
- "What will be the economic benefit if we come out with a successful solution?"
- "Describe the environment in which the solution is to be used."
- "Are the questions which I have been putting up relevant to the problem that on hand?"

Such questions help to focus on the overall goals, get a better understanding of the problem, the customer, and the effectiveness of the meeting.

➔ **Questionnaires** - Are used when a wide spectrum of responses is required and the consultants need definite answers to certain questions related to the problem area. For instance, getting responses from workers at a factory site. This is an ideal situation for floating questionnaires as a lot of time can be taken up interviewing such a large group. Moreover, interviews could result in vague and varied responses.

➔ **Record Reviews** - Is a technique used in all projects. It is only when existing records are reviewed that an analyst can understand the data, its movement, and usage within an organization. This cuts down on the time spent in asking people within the organization basic questions that relate to the day to day functioning of a system.

➔ **Observation** – Helps the analyst in detecting problems that exist in the current system. Users are either unaware of the problems or they are hesitant to talk about them to outsiders, thinking that they may be misinterpreted. Even when willing to talk about a problem, the user may ignore certain minute and delicate details, which may be of extreme importance to the analyst. Such problems may come to light when an analyst observes the way an organization functions.

Once the requirements have been gathered, the development team needs to examine and categorize them. They need to prioritize the requirements and give a solution stating which part of the system should be automated first, so that a major part of the problem area can be taken care of.

2.4.2 Defining Requirements

Having identified the problem, it is now to time to start developing the system. The process of system development starts with System Analysis. It is known that 'A System is a collection of interrelated components that work together to achieve some common objective' and 'System Analysis is the specification of what the system is required to do'.

System Analysis is conducted with the following objectives in mind:

➔ Identification and analysis of customer needs

➔ System evaluation for feasibility

➔ Performing economic and technical analysis

➔ Allocating functions to manpower, database, hardware, software and other system elements

➔ Establishing cost and schedule constraints

➔ Creating a system definition that forms a foundation for all subsequent development activities

System Analysis is an art, and needs immense patience backed by years of experience to get the desired system functionality and performance. System Analysis study normally conducts a feasibility study, based on its result, the translation phase of inputs into formal specified requirements is performed.

➔ **Feasibility Study**

There is a very old story about a very capable tailor. Once he used all his skill and insight to stitch a beautiful coat. When a prospective buyer pointed out that the sleeve was rather short, the tailor nonchalantly replied that the coat was so beautiful, surely the gentleman wouldn't mind cutting his hand to fit into it!

Most of us in the computer industry are often accused of having a similar affliction. That automation is indeed the solution to most organizational problems, seems to be a foregone conclusion for many a systems engineer. It is in order to temper such an over enthusiastic prescription of automation as a panacea to all problems that the Software Engineering practice has inbuilt checks and controls. One such first and foremost step is the Feasibility Study.

All projects are feasible – given unlimited resources and infinite time! Hence, before proceeding to commit on a large expenditure that is involved in the development of a new system, it is important for any organizations to be reasonably confident that the proposed system is indeed going to deliver the goods and solve the problems on hand within the expected schedules and costs. A feasibility study is a means of enabling this.

A feasibility study is conducted to find out whether the proposed system will be:

- **Possible** – to build it with the given technology and resources
- **Affordable** – given the time and cost constraints of the organization
- **Acceptable** – for use by the eventual users of the system

Feasibility study provides answers to troublesome questions like – "Can I undertake this project?", "What are my market constraints?", "What is competition in this field like?", "How do I price my product?", and "What benefits will accrue from this project?

A feasibility study is initiated by an organization, which feels there is a need for change in the current system of functioning. This need may be felt due to lapses in the current system that may have been brought to light, or the organization may wish to undergo a change in its system in order to introduce better, more efficient systems. A feasibility study is needed to ascertain the pros and cons of instituting a new system.

The purposes of this study are to:

- Determine the need for change within an organization (need analysis)

- Study the effect of the change on the economics of the organization (cost benefit analysis)
- Evaluate various technologies that can be used to implement the suggested change, given the cost and resource constraints of an organization (technical feasibility)
- Evaluate the legal procedures, if any should come into play to implement the suggested change (legal feasibility)
- Evaluate the various alternatives that will be thrown up with regard to resolving the problems of an organization, and recommend the best suited one (evaluating alternatives)

Based on the outcome of the feasibility study, the management decides to proceed to the next phase of requirement analysis.

➔ Translation of Inputs

This phase translates these informal inputs from the users into a set of formal specified requirements. The process of requirement analysis helps to bridge the communication gap between the customer and the developer. The technical development team works with the customers and system end-users to identify the application domain, functions, services, performance capabilities, hardware constraints, and so on related to the system to be developed.

The requirements must be specified in the **Requirement Specification Document (RSD)**.

The contents of the RSD are:

- Functional and performance requirements of a system
- Input-output formats
- Design constraints

After a system has been analyzed and its requirements determined, they must be placed in the context of the functions that will be ultimately incorporated in the system. Functional analysis relates to the precise definition of the functions and sub-functions to be performed by the system. The functions have certain performance requirements, as also certain design constraints that have to be borne in mind.

Example: A car is to be developed for the middle-income group of the population. Initial study is conducted with the target group of the population. Requirements state that it must be an affordable vehicle, fuel efficient, easy to run, good braking system for heavy traffic manoeuvers, and compact size for easy parking. The proposed solution should consider:

- **Functions of the car** – power brakes, easy shift floor gear system, power steering
- **Performance requirements** – fuel-efficient engine to give 'x' miles per gallon
- **Design constraints** – size of the chassis

Based on the given requirements and expected functionality, a suitable car is designed for the middle income population.

Figure 2.2 summarizes the process of defining Requirements Analysis. It starts with the process of identifying the user requirements and proposing a solution in the form of Software Requirements Specification document or Project Proposal document.

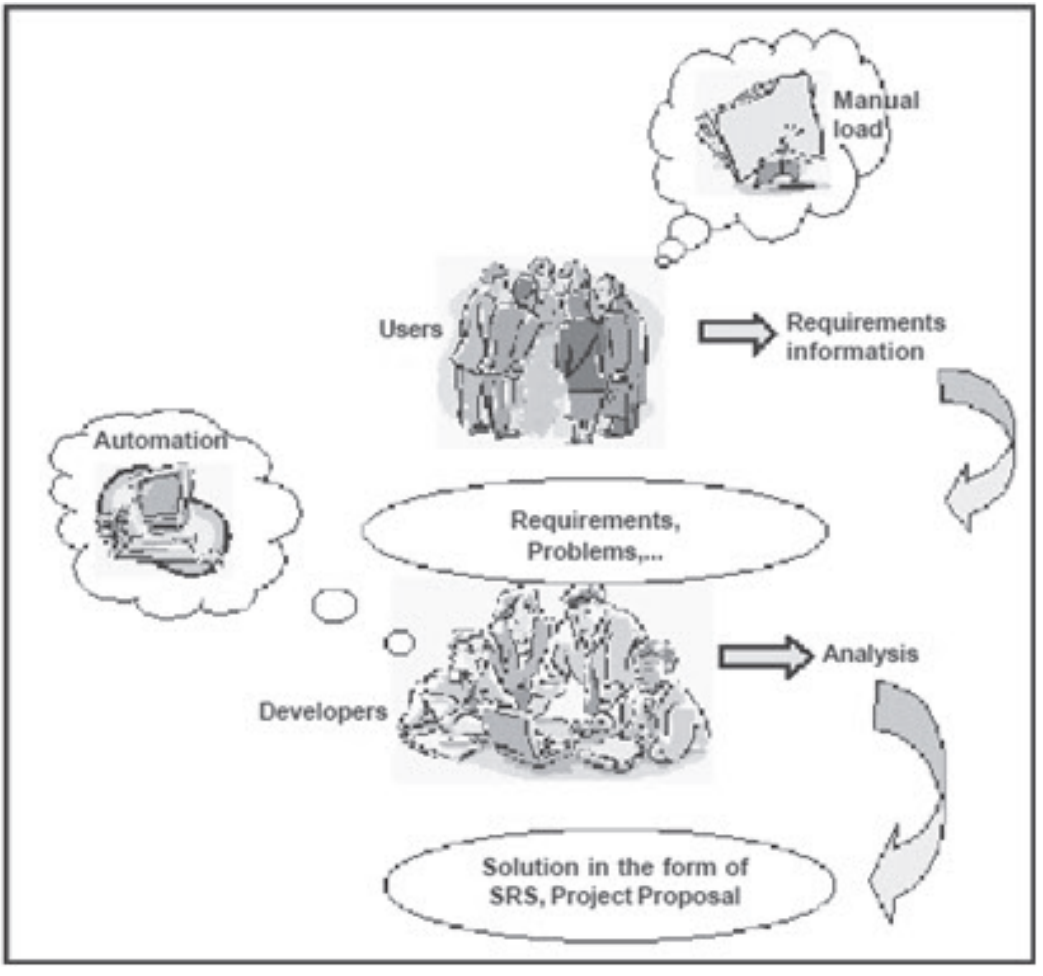


Figure 2.2: Requirement Analysis Process

2.4.3 Designing Architecture

One of the most important stages during the SDLC is the design stage. The process of design is intricate and complex, and its final goal is to produce a model or a prototype of the system that will eventually result into the proposed system. In this chapter, we will explore the basic concepts and principles that are applicable to software design.

Design can be defined as the process of applying various techniques and principles to define a device, process or system in sufficient detail to permit its physical realization.

Design is an iterative process during which requirements are translated into an outline for constructing the software.

Software design is situated at the core of the software engineering process and is always applied irrespective of the software process model used.

Concepts

The different elements are used to provide information necessary to create a design model. These are as follows:

- Entity Relationship (ER) diagram
- Data Flow Diagram (DFD)
- Control Flow diagram
- Data dictionary

The design model comprises data design, architectural design, interface design, and procedural design. The information domain model created during analysis is transformed using Data design into the data structures necessary to implement the software. Entity Relationship (ER) diagrams and data dictionary created during the analysis phase act as a source of information about data objects and relationships which will form the base for activities in data design.

- **Architectural design** - Defines the relationship among the major structural elements of the program. The modular framework of a program can be derived from the analysis model and the interaction of subsystems defined within the analysis model.
- **Interface design** - Describes how software communicates with systems that interoperate within it and with people who use it. Data and control flow diagrams provide the information required for interface design.
- **Component-level design** - Transforms structural elements of the program architecture into a procedural description of software components.

The following three characteristics of design have been suggested by McGlaughlin and they serve as a guide for the evaluation of a good design.

- The design must implement all the explicit requirements contained in the analysis model and it must accommodate all the implicit requirements desired by the customer.
- The design must be a guide that is easily understood by those who write code and for those who test and subsequently maintain the software.
- The design should provide a complete picture of the software, address the data, and functional and behavioral domains from an implementation perspective.

Technical criteria to establish a good design can be summarized as follows:

- A design should exhibit a hierarchical organization that makes intelligent use of control among elements of software.

- A design should be modular, which means software should be logically partitioned into elements that perform specific functions and sub-functions.
- A design should contain both data and procedural abstractions.
- A design should lead to interfaces that reduce the complexity of connections between modules and the external environment.
- A design should be derived using a repetitive method that is driven by information obtained during software requirements analysis.

2.4.4 Building or Construction

Life is never quite as simple as that! It is estimated that more than 80% of the problems encountered in software development project are thrown up only at the time of software building or implementation. This is the phase where the actual coding of a software application takes place, and a product is built from scratch.

In the implementation phase, a number of activities running in parallel have to culminate. The implementation phase ensures that all these activities are provided converge at the right time and at the right place without any delays on account of unforeseen problems, and that the software begins functioning as it should.

Besides systems and business issues, there are personal issues. Users are apprehensive of computers, 'afraid' of learning technology, fearful of technology savvy juniors, fearful of being termed redundant, and so on. These emotional issues complicate the process of implementing a new system within an organization. Although not beyond redemption, these issues should be accorded due importance and handled accordingly.

Therefore, during the implementation phase, besides physical implementation of hardware and software, the analyst has to more importantly ensure that the new system is implemented in the minds of the users and accepted by them.

Unlike the other phases, there are no standard methodologies for implementation. However, **Implementation Planning** is an important activity of the SDLC. The specific approach adopted for implementation is largely dependent on the size of organization, the nature of applications and the standard practices that may have been evolved by the project team.

However, despite such individual variations there are some kinds of standard activities to be performed during the implementation process:

- Creation of an Installation Plan
- Implementation of Physical Procedures
- Data Preparation and Conversion

- Conducting User Training
- Running the System
- Installation Plan

Preparing a detailed installation plan, is the beginning of the implementation process. The installation plan includes schedules for:

- Purchase of hardware
- Preparation of site – wiring, providing for communication points, air conditioning and so on
- Purchase of environmental software
- Installation of hardware and environmental software
- Installation of the developed software
- Training
- **Implementation of Physical Procedures**

  Physical Procedures are those that an organization will follow side by side with the automated system and which will complement its functioning. These procedures also spell a change from the past and have to be in place before the automated system is implemented. Some of these systems include change in coding schemes or introduction of a coding scheme, switchover to changed input and output document formats, change in sequence of operations and other such.

  The successful implementation of systems depends as much on the adaptability of the people using them, as it does on the quality of software, and hence the need for use of Physical Procedures. Users need to adapt to the new way of functioning. For instance, in the old system of functioning there would have been multiple paper copies of various documents, but in the new system the paper copies have been eliminated. Documents are stored in one place within the computer and retrieved when required. Sometimes, users are so used to writing out documents that keying them into the computer is not security enough for them, so many users may want to continue making documents manually too (at least initially). The absence of paper can create a vacuum and the users may need time to get used to this paperless concept. The best way to deal with this kind of user reluctance is to make them aware of the same, and provide training on these changed procedures before implementing the new system.

  Physical procedures are an important aspect of implementation because it spells the changeover from old to the new system and requires handling of user training on the same. Physical procedures are handled by spreading awareness of the new system through well-designed training programmes and handling user queries skillfully and patiently.

➔ Data Preparation and Conversion

Data preparation is normally the most time consuming and tedious task in the implementation of most systems. During the implementation of a new system, all data backlog has to be transcribed to fit into the new formats and coding schemes, and errors are rigorously checked to ensure no loss of information.

Data conversion is the process of converting data from the old system into a format required by the new, automated system. This usually amounts to creating the master database and entering the master data in the record formats that have been designed for the system, from the documents that exist in the system.

As a matter of fact, organizations will not want data entry people running around for data as it shifts their concentration from what they are doing, which may induce errors in the data that is being entered. Data should be consolidated and validated by someone responsible before it is entered into the system.

Data conversion is required even if an automated system already exists. In such case, a program maybe written to read data from the existing format (if the old data format is readable by new software) into the new format. Alternatively, data has to be physically read from printed dumps of the old system master file and entered into the new system. Even if the old data formats are not compatible with the new DBMS formats, a small program can be written in a language like C, which can handle any type of data formats.

This may become a cumbersome procedure and should be planned for in advance.

The process of data conversion requires the following:

1. List of all existing files to be converted (for instance, some level of automation existed in the accounts function so some data required for our system's invoice related databases will need to be converted from an existing file format into the new file format)
2. List of new files to be created and data required for it (for example, if an existing system did not have file for storing product discount details, but the new system requires it)
3. List all new documents and procedures that go into use during conversion
4. Identify controls for the conversion process, how to determine that all records have been entered in the new database
5. Prepare conversion schedule
6. Assign responsibility for each task

➔ Conducting User Training

The successful implementation of any software is dependent in good measure on the quality of training imparted to the users of the system. Different user groups need to be identified.

A Training Need Analysis has to be done for each of these groups to find the kind of training that is required for each. Training Needs Analysis can be considered as the process of determining individual or group training needs for an organization.

There are usually several types of users within an organization, and all types of users do not need all kind of training. Usually two types of users can be identified for the system:

- Hands-on users
- End users

➔ **Running the System**

The period of run varies from organization to organization and from application to application. Although there are no hard and fast rules for this phase, it is attempted to ensure that the following factors are looked into:

- There are no major faults in the package
- Important figures tally
- All modules are used and found satisfactory
- There is fair amount of confidence amongst users in being able to use the software

2.4.5 Testing

Testing is very important in SDLC as the quality of the product depends on this phase. The efficiency of the testing phase can make or break a software product. Poor quality can lead to product failure and loss of money and business for the developers as well as the customers. The reputation and success of a software development company depends on how strong their quality assurance process is.

Testing is also called as Quality Assurance (QA). The different types of testing conduct on the software are as follows:

➔ **Unit testing**: Refers to testing one component of the product. This is done by the developer after he or she finishes coding for that component. This makes sure that the component is working properly. This saves time as a component's code need not be passed to and fro between the tester and the developer.

➔ **Sanity testing**: This is done to test if all the components compile easily with each other. When multiple developers work on the same project, testing has to be done to make sure that all the components are in agreement with each other and there is no conflict.

➔ **Regression or stress testing**: This kind of testing happens over a longer period of time. Some software issues are thrown up only when the software has been run for a certain period. As it takes a lot of time, regression testing can be carried out on a continuous basis so that there is not much delay in the final product delivery.

➔ **Functional testing**: Refers to testing the software in its entirety to make sure it serves the purpose it was intended for. This is not done by developers, but professional testers whose job it is to test software for functional and technical integrity.

2.4.6 Deployment and Maintenance

The process of software engineering does not stop with the delivery and installation of software but also includes maintenance activities.

Maintenance activities involve making enhancements to the software products developed in the earlier stages of the life cycle, adapting products to new environments and correcting problems. Enhancement of the product may in turn involve providing new operational capabilities, improving the user screens and modes of interaction and improving the performance of the system in general.

Adapting the product may require moving the project to a different machine or platform, or accommodating new protocols or even new hardware components such as an additional disk drive. To correct problems, we may need to modify the code and revalidate the software. It has been practically observed that maintenance activities account for a large portion of the system life cycle costs, even as much as 60-70 percent of the total costs set aside for the project. Software maintenance like software development requires a combination of managerial control and technical expertise.

2.5 Case Study

ABC Insurance Co. is planning to create a customer information system. The company feels this will help them track the customer's policy and profile details. The system will provide better visibility on plans that are currently the best sellers in the market, besides giving the marketing team the inputs to build on these plans. It will also provide them an opportunity to revisit the plans which are not doing well.

Michael Philips, an industry expert in Software modeling, is hired by the company to help them realize this goal.

Michael recognizes from the outset that this particular project would follow the stages of a traditional SDLC. Considering the current level of expertise in the organization, he also proposes that the testing of the application be outsourced to third-party vendors to minimize the turnaround time of the application and provide accurate test case results.

How will the SDLC phases look for this case?

➔ **Requirements Gathering and Analysis**

The first step in the SDLC is to define the problem as part of the Analysis phase.

- The current approach of the company was to manually track the sales of the policies and also the customer profile and policy information. Prior to expansion, a major roadblock for the company was to automate their systems to better manage the overall business life cycle.

In the Requirements Analysis stage, special care is taken to ensure that the proposed system meets the management's objectives. The key requirements include:

- Customer profile and policy management
- Sales tracker

- Policy performance tracker
- Weekly and monthly sales reports
- Renewals status
- Customer service module to resolve customer issues

➔ Designing Architecture

- Considering the scope of the application, Michael has to provide a detailed design of the scope and flow of the application.
- This requires designing DFDs, flowcharts, defining the architecture and layout, deciding on the security policies and mechanisms, defining the access roles to the software, and the database design of the application.

➔ Building and Testing

- In this phase, Michael has to guide a team of developers to code the software and build it on the initial framework that was designed in the design phase.
- At the same time, Michael also has to guide the third party testing vendor to write test cases to validate the developed software.
- Once the software is coded, it is tested and necessary bug fixes are done.
- The software also undergoes unit testing, integration testing, and system testing.

➔ Operations and Maintenance

- In this phase, the software is ready to be installed.
- Before that sufficient training is provided to the end-user team with demonstrations and their feedback is incorporated.
- Once the team is trained and comfortable with the new software, all the existing customer details are migrated manually into the system as a one-time activity.
- Any critical issues during this process are addressed and fixed immediately.

2.6 Check Your Progress

1. SDLC is _____.

(A)	A process that consists of a sequence of activities or phases that govern the entire life cycle of a software development
(B)	A process consists of a sequence of testing stages that govern the testing of the components
(C)	An activity to detect defects in the developed software product
(D)	A framework to design the architecture of the system

2. Arrange the phases of SDLC followed in sequence for ant software product.

(A)	Designing Architecture
(B)	Planning and Requirement Analysis
(C)	Defining Requirements
(D)	Building or Construction
(E)	Deployment and Maintenance
(F)	Testing

(A)	C, D, B, F, A, and E	(C)	D, A, B, C, E, and F
(B)	B, C, A, D, F, and E	(D)	E, F, D, C, B, and A

3. Which of the following statements are true regarding system analysis?

(A)	Identification and analysis of customer needs
(B)	Allocating functions to manpower, database, hardware, software, and other system elements
(C)	Establishing cost and schedule constraints
(D)	Designing the architectural design

(A)	B, C, and D	(C)	A, B, and C
(B)	A, C, and D	(D)	Only D

4. ____ is an iterative process during which requirements are translated into an outline for constructing the software.

(A)	Planning	(C)	Designing
(B)	Analysis	(D)	Testing

5. Identify some standard activities to be performed during the implementation phase.

(A)	Creation of an installation plan
(B)	Conducting user training
(C)	Logical designs
(D)	Maintenance of the system

(A)	B and C	(C)	C and D
(B)	A and C	(D)	A and B

2.6.1 Answers

1.	A
2.	B
3.	C
4.	C
5.	D



Summary

- Software Development Life Cycle (SDLC) is a process applied to the development of a software product. This process consists of a sequence of activities or phases that govern the entire life cycle of a software development.
- A typical SDLC consists of the phases namely, Planning and Requirement Analysis, Defining Requirements, Designing Architecture, Building or Construction, Testing, and Deployment and Maintenance.
- System Analysis study normally conducts a feasibility study and based on its result, the translation phase of inputs into formal specified requirements is performed.
- Design can be defined as the process of applying various techniques and principles to define a device, process or system in sufficient detail to permit its physical realization.
- Implementation Planning is an important activity of the SDLC. The specific approach adopted for implementation is largely dependent on the size of organization, the nature of applications and the standard practices that may have been evolved by the project team.
- Testing is very important in SDLC as the quality of the product depends on this phase. The efficiency of the testing phase can make or break a software product.
- Maintenance activities involve making enhancements to the software products developed in the earlier stages of the life cycle, adapting products to new environments, and correcting problems.

Onlinevarsity

your e-way to learning

GROWTH

RESEARCH

OBSERVATION

UPDATES

PARTICIPATION



www.onlinevarsity.com

Session - 3

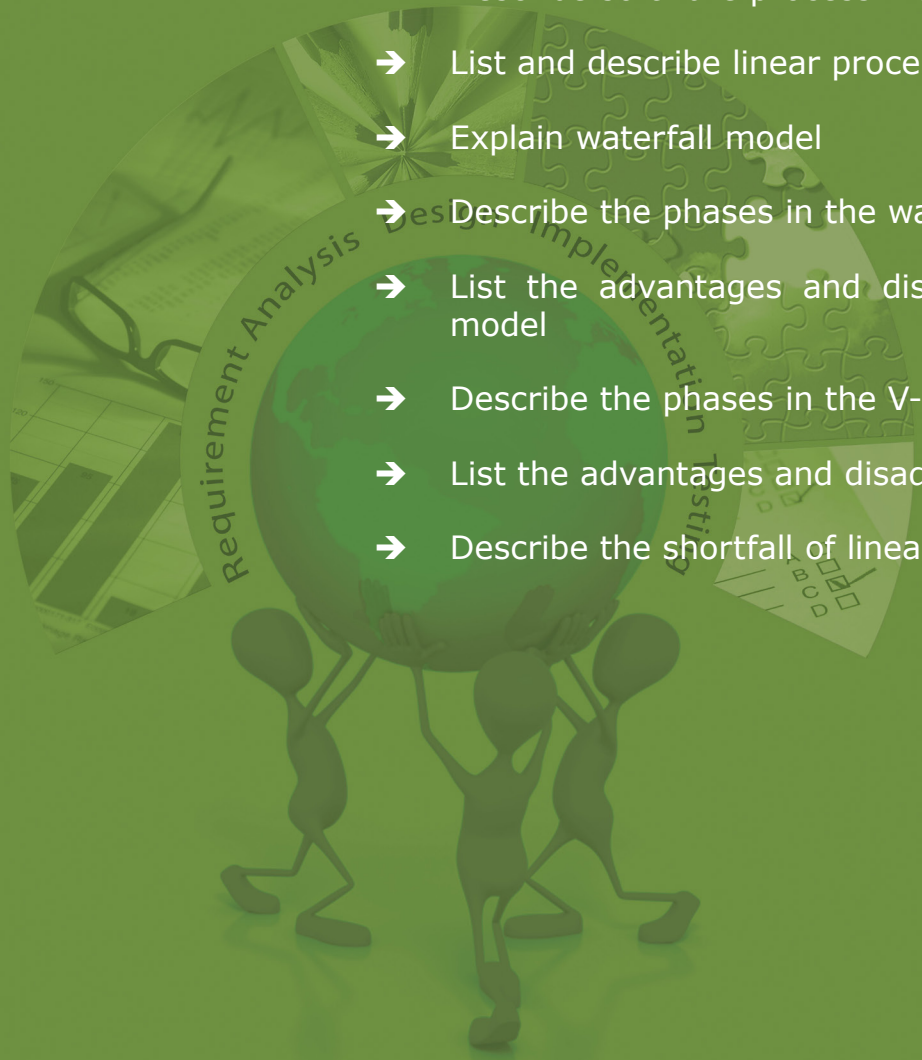
Linear Process Models

Welcome to the Session, **Linear Process Models**.

This session describes the software process models and their types used for software development. The session describes the linear process model. The session explains the waterfall model and describes its development phases. The session also lists the advantages and disadvantages of waterfall model. Further, the session describes the V-model and lists its pros and cons. Finally, the session describes the drawbacks of linear process model.

In this Session, you will learn to:

- Describe software process models
- List and describe linear process models
- Explain waterfall model
- Describe the phases in the waterfall model
- List the advantages and disadvantages of the waterfall model
- Describe the phases in the V-model
- List the advantages and disadvantages of the V-model
- Describe the shortfall of linear models



3.1 Introduction

The basic difference that makes software different from other products is that it is not a physical entity. Software exists as a logical entity and that starts all the complications.

How does one develop software? How does one manufacture the product designs? We only need to ask a software manager to set up a factory to 'design his products' and watch his reactions to understand the problems he has to face in software product design.

Software Engineering essentially consists of a set of steps that comprises methods, tools, and procedures that assist a software manager to manufacture a software. Depending on the type of software that we plan to develop at our factory, whether it is an office automation product, which is going to be sold in millions or flight simulation software of which only one copy will be made, one needs to identify the approach that is to be used.

All software development organizations will have to carry out SDLC activities, if they have to develop a product. The sequence and detailed level of these activities will depend on the nature of the project and application. Based on product generalizations, various software process models or paradigms are available.

3.1.1 Software Process Models

A process is a standardized approach to plan, organize, and execute a software development project. Software process models provide a way in which we can organize our software development into a series of activities to ensure the completion, acceptance, or appreciation of our software product by our customer.

Like any successful product, the development effort will have to start from an insight into the customer requirements. These requirements have to be clearly communicated to the personnel involved in developing the product.

Some amount of 'original thinking' will have to go into designing a solution that will meet the requirements. The design will have to be converted into software that may be bought, and code that may be developed internally. The bits and pieces developed separately will have to be integrated; the final product tested and delivered to the customer. The customer will have to be educated about the product, and if required, the product will have to be modified and maintained.

The process you adopt depends on the software being built. One process might be good for a particular software, while an entirely different process would be used for another software.

The use of process models also makes it easier to predict the budget and to review the progress at the end of each phase.

The process models prevalent in the software industry can be broadly categorized as:

- Linear Process Models
- Evolutionary Process Models

Concepts

3.2 Linear Process Model

Linear process models, also referred to as traditional models, have been used since early times in software development. In this approach, the process of software development is represented by a sequence of activities or phases. Each phase must be completed, before you can move to next phase.

Figure 3.1 shows the linear process model.

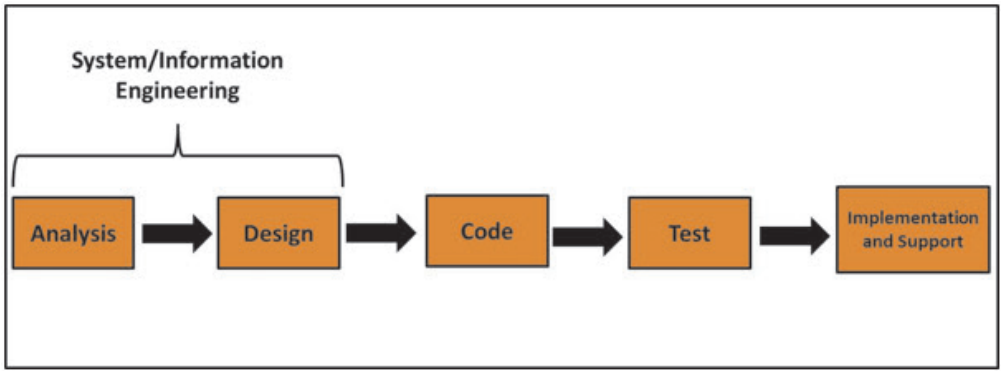


Figure 3.1: Linear Process Model

As shown in the figure, the linear process model is a systematic sequential approach to software development that begins at the system level and progresses through the phases of analysis, design, coding, testing, implementation, and support. The sequential phases make this model linear, simple, and systematic.

The linear models are built on the assumption that the software to be developed can be completely understood and described before a solution is designed. A design that satisfies all aspects of the problem can be specified before implementation. All implementation can be done before validation and delivery.

Tight control is maintained over the life of the project with the help of extensive written documentation, formal reviews, and approval/sign-off by the management and customer at the end of every phase.

3.2.1 Advantages of the Linear Process Models

The advantages of the linear sequential model are as follows:

- ➔ It is easy to understand and implement.
- ➔ It allows for a clear demarcation between two phases and a clear hand-off from one phase to the next.
- ➔ It ensures a proper forward-looking approach to software development.
- ➔ It emphasizes on requirements, before designing the solution.
- ➔ It has Milestone reviews which encourage close scrutiny of phase exit and entry criteria.

- Its sequential progression through phases readily maps to configuration control points and the establishment of baselines.

- The proposed model structure must be clearly understood and communicated to all stakeholders.

3.3 Waterfall Model

The waterfall model, sometimes called the linear sequential model or 'classic life cycle' is illustrated in figure 3.2.

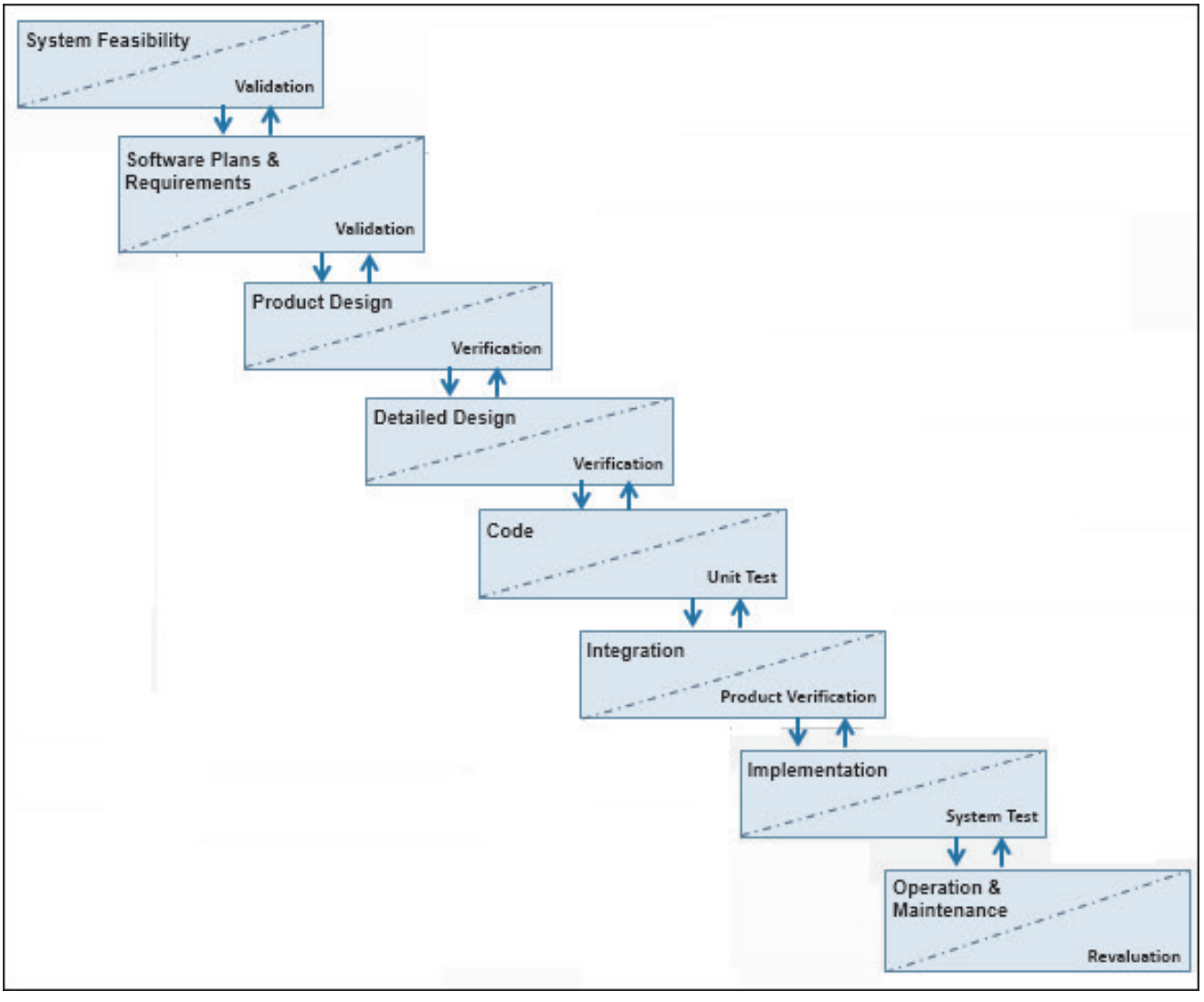


Figure 3.2: Waterfall Model

V 1.0 © Aptech Limited

This model suggests a systematic, sequential approach to software development that begins at the system level and progresses through analysis, design, coding, testing, and maintenance.

➔ System/Information Engineering and Modeling

Software being a part of a larger system (or business), the development process has to first establish the requirements for the entire proposed system, and then identify a part of it to be delivered by the software component. This system view is essential when software must interface with other elements such as hardware, people, and databases. System engineering and analysis encompasses requirements gathering at the system level, with a small amount of top-level analysis and design. A feasibility of the proposed system would have to take place at this stage.

➔ Software Requirements Analysis

The requirements gathering process is intensified and focused specifically on software. To understand the nature of the program(s) to be built, the software engineer ('analyst') must understand the information domain for the software, as well as the required function, behavior, performance, and interfacing. Requirements for both the system and the software are documented and reviewed with the customer.

➔ Software Design

It is a multi-step process, focusing on four distinct attributes of a program: data structure, software architecture, interface representations, and procedural (algorithmic) detail. The design process translates requirements into a representation of the software that can be assessed for quality before code generation begins. Like requirements, the design is documented and becomes part of the software configuration.

➔ Detailed Design

During detailed design, the internal logic of each module specified in system design is decided. During this phase, further details of data structures and algorithmic design of each of the modules is specified. The logic of a module is usually specified in a high-level design description language, which is independent of the target language in which the software will eventually be implemented.

➔ Program Design Language

PDL is one way in which the design can be communicated precisely and completely to whatever degree of detail desired by the designer. It can be used to specify the system design and to include the logic design.

➔ Coding

The coding phase translates the design of the system, produced during the design phase, into code in a given programming language. This can be executed by a computer that performs the computation specified by the design. The coding phase affects both the testing and maintenance phase and aims to implement the design in the best possible manner.

During implementation of this phase, it is essential to keep in mind that the code being written is not only easy to write, but also easy to read and maintain.

All design considerations contain hierarchies. This is a natural way to manage complexity. Coding can either be top-down or bottom-up. In top-down approach, the main module is implemented first, then their subordinates, and so on. In this, implementation starts from the top of the hierarchy and proceeds to the lower level. In a bottom-up implementation, the modules at the bottom of the hierarchy are implemented first, and then the implementation proceeds through the higher levels until it reaches the top.

The advantage of this type of implementation is that the problem is tackled in steps rather than building the whole system in one go. It helps because it is not feasible or desirable to build the whole system in one attempt and then test it. It is better to build parts of the system and test them before putting them together to form the system.

➔ Testing

Once code has been generated, program testing begins. The testing process focuses on the logical internals of the software, assuring that all statements have been tested, and on the functional externals, that is, conducting tests to uncover errors and ensure that defined input will produce actual results that agree with required results. The testing activity is carried out in two phases, the components before integration, and then the integrated product.

➔ Integration

Developing a strategy for integration of the components into a functioning whole requires careful planning so that the modules are available for integration when needed. The integrated product can then be tested to demonstrate that the implemented software meets the requirements stated in the requirements document.

➔ Operations and Maintenance

Software will undoubtedly undergo change after it is delivered to the customer (a possible exception is embedded software). Change will occur because errors have been encountered; the software must be adapted to accommodate changes in its external environment (for example, a change required because of a new operating system or peripheral device) or because the customer requires functional or performance enhancements. Software maintenance reapplies each of the preceding phases to an existing program rather than a new one.

Each phase in this model ends in a verification and validation activity, the objective of which is to eliminate as many problems as possible in the products of that phase. The linear sequential model is the oldest and the most widely used paradigm for software engineering. For some time the Waterfall model provided personnel in the software field a sequence of common anchor points, a set of milestones around which people can plan, organize, monitor, and control their projects. Many companies, government organizations, and standard groups established interlocking regulations, specifications, and standards based on the model.

3.3.1 Application of Waterfall Model

Every software developed is different and requires a suitable SDLC approach to be followed based on internal and external factors. Some situations where the use of waterfall model is most appropriate are as follows:

- The project requirements are clear, fixed, and very well documented.
- The problem definition is stable.
- There are sufficient resources available with required experience to support the project development.

3.3.2 Waterfall Model Benefits and Drawbacks

The main advantage of waterfall development approach is that the management can set the timelines for each phase of development. Also, development is completed phase by phase.

Some of the advantages are as follows:

1. Simple and easy to understand.
2. Easy to arrange tasks and clearly defined stages.
3. Requirement should be clear before going to next phases.
4. Each phase of development has well-defined milestones.
5. Works well for smaller projects.

Some of the disadvantage of waterfall development are as follows:

1. Users can judge quality of the ready product only at the end of the development life cycle.
2. Changes required at any phase may end the project development.
3. Involves high level of risk and uncertainty in a project as each phase is closely linked with the earlier phase.
4. It follows the 'Big-bang' approach where the entire software product is integrated at the end in the system. This may sometimes lead to challenges that may not be identified in the beginning of the project development.

3.4 V-Model

V-Model is an extension to the waterfall model. This model associates each phase with a corresponding testing phase. This is also a highly disciplined model and each phase starts only after completion of the previous phase. The V-Model is also known as the Verification and Validation model.

➔ **Validation** - The assurance that a product, service, or system meets the needs of the customer and other identified stakeholders. It often involves acceptance and suitability with external customers. Contrast with verification.

➔ **Verification** - The evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition. It is often an internal process. Contrast with validation.

It is said that validation can be expressed by the query "Are you building the right thing?" and verification by "Are you building it right?".

Figure 3.3 shows the V-model.

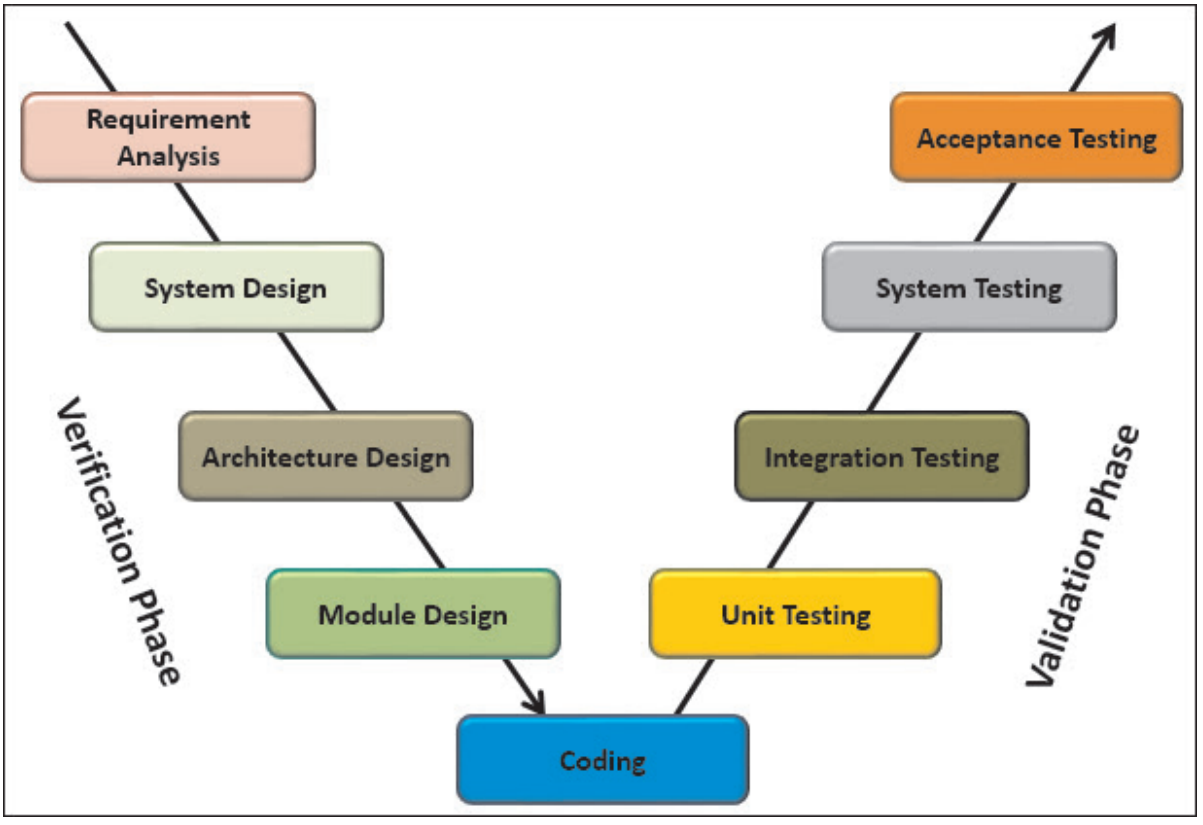


Figure 3.3: V-Model

The left side of the 'V' represents the decomposition of requirements and creation of system specifications. The right side of the 'V' represents integration of parts and their validation. Testing is emphasized more than in the waterfall model. The testing procedures are developed early in the life cycle before any coding is done, during each of the phases preceding implementation. Requirements begin the life cycle model just similar to the waterfall model. A system test plan is created before any development is started and targets to meet the functionality specified in requirement gathering.

The main focus of the high-level design phase is system architecture and design. In this phase, an integration test plan is created to test the pieces of the software system's ability to work together. However, the actual software components are designed in the low-level design phase. Unit tests are also

Concepts

created in this phase.

The implementation phase is where all the coding takes place. Once the coding is complete, the path of execution continues to progress up the right side of the V where the test plans developed earlier are put to use.

3.4.1 Verification and Validation Phases

Verification and validation consists of four phases that represents the system design with the corresponding validations performed on the designed system.

- **Requirements Analysis**: The requirements of the proposed system are collected by analyzing the requirements of the users. The user requirements document typically describes the system's functional, interface, performance, data, security, and other requirements as expected by the user. It is used by business analysts to communicate their understanding of the system to the users. The users carefully review this document as it would serve as the guideline for the system designers in the system design phase. The user acceptance tests are also designed in this phase. In this phase, the customers decide if they want to accept the system or not.

- **System and Architecture Design**: These phases are also referred to as high-level design phases. They provide entire design of the system including hardware, services, software, and so on. A system test plain created which validates the system design. System testing compares the system specifications against the actual system. Similarly, an integration plan is created to test the software integrity within the system. Black box testing is usually done in this stage as the code is not directly checked for errors.

- **Module Design**: This phase is also referred to as the low-level design phase. The designed system is broken up into smaller units or modules and each of them is described so that the programmer can start coding directly.

  The low-level design document or program specifications contain a detailed functional logic of the module. The unit test design is developed in this stage to test component functionality. The purpose of unit testing is to verify the internal logic of the code by testing every possible branch within the function. This is also known as test coverage.

- **Coding Phase**: The actual coding of the system modules designed in the design phase is taken up in the coding phase. The programming language suitable for the project is decided based on architectural requirements. The developed code goes through numerous code reviews and is optimized for best performance before the final build is added into the repository.

3.4.2 Advantages and Disadvantages of V-Model

The advantages and disadvantages of the V-model are depicted in table 3.1.

Advantages	Disadvantages
Simple and easy to understand.	Very rigid similar to the waterfall model.
Each phase has specific deliverables.	Little flexibility and adjusting scope is difficult and expensive.
Higher chance of success over the waterfall model due to the early development of test plans during the life cycle.	Software is developed during the implementation phase, so no early prototypes of the software are produced.
Suitable for those projects which have clear requirements.	This Model does not provide a clear path for problems found during testing phases

Table 3.1: Advantages and Disadvantages of V-Model

3.5 Shortfalls of Linear Models

For a small period in the mid-1970's, it appeared that the software field had found a sequence of common anchor points: a set of milestones around which people could plan, organize, monitor, and control their projects. These were the milestones in the waterfall model, typically including the completion of system requirements, software requirements, preliminary design, detailed design, code, unit test, software acceptance test, and system acceptance test. Unfortunately, an initial design will likely be flawed with respect to its key requirements. Late discovery of design defects results in costly over-runs and in some cases even project cancellation.

The ideal of a software acceptance test keyed to demonstrating compliance to a set of documented requirements specifications encountered problems as well. Foremost among these was that static tests of input-output transformations provided little insight on whether the software would acceptably support common user task. The acceptance test also failed to cover many off-nominal problem situations subsequently encountered in beta-testing, field-testing, hardware-software integration, or in actual mission operations.

3.6 Check Your Progress

1. Which of the following phase of waterfall model performs a feasibility study of the proposed system?

(A)	System/Information engineering and modeling	(C)	Coding
(B)	Software design	(D)	Integration

2. Arrange the following phases in the waterfall model in the correct sequence.

Waterfall Model	
1.	System and Information Engineering
2.	Operations and Maintenance
3.	Software Requirement Analysis
4.	Coding
5.	Designing
6.	Testing and Integration

(A)	2, 1, 5, 3, 4, 6	(C)	1, 3, 5, 4, 6, 2
(B)	1, 2, 3, 4, 5, 6	(D)	6, 5, 4, 3, 2, 1

3. Which of the following phases in V-Model is referred to as the low-level design phase?

(A)	Architectural Design	(C)	System Design
(B)	Module Design	(D)	Business Requirement Analysis

4. Which of the following statements are correct for linear process models?

(A)	It does not provides a clear demarcation between two phases and a clear hand-off from one phase to the next.
(B)	It has Milestone reviews which encourage close scrutiny of phase exit and entry criteria.
(C)	The proposed model structure must be clearly understood and communicated to all stakeholders.
(D)	It emphasizes on requirements, before designing the solution.

(A)	A, B, and C	(C)	A, C, and D
(B)	B, C, and D	(D)	C and D

5. Match the following validation phases with the corresponding verification phases.

Validation Phase		Verification Phase	
1.	Unit Testing	a.	Architectural Design
2.	Integration Testing	b.	Business Requirement Analysis
3.	System Testing	c.	Module Design
4.	Acceptance Testing	d.	System Design

(A)	1-a, 2-b, 3-c, 4-d	(C)	1-b, 2-a, 3-d, 4-c
(B)	1-c, 2-d, 3-a, 4-b	(D)	1-c, 2-a, 3-d, 4-b

Concepts

3.6.1 Answers

1.	A
2.	C
3.	B
4.	B
5.	D



Summary

- Software process models provide a way in which we can organize our software development into a series of activities to ensure the completion, acceptance, or appreciation of our software product by our customer.
- The process models prevalent in the software industry can be broadly categorized as Linear Process Models and Evolutionary Process Models.
- In Linear process model, software development is represented by a sequence of activities or phases.
- Waterfall model suggests a systematic, sequential approach to software development that begins at the system level and progresses through analysis, design, coding, testing, and maintenance.
- V-Model is also a highly disciplined model and each phase starts only after completion of the previous phase.
- The V-Model is also known as the Verification and Validation model.
- The ideal of a software acceptance test keyed to demonstrating compliance to a set of documented requirements specifications encountered problems as well.

Session - 4

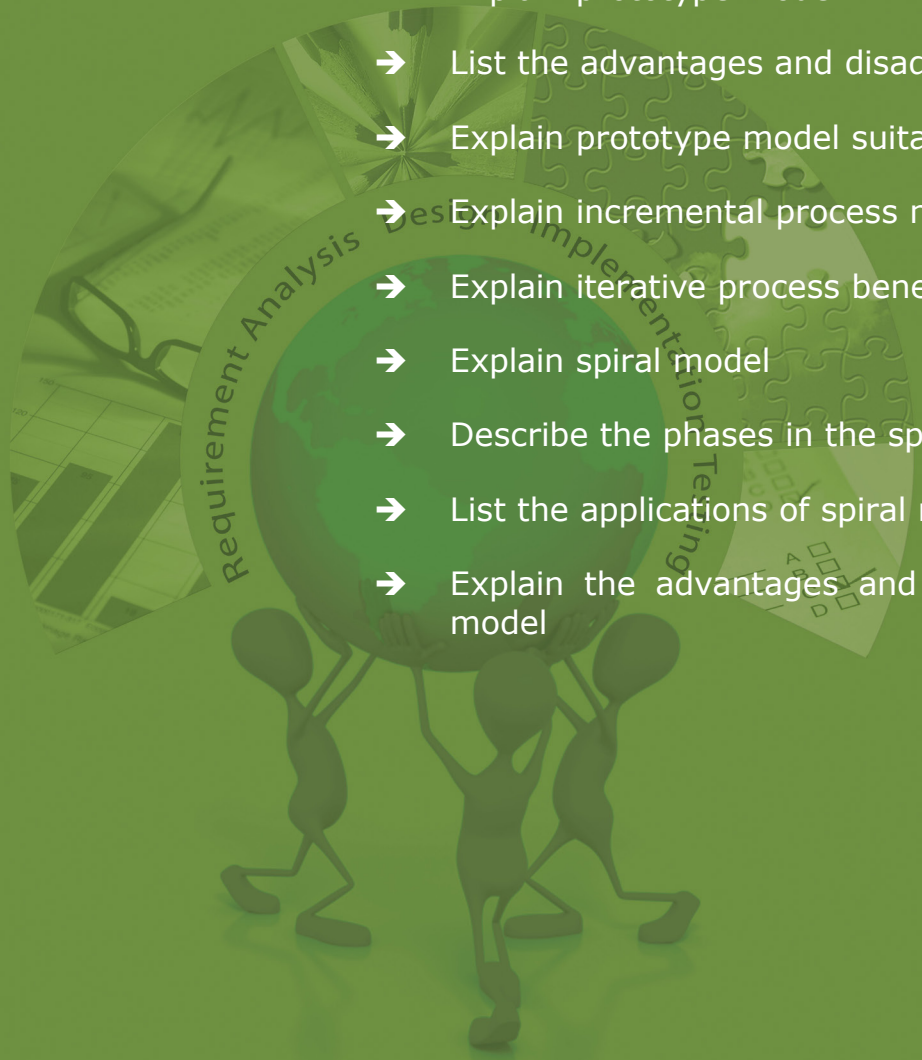
Iterative Process Models

Welcome to the Session, **Iterative Process Models**.

This session explains the iterative process model. The session begins with prototype model and explains the factors which can be used to decide the use of prototype model in the software development. Further, the session explains evolutionary process model which is based on iterative approach and its benefits. Finally, the session explains the spiral model with its advantages in software development.

In this Session, you will learn to:

- Explain prototype model
- List the advantages and disadvantages of prototyping
- Explain prototype model suitability in project development
- Explain incremental process model and its approach
- Explain iterative process benefits and drawbacks
- Explain spiral model
- Describe the phases in the spiral model
- List the applications of spiral model
- Explain the advantages and disadvantages of the spiral model



4.1 Introduction

In the Waterfall model, the software development team has to gather all the requirements in the beginning, complete the design, coding, and testing in succession and deliver the product, all at once. The risk of failure is quite enormous in such a case, as defects results in costly over-runs and in some cases even project cancellation.

The drawbacks of the Waterfall model resulted in an iterative process models which ensured faster results, with less information in advance and much more flexible.

4.1.1 Prototyping Model

Prototyping was developed to overcome the limitations of the Waterfall model. In this model, a prototype is created based on the existing requirements. Prototypes are similar to models or simulation of the real thing and may display the functionality of the product under development, but may not actually hold the exact logic of the final software.

One of the common software development model is software prototyping, as it assists to know the customer requirements at an initial stage of software product development.

It also helps to get valuable feedback from the customer and enables software designers and developers learn about what is expected from the desired product.

Organizations such as Microsoft, Disney, and Boeing follow prototyping, which enables innovation, better communication with the customers and internal people, development and improvement of the final product. The various phases of prototyping model are depicted in figure 4.1.

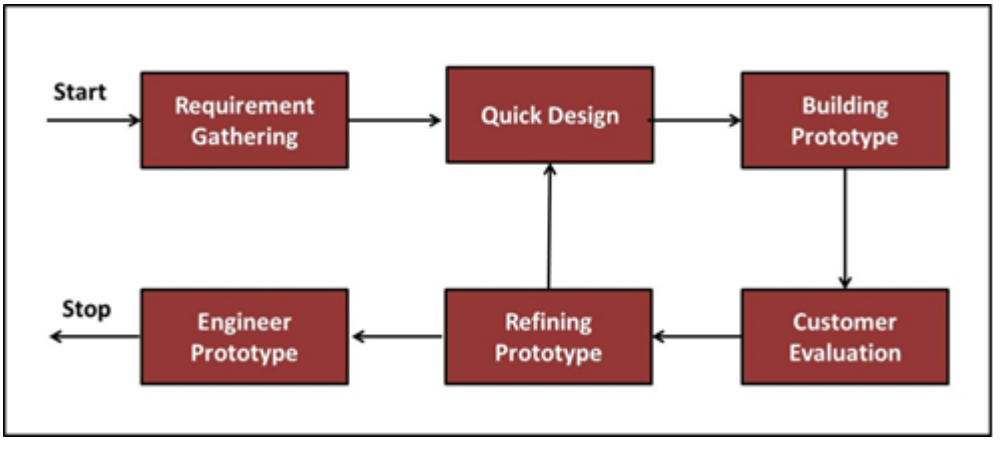


Figure 4.1: Prototyping Model

4.1.2 Steps to Design a Software Prototype

The approach taken to design a software prototype is as follows:

➔ **Identification of Basic Requirements**: This step identifies the very basic product requirements, especially in terms of user interface. Extensive details such as the internal design and external aspects such as performance and security are not considered in this stage.

Concepts

- **Development of the Initial Prototype**: The development of the initial prototype happens in this stage. Here, the very basic requirements are illustrated and user interfaces are designed. These features may not exactly work in the same manner internally in the final software product. The workarounds in the prototype give the same look and feel to the final product.

- **Review of the Prototype**: In this stage, the developed prototype is presented to the customer and the other important stakeholders in the project for their review. The feedback is collected and documented in a systematic way and is used for further improvements to products under development.

- **Revision and Enhancement of the Prototype**: The feedback and the review comments from the customer are evaluated during this stage. Factors such as time, budget constraints, and technical feasibility of the actual implementation are negotiated with the customer. The accepted changes are incorporated into the new prototype developed and the cycle repeats until customer expectations are met.

Prototypes can have vertical or horizontal dimensions. A vertical prototype is a detailed explanation of a specific function or a sub system in the product. Vertical prototypes are technical in nature and are used to get details of the exact operation of the sub systems.

Horizontal prototype, on the other hand, displays the user interface for the product and gives a wider view of the entire system, without focusing on the internal functions. Horizontal prototypes are used to get more information at the user interface level and the business requirements. It may even be used in sales demos to get business in the market.

4.1.3 Applications of Prototype Model

The prototype model is used for projects with the following criteria:

- Requires a lot of interaction with the end users. Generally, online systems and Web interfaces interact with the end users. The prototype model is best suited for such projects.

- To design human-computer interface systems. It is very important to take end user feedback and include it, as they are the ultimate users of the product. Prototyping helps in implementing this. Software that involves too much of data processing and very little user interface does not usually benefit from prototyping.

4.1.4 Deciding Whether to Prototype or Not

Each systems development effort has its own scope, requirements, complexity, skill-set, development time, users, management, levels of innovativeness, and organizational culture.

Prototyping may be useful or create problems to the development process. Based on the requirement, it should be employed. Here are few situations where prototyping affects the outcome of a system under development.

Table 4.1 lists the situations and corresponding prototype model considerations.

Situation	Reason to Consider Prototyping
Users are uncomfortable with hypothetical models.	Gives users a prototype to interact with the model, before actual product.
Project has a long development time.	Gives users and developers something to work in the early phase of the development cycle.
Requirements are uncertain.	Provides opportunities to rework on requirements.
A complete new system with no earlier history.	Provides chances to do multiple trials, detect and correct errors.
Product development needs the help of tools such as simulation, experimentation, or progressive assessment.	Allows for simulation, experimentation, and incremental evaluation.
A critical system is needed immediately.	Prototyping tools are designed for quick implementation. Can begin requirements gathering soon.
Users are available.	Allows for high user participation.

Table 4.1: Prototype Model Consideration

4.1.5 Advantages and Disadvantages of Prototype Model

Some of the advantages of prototype model are as follows:

- There is active involvement of users in the project.
- The working prototype model gives a better understanding of the desired product.
- The problems that can arise during the development are detected earlier.
- The users feedback helps in providing better solutions.
- The complex modules are identified and a quick implementation can be provided for them.

Even though the benefits of prototyping are evident, there are disadvantages and potential risks associated with it. Excessive change requests are one of the major concerns. Users tend to request changes repeatedly, just because of the nature of the iterative process. As they re-examine the prototype at each stage, chances are that they may think of new features that are beyond the scope of the original project. However, this may be controlled with proper planning; it is still a major concern.

4.2 Evolutionary Process Model

There is a growing recognition that software, like all complex systems, evolves over a period of time. Business and product requirements often change as development proceeds, making a straight line path to an end product unrealistic; tight market deadlines make completion of a comprehensive software product impossible, but a limited version must be introduced to meet competitive or business pressure; a set of core product or system requirements is well understood, but the details of product or system extensions have yet to be defined. In these and similar situations, software engineers need a process model that has been explicitly designed to accommodate a product that evolves over time.

The linear sequential model is designed for straight-line development. In essence, this Waterfall approach assumes that a complete system will be delivered after the linear sequence is completed. The prototyping model is designed to assist the customer (or developer) in understanding requirements. In general, it is not designed to deliver a production system. The evolutionary nature of software is not considered in this classic software engineering paradigms.

Evolutionary models are iterative. They are characterized in a manner that enables software engineers to develop increasingly more complete versions of the software. The basic difference between the traditional Waterfall model and the iterative process is in how the project tasks are contained in the plan. In the Waterfall model, they are functionally contained, while in the iterative model, they are time-based.

4.2.1 Iterative Approach

Unlike the Waterfall model, the iterative model does not start with all the requirements upfront. Development begins by determining and implementing a part of the software project, which is then reviewed to identify further requirements. The process is then repeated and a new version of the software is created for each cycle of the model.

In an iterative process, a portion of the overall requirements is considered for implementation. Successive iterations of the implementation helps in software upgradation and evolution of different versions until the final product is implemented and deployed. Such a process helps to identify results in an early stage and also obtain timely feedback from the users. Every iteration acts like a mini Waterfall model and the feedback from one phase is fed as an input to the next phase.

At the end of each iteration working software is delivered which is fed to the next iteration.

The key to successful use of an iterative software development life cycle is correct validation of requirements, verification, and testing of each version of the software against the requirements in each cycle of the model.

The iterative model is as shown in figure 4.2.

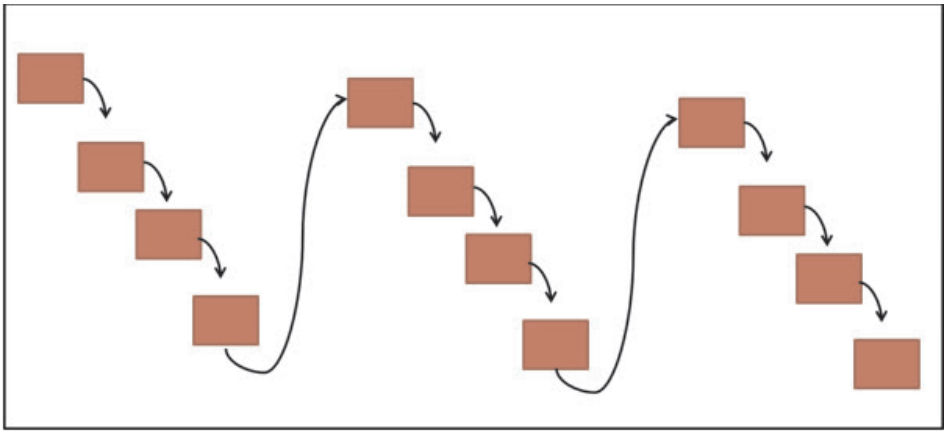


Figure 4.2: Iterative Software Development

The main features of iterative development are as follows:

- Time-boxed phases
- Test early, test often
- Deliver early, deliver often
- Production quality

4.2.2 Benefits of Iterative Approach

An iterative approach is generally superior to a linear or Waterfall approach for many different reasons:

- **It tolerates changing requirements** - The reality is that normally requirements change. Requirement change and requirements 'creep' have always been a primary source of project trouble, leading to late delivery, missed schedules, unsatisfied customers, and frustrated developers.
- **Elements are integrated progressively** - Integration is not one 'big bang' at the end. The iterative approach is almost a continuous integration. What used to be a long uncertain and troublesome time, taking up to 40% of the total effort at the end of a project, is now broken into six to nine smaller integrations that begin with fewer elements?
- **Risks are mitigated earlier** - Integration is generally the only time risk factors are discovered or addressed. As we unroll the early iterations we go through all core process workflows, exercising many aspects of the project: tools, off-the-shelf software, people skills, and so on. Some perceived risks may prove not to be risks at all, and new, unsuspected risks may emerge.

- **It allows the organization to learn and improve** - Team members are able to learn along the way, and the various competencies and specialties are more fully employed during the whole life cycle. Testers start testing early, technical writers write early, and so on. In non-iterative development, the same people will wait to begin their work, make plans, and hone their skills. Training needs or the need for additional (perhaps external) help is spotted early on during assessment reviews.

- **It facilitates reuse** - Design reviews in early iterations, allows architects to identify unsuspected potential reuse, and develop and mature common code in subsequent iterations.

- **It results in a more robust product** - As we are correcting errors over several iterations. Flaws are detected even in the early iterations as the product moves beyond inception. Performance bottlenecks are discovered when they can still be addressed, instead of on the eve of delivery. The outcome is higher level of quality.

- **It tolerates tactical changes in the product** - For example, to compete with existing products. We can decide to release a product with reduced functionality earlier to counter a move by a competitor, or we can adopt another vendor for a given technology.

- **The process itself can be improved and refined along the way** - The assessment at the end of an iteration not only looks at the status of the project from a product and schedule perspective, but also analyzes what should be changed in the organization, and in the process to perform better in the next iteration.

  A customer once said: “With the Waterfall approach, everything looks fine until near

  the end of the project, sometimes up until the middle of integration. Then everything

  falls apart. With the iterative approach, it is very difficult to hide the truth for very long”. Unfortunately, project managers often resist the iterative approach, seeing it as a kind of endless ’hacking’.

- **Accommodates changes** - It allows us to take into account changing requirements. Users will change their mind along the way. This is part of human nature. Forcing the users to accept the system as they originally imagine it is wrong. They change their mind because the context changes, they learn more about the environment, the technology, and they see intermediate demonstration of the product, as it is being developed.

- **It allows technological changes on the way** - If the technology changes or it becomes a standard, or new technology appears, the project can take advantage of it. This is particularly true for platform changes or lower-level infrastructure changes.

- **Increasing reuse** - It facilitates reuse, because it is easier to identify common parts as they are partially designed or implemented, compared to identifying all commonality up front. It is difficult to identify and develop reusable parts.

- Design reviews in early iterations allow architects to identify unsuspected, potential reuse, and subsequent iterations allow us to develop and mature this common code.

- **It is easier to take advantage of Commercial-Off-the-Shelf (COTS) products** - We have several iterations to select them, integrate them, and validate that these fit into the architecture.

- **Learning** - The developers can learn along the way, and the various competencies and specialties are fully employed during the whole life cycle. Testers and technical writers can start testing and documentation work earlier, rather than waiting for a long time, and simply making plans and honing skills. The need for additional training or external help can be detected in the early iteration assessment reviews.

  The process itself can be improved and refined as it develops. The assessment at the end of iteration not only looks at the status of the project from a product-schedule perspective, but also analyzes what should be changed in the organization, and the process to perform better in the next iteration.

- **It results in a more thoroughly tested product** - The critical functions have had many opportunities to be tested over several iterations. The tests themselves and any test software will have had time to mature, as opposed to tests run once towards the end of the project.

4.2.3 Problems with Iterative Approach

The critical milestone in evolutionary development is the initial release: a package of software with sufficient capability to serve as a basis for user exercise, evaluation, and evolutionary improvement.

However, this 'initial release' milestone frequently has the following problems:

- **Inflexible point-solutions**

  Frequently, the initial release is optimized for initial demonstration-mode and exploratory mode success. For example, it may store everything in main memory to provide rapid response time. Then, when users want to transition to large-scale use, the initial point solution architecture will not scale up.

- **High-risk downstream capabilities**

  Frequently, the initial release will defer such considerations as security, fault-tolerance, and distributed processing in the interest of providing early functionality and user interface capabilities. The users may like the results, and expect the deferred considerations to be delivered equally rapidly. Usually, this puts the project in big trouble because the initial release's architecture cannot be easily extended to support the desired security, fault tolerance, distributed processing, or other key considerations.

➔ **Off-target initial release**

Evolutionary developers often begin by saying, "Let us find out what the user needs by building an initial release and seeing what the users want improved." The lack of initial user activity analysis frequently leads to an initial release, which is so far from user needs that the users never bother learning and using it.

4.3 Spiral Model

These difficulties with the Waterfall and evolutionary development models have led to the development and use of a number of alternative process models: risk-driven (spiral), reuse-driven, legacy-driven, demonstration-driven design to cost or schedule, incremental, and hybrid combinations of these with the Waterfall or evolutionary development models.

Boehm originally proposed the Spiral model in 1988 as an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the linear sequential model. This was observed as an attempt to combine the strengths of the various models. It provides the potential for rapid development of incremental versions of the software. The model also takes into account results of risk assessment activities whose outcome determines taking up of the next phase of the development activity.

This model views software development as a spiral process with the software being developed in a series of incremental releases. Typically, the inner cycles represent the early phases of requirement analysis along with prototyping to refine the requirement definitions, and the outer spirals are progressively representative of the classic software life cycle. The spiral model is divided into a number of framework activities, also called task regions. Typically, there are between three and six **task regions** as shown in figure 4.3.

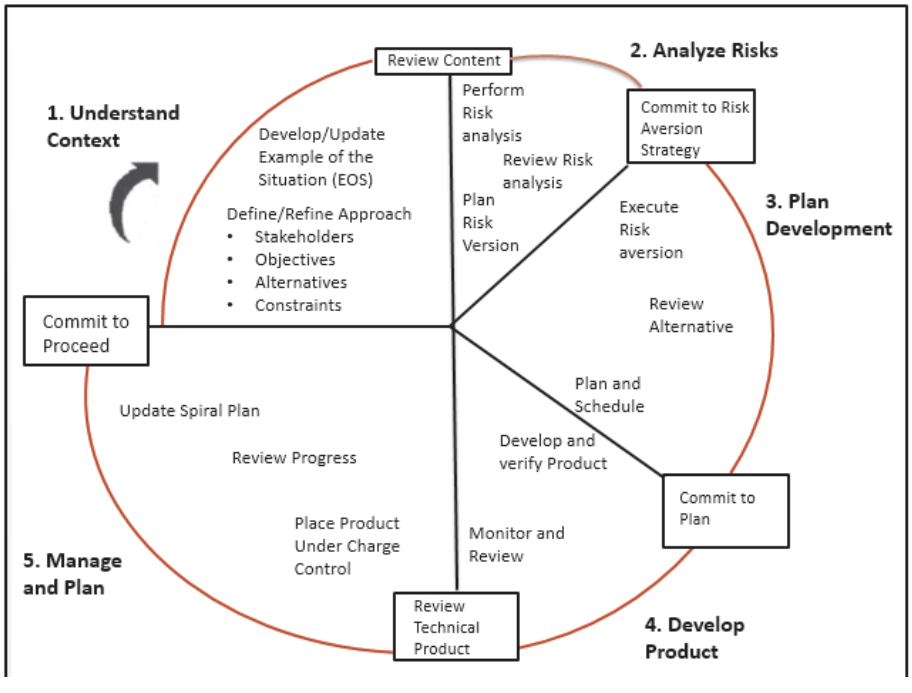


Figure 4.3: Spiral Model

As shown in figure 4.3 depicts a spiral model that contains six task regions are as follows:

- **Understanding the Context-** These tasks are required to establish effective communication between developer and customer.
- **Planning-tasks** - They define resources, timelines, and other project related information.
- **Risk analysis-tasks** - These are required to assess both technical and management risks.
- **Engineering-tasks** - These are required to build one or more representations of the application.
- **Construction and release**-These tasks construct, test, install, and provide user support (for example, documentation and training).
- **Customer Evaluation-tasks** – These are required to obtain customer feedback based on evaluation of the software representations created during the engineering stage, and implemented during the installation stage.

Each of the defined six regions has a series of work tasks. These tasks are dependent on the type of project being undertaken. For small projects, the number of work tasks and their formality is low. For larger, more critical projects, each task region contains more work tasks that are defined to achieve a higher level of formality. In addition, all support activities like Configuration Management, Project Management have to happen.

As this evolutionary process begins, the software engineering team moves around the spiral in a clockwise direction, beginning at the core. The first circuit around the spiral may result in the development of a product specification; subsequent passes around the spiral may be used to develop a prototype, and then progressively more sophisticated versions of the software. Each pass through the planning region results in adjustments to the project plan. Cost and schedule are adjusted based on feedback derived from customer evaluation. In addition, the project manager adjusts the planned number of iterations required to complete the software.

4.3.1 Advantages and Disadvantages of Spiral Model

The spiral model is a realistic approach to the development of large-scale systems and software. Since software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level. The spiral model uses prototyping as a risk reduction mechanism. It also enables the developer to apply the prototyping approach at any stage in the evolution of the product.

It maintains the systematic stepwise approach suggested by the classic life cycle, but incorporates it into an iterative framework that more realistically reflects the real word. The spiral model demands a direct consideration of technical risks at all stages of the project, and if properly applied, should reduce risks before they become problematic.

However, like other paradigms, the spiral model is not a solution to all the problems faced by software developers. It may be difficult to convince customers (particularly in contract situations) that the evolutionary approach is controllable. It demands considerable risk assessment expertise, and relies on this expertise for success. If major risks are not uncovered and managed, problems will undoubtedly occur.

4.3.2 Application of Spiral Model

Spiral model is very widely used in the software industry. It is chosen for projects possessing the following criteria:

- ➔ Medium to high-risk projects.
- ➔ Long-term project commitment due to changes in economic priorities as requirements change with time.
- ➔ Customer is not sure of their requirements.
- ➔ Complex requirements, which further require clarification.
- ➔ New product line, which needs to be released in phases to get enough customer feedback.
- ➔ Major changes are expected in the product during the development life cycle.

4.4 Check Your Progress

1. Which of the following is not a feature of the iterative model?

(A)	Time-boxed	(C)	Results are obtained periodically
(B)	Functionality-boxed	(D)	Supports changing requirements

2. Which of the following are the true regarding the benefits of iterative approach?

(A)	Tolerates changing requirement
(B)	Risks are identified late
(C)	Facilitates reuse
(D)	Allows technological changes on the way

(A)	A, B, and C	(C)	B and C
(B)	A, C, and D	(D)	Only B

3. Arrange the six task regions in the spiral model in the order in which they happen.

(A)	Understanding the Context
(B)	Risk analysis-tasks
(C)	Construction and release
(D)	Customer Evaluation-tasks
(E)	Planning-tasks
(F)	Engineering-tasks

(A)	A, E, B, F, C, D	(C)	B, C, E, A, F, D
(B)	D, C, A, E, F, B	(D)	C, D, B, E, F, A

4. Evolutionary models are _____.

(A)	Sequential	(C)	Parallel
(B)	Linear	(D)	Iterative

5. The prototype model is not designed to deliver a _____.

(A)	Production system	(C)	Implemented system
(B)	Prototype system	(D)	Developed system

4.4.1 Answers

1.	B
2.	B
3.	A
4.	D
5.	A



Summary

- The drawbacks of the Waterfall model resulted in an iterative process models which ensured faster results, with less information in advance, and much more flexible.
- Prototypes are similar to models or simulation of the real thing and may display the functionality of the product under development.
- In prototype, excessive change requests are one of the major concerns. Users tend to request changes repeatedly, just because of the nature of the iterative process.
- Evolutionary models are iterative. They enables software engineers to develop increasingly more complete versions of the software.
- The key to successful use of an iterative software development life cycle is correct validation of requirements, verification, and testing of each version of the software against the requirements in each cycle of the model.
- The Spiral model is as an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the linear sequential model.
- The spiral model is not a solution to all the problems faced by software developers. It may be difficult to convince customers (particularly in contract situations) that the evolutionary approach is controllable.

Onlinevarsity
your e-way to learning

WRITE-UPS BY

EXPERTS AND **LEARNERS**

TO PROMOTE NEW AVENUES AND
ENHANCE THE LEARNING EXPERIENCE



FOR FURTHER READING, LOGIN TO

www.onlinevarsity.com

Session - 5

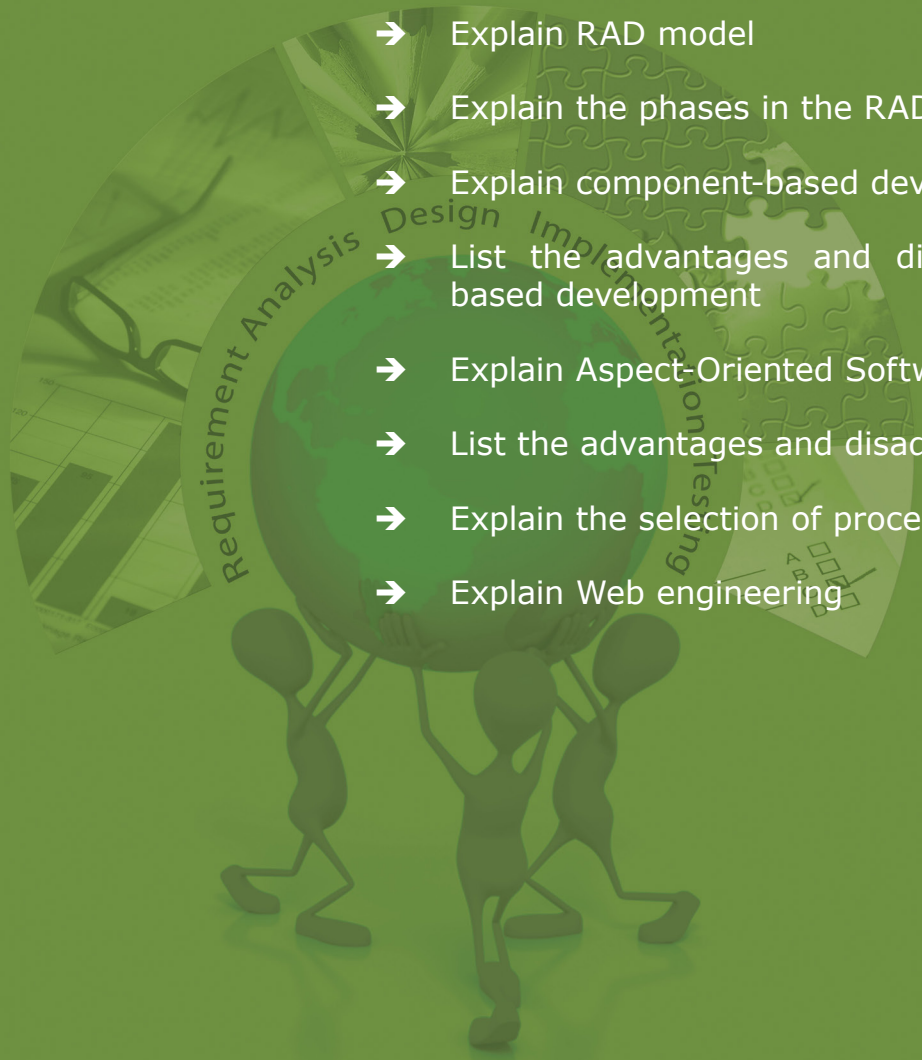
Specialized Process Models

Welcome to the Session, **Specialized Process Models**.

This session explains the Rapid Application Development (RAD) that is used to deliver project in prototypes. The session explains component-based development and gives the advantages and disadvantages in the software development. Further, the session explains Aspect-Oriented Software Development (ASOD) along with its advantages and drawbacks. The session also explains choosing a process model. Finally, the session concludes with the Web engineering.

In this Session, you will learn to:

- Explain RAD model
- Explain the phases in the RAD model
- Explain component-based development
- List the advantages and disadvantages of component-based development
- Explain Aspect-Oriented Software Development (AOSD)
- List the advantages and disadvantages of AOSD
- Explain the selection of process model
- Explain Web engineering



5.1 Introduction

Traditional methods of software engineering are considered to be process-heavy and cumbersome. Large, complex, and slowly evolving software require procedure-centric, bureaucratic methods of development and handling. It take years together to complete the project.

Software engineering provides parallel process model that allow building different components in an incremental model. One such type of process model is Rapid Application Development (RAD). RAD model is flexible and adaptable to changes as it employs short development cycles.

5.2 Rapid Application Development (RAD)

RAD is a software development methodology that uses very little planning. The concepts of parallelism is used in this model, several prototypes representing functional modules are developed in parallel, which are later combined to get the final product. This process aids in quicker product delivery.

This process employs minimal planning and hence is able to accommodate changes. It uses iterative and incremental development strategies. Small dedicated teams work progressively on their modules and develop reusable prototypes.

It involves user participation thereby increasing chances of early user community acceptance and an overall reduction in project risk. The result is high quality software in the least possible time duration and with higher customer satisfaction.

The RAD process has three main objectives to meet:

- ➔ Timely and speedy delivery of a working application
- ➔ Helping the customer to identify the requirements properly
- ➔ Meet the customer's requirements with accuracy using prototypes and iterations

5.2.1 RAD Phases

The RAD life cycle includes all the activities such as analysis, design, construct, and test in the series of cycles that promote short and iterative development.

Figure 5.1 shows the RAD phases in software development.

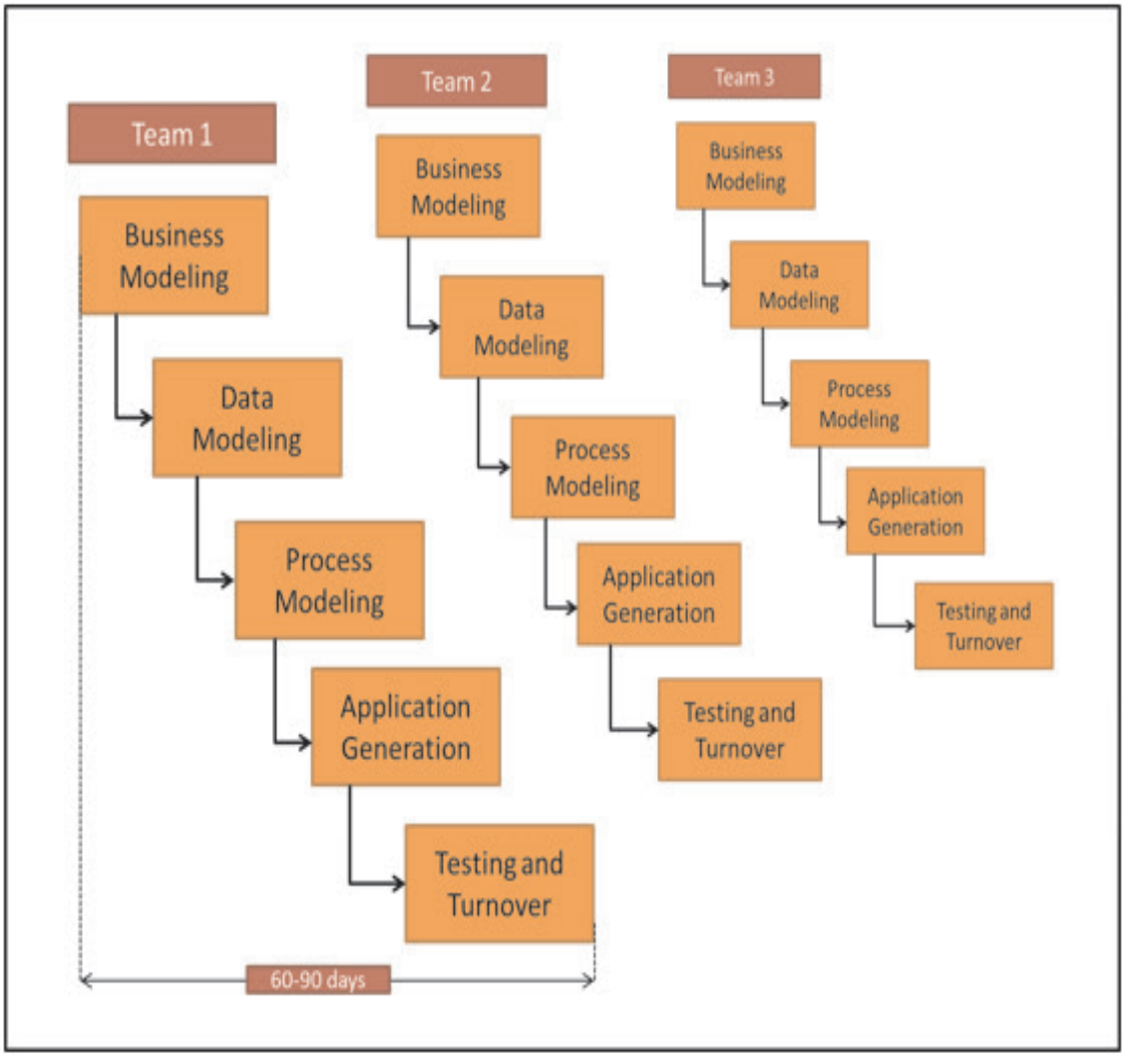


Figure 5.1: RAD Phases

The phases in the RAD model are as follows:

- **Business Modeling** - In this phase, a business model of the developed product is designed. The design model identifies the flow of information between the various functions of the business. This plays an important role in business analysis.

- **Data Modeling** – In this phase, the information gathered during the business analysis is reviewed in terms of the data objects involved in the business. For example, in a banking application development, the valid data objects will be information related to customers, accounts, and so on. The data objects are represented with a set of attributes and also their relationship with other data objects is identified.

Figure 5.2 shows the data modeling for banking.

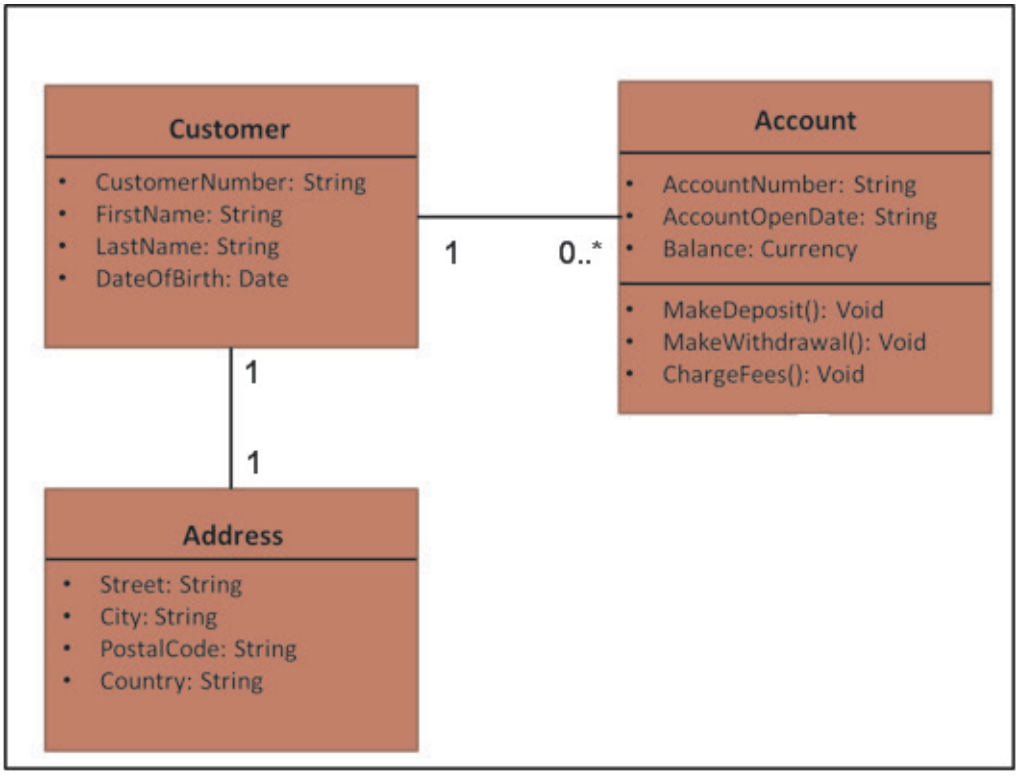


Figure 5.2: Data Modelling

➔ **Process Modeling** – In this phase, the identified data objects are implemented to achieve indentified information flow in the business. The detailed descriptions are provided on how operations such as insertion, deletion, or updation is performed on a data object.

➔ **Application Generation** – In this phase, actual prototypes of actual systems are built by providing implementation for identified process and data models.

➔ **Testing and Turnover** – As each prototype is independently tested in RAD model, so the total time taken to test overall system is reduced. This may also reduces the major problems that may be identified when the final project is tested.

5.2.2 Applications of RAD Model

RAD model can be chosen for projects with the following criteria:

➔ When there is a need to develop system as a set of working modules that are delivered in increments.

➔ Resources such as modeling designers, high budget, and automated code generating tools are available.

V 1.0 © Aptech Limited

- Skilled manpower with strong business knowledge is available.

5.2.3 Advantages and Disadvantages of RAD Software Development

Some of the advantages of RAD model are as follows:

- As the project is delivered in small increments, so the time taken to build the project is reduced.
- The project manager can estimate project costs accurately.
- Reusability of prototypes saves cost in terms of project budget and project time.
- Components ready for the repository are already tested and need not be tested again.
- Requirements are gathered dynamically and a suitable prototype is developed. Any additional requirement is addressed in the next version of the prototype.
- There is a strong participation by the customer who gives feedback on a continuous basis.

The disadvantages of RAD are as follows:

- Higher dependency on technical skills to identify business requirements.
- Not suitable for projects with low cost, as generation of automated code is high.
- Highly skilled manpower is required for such projects.
- Rapid development sometimes compromises quality parameters such as standardization, reliability, and consistency leading to issues later on.
- Promotes component-based development which can be scaled as per the business needs.

5.3 Component-Based Development

Component-based development is a variation on general application development, in which:

- The application is built from discrete executable components, which are developed and deployed relatively independently of one another, potentially by different teams.
- The application may be upgraded in smaller increments by upgrading only some of the components that comprise the application.

- Components may be shared between applications, creating opportunities for reuse, but also creating inter-project dependencies.

One of the various ways of approaching software development is trying to model the real world as closely as possible. The object-oriented paradigm that is used to model real world situations as a series of objects emphasizes the creation of classes that encapsulate both data and the algorithms that are used to manipulate the data.

Object technologies provide the technical framework for a component-based process model for software engineering. If properly designed and implemented, object-oriented classes are reusable across different applications and computer-based system architectures. The component assembly model incorporates many of the characteristics of the spiral model. It is evolutionary in nature, demanding an interactive approach to the creation of software. However, the component assembly model comprises applications from prepackaged software components, sometimes called 'classes'.

The engineering activity begins with the identification of candidate classes. The data that are to be manipulated by the application and the algorithms that will be applied to accomplish the manipulation are examined to identify the required components. Corresponding data and algorithms are packaged into a class.

Once candidate classes are identified, the class library is searched to determine if these classes already exist. If they do, they are extracted from the library and reused. If a candidate class does not reside in the library, it is engineered using object-oriented methods. The first iteration of the application to be built is then composed, using classes extracted from the library and any new classes built to meet the unique needs of the application. Process flow then returns to the spiral, and will ultimately re-enter the component assembly iteration during subsequent passes through the engineering activity.

The component assembly model leads to software reuse and reusability provides the software engineers with a number of measurable benefits.

Based on reusability studies, reports indicate that component assembly leads to a 70% reduction in development cycle time, a 84% reduction in project cost, and a productivity increase of 26.2, compared to an industry norm of 16.9. Although, these results are a function of the robustness of the component library, there is little question that the component assembly model provides significant advantages for software engineers.

Architectures that are commonly used in BD are as follows:

- ActiveX
- Common Object Request Broker Architecture (CORBA)
- Remote Method Invocation (RMI)
- Simple Object Access Protocol (SOAP)

5.3.1 Advantages of Component-based Development

➔ **General advantages**:

- **Flexible**: A runtime component can work independently and are less dependent on the software environment.
- **Reusable**: A developed component can be reused across programming languages and operating systems.
- **Easy to maintain**: Components are easier to test and maintain as their functionality is implemented only once.
- **Cost and time-efficient**: Lower development cost and lesser time to develop.

➔ **Improving business processes**:

It is easier to reconfigure component-based software to match changes in the business process. The changes can be implemented in the following ways:

- **Simplification**: Involves removing unnecessary and redundant steps, and reducing variety in processes. This can be reflected in the software by replacing components and improving the application step-by-step.
- **Integration**: Requires combining unconnected processes into a larger coordinated process. This can be done by including additional components in the software and linking them to the existing components.
- **Transformation**: The entire business process is overhauled in this case. Components are disassembled and put together in a different way to transform the way a business process is carried out.

➔ **Improving software**:

- Greater functionality can be delivered faster by using already existing components.
- Replacing individual components can be replaced easily making maintenance easier.
- A single graphical user interface can be used for standard components.
- Performance can be increased on a component-to-component basis, by optimizing the performance within each component. This can improve the overall efficiency. As components can be made to suit any platform, their usability is very high.
- If an application can be broken down into components such that each component can completely implement the requirement for which it is designated, then the overall software's reliability is considered to be excellent.
- Components are portable and can quickly adjusted to work in different platforms without affecting other components.

5.3.2 Disadvantages of Component-based Development

- Building the environment that fits the components is challenging.
- There are not sufficient standards governing middleware in which components work.
- Comparison between the available standards is not possible, as only CORBA is language-independent.
- It may not always be possible to find the best fit components for the requirements specification.

5.4 Aspect-Oriented Software Development

New software development methodologies have been created to solve problems that have become larger and more complex over the years. Even though object-oriented methods have allowed better designing of systems, there are some shortcomings in these methods which led to more research.

Aspect-Oriented Software Development (AOSD) is the result of the research efforts of the Xerox Palo Alto Research Center.

AOSD is a new method of developing software applications which involves breaking down a software systems into components known as modules, so as to separate secondary functions from the main program's business logic. The focus of AOSD is on identification, specification, and representation of cross-cutting concerns and their modularization into separate functional units and combining them back automatically into a working system. Cross-cutting concerns are those changes that cannot be isolated in any particular class and have to be in-corporated at many places across the system.

5.4.1 AOSD Concepts and Terminology

- **Concern**: A concern is any demand, requirement, or expectation on a software system by any stakeholder of that system. A stakeholder may be the developer, the end-users, or even the system itself.
- **Cross-cutting Concern**: A cross-cutting concern is a concern which deals with different areas of the software system.
- **Aspect**: An aspect is a module that contains a concern. Aspect-oriented programming provides good support for localizing concerns into aspects.
- **Join point**: A join point is a well-defined point in the program's execution where an aspect is invoked.
- **Pointcut**: A pointcut is a specifically defined subset of join points.

➔ **Advice body**: An advice body is code that is executed when a join point is reached.

➔ **Weavers**: Weavers are specific type of compilers that compose the aspect's implementation with the other modules.

5.4.2 Advantages and Disadvantages of AOSD

The advantages and disadvantages of AOSD are summarized in table 5.1.

Advantages	Disadvantages
Increased modularity.	Reduced efficiency due to increased overhead aspects.
Increased maintainability.	Security risks involving aspects.
Increased reusability.	Lack of a formalized process.
Reduction in the size of the code.	Lack of UML support. Lack of systematic methods of testing.

Table 5.1: Advantages and Disadvantages of AOSD

5.5 Choosing a Process Model

Software development organizations are increasingly realizing that a core set of tasks need to be done irrespective of the process model they follow. Depending on the time frame available or execution of project, the type of product/project being developed, the detail in which specifications are available, the organizations having previous experience with similar projects, and so on, these tasks are organized in differing sequences.

Organizations choose the process paradigm they want to follow in developing the software. In case different processes are followed, outputs in terms of productivity and quality also vary. Processes tend to become less predictable and past data cannot be used for estimation, or new projects learn from the failures of old projects. Standardized processes are essential for predictability, good estimation capability, and good learning.

Organizations may find it difficult to follow a single standard process for all its projects. One approach is to define a standard process that describes a sequence of activities or a task, but allows for tailoring to suit a particular project. Guidelines for tailoring are provided, and the types of processes that can be followed are restricted to the allowable variations.

5.5.1 Process Tailoring

Tailoring is defined as the process of adjusting the standard process of an organization to obtain a process that is suitable for the particular business or technical needs of a project. Documented guidelines are in many cases provided on the reason of deviation and to what extent deviations are possible from the standard processes so as to achieve the project goals.

Process modification requires that the unique aspects of the project are identified first, and then these features are used to make adjustments to the standard process. Tailoring guidelines try to capture the experience and judgment of people regarding how tailoring can be done effectively. The past experience of people who have used the process becomes available so that even a newer person, who may not have the necessary experience, can select the optimum process for a project.

The process may be tailored with respect to its scope, formality, frequency, and granularity. For each process element that can be tailored, one needs to define the options that are available and the circumstances in which each of these options can be exercised.

Tailoring can happen at a macro (Summary- How certain general activities are performed) level or at a micro level (Detailed – all activities in a process for the various phases and how they can be tailored). Some of the factors that may need to be considered for tailoring the process include the skill level of the team, the peak team size, and the criticality of the application.

5.6 Web Engineering

Web Engineering is an adaptable and incremental (evolutionary) process. It is populated by a set of framework activities that occur for all business-critical WebApp projects, regardless of the size or complexity.

The following framework activities may be considered for Web engineering:

- **Formulation** - Identifies the goals and objectives of the WebApp and establishes the scope for the first increment.
- **Planning** - Estimates overall project cost, evaluates risks, and defines a development schedule for the initial WebApp increment.
- **Analysis** - Establishes requirements for the WebApp and identifies the content that will be incorporated.
- **Modeling** - Incorporates two parallel tasks sequences: content and user interface design. Content design involves the creation or acquisition of content for the WebApp. User interface design involves defining the architecture, navigation, and graphic design, which establish the structure of the WebApp and the flow of the user's interaction with it.
- **Page generation and testing** - Constitute requisite construction activities.
- **Customer evaluation** - Reviews each increment produced as part of the Web engineering process. At this point, changes are requested (scope extensions occur). These changes are integrated into the next path through the incremental process flow.

5.7 Check Your Progress

1. Which of the following phase in RAD gathers data objects involved in the business?

(A)	Business modeling	(C)	Information modeling
(B)	Data modeling	(D)	Application generation

2. The _____ paradigm that is used to model real world situations as a series of objects.

(A)	Object-oriented	(C)	Web-based
(B)	Structured	(D)	Modular

3. Which of the following process model is adopted in Web engineering?

(A)	Linear	(C)	Adaptive and Incremental
(B)	Component-based	(D)	Parallel

4. A _____ is a module that contains a concern.

(A)	Aspect	(C)	Point cut
(B)	Join point	(D)	Weaver

5. An aspect weaver is a _____.

(A)	Aspect language for writing aspects
(B)	General purpose programming language
(C)	Pre-processor coded in the general purpose programming language
(D)	Module that contains a concern

5.7.1 Answers

1.	B
2.	A
3.	C
4.	A
5.	C



Summary

- RAD is a software development methodology that uses very little planning. The concepts of parallelism is used in this model, several prototypes representing functional modules are developed in parallel for quicker product delivery.

- The phases in RAD are namely, Business modeling, data modeling, process modeling, application generation, and testing and turnover.

- Object technologies provide the technical framework for a component-based process model for software engineering.

- The component assembly model leads to software reuse and reusability provides the software engineers with a number of measurable benefits.

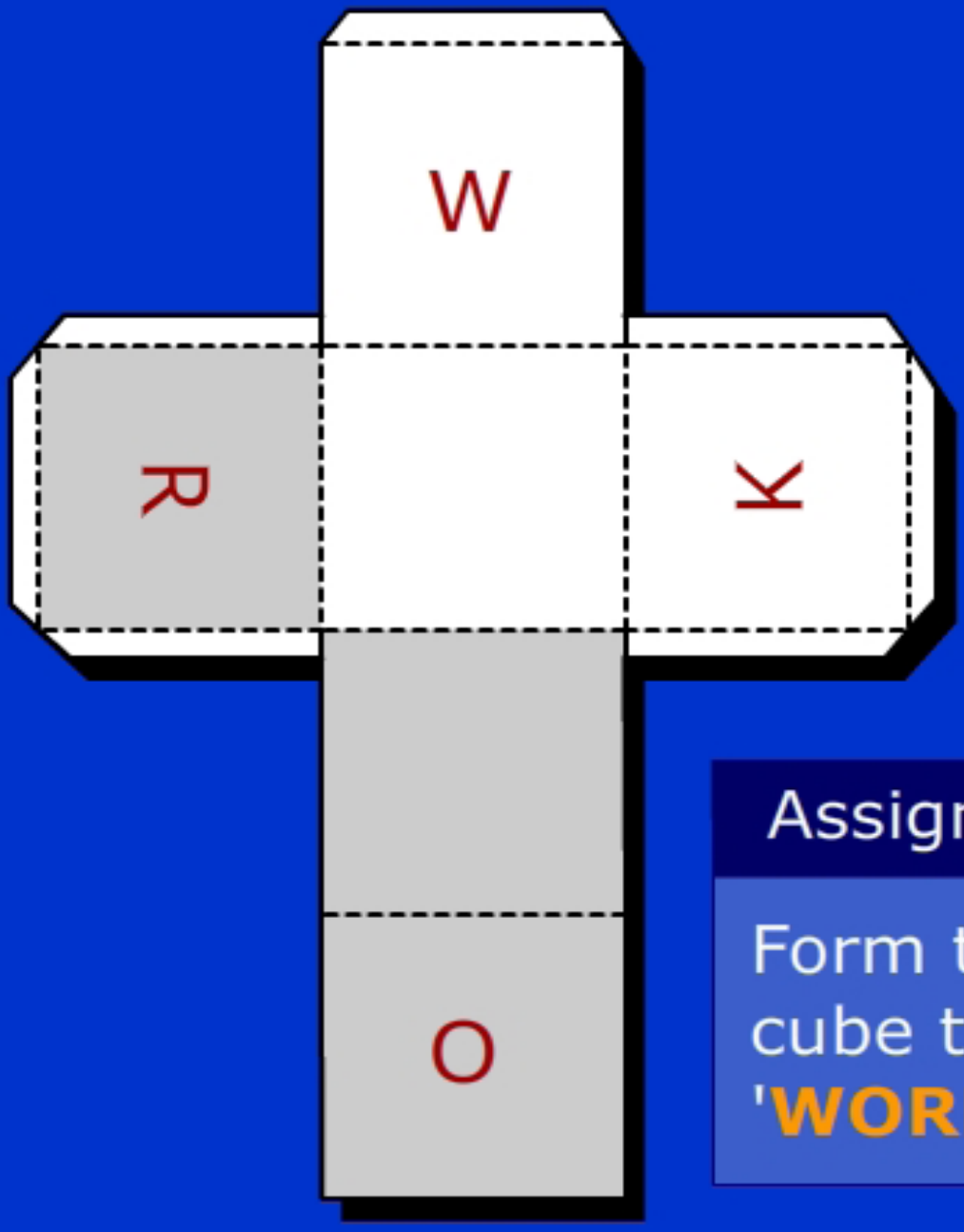
- AOSD is a new method of developing software applications which involves breaking down a software systems into components known as modules, so as to separate secondary functions from the main program's business logic.

- The focus of AOSD is on identification, specification, and representation of cross-cutting concerns and their modularization into separate functional units and combining them back automatically into a working system.

- Web Engineering is an adaptable and incremental (evolutionary) process. It is populated by a set of framework activities that occur for all business-critical WebApp projects, regardless of the size or complexity.

WORK ASSIGNMENT

Onlinevarsity
your e-way to learning



Assignment

Form the cube to read '**WORK**'.

"Practice does not make perfect. Only perfect practice makes perfect."
- Vince Lombardi

Practice the Practicals for Perfection @

www.onlinevarsity.com

Session - 6

Agile Software Development - I

Welcome to the Session, **Agile Software Development - I**.

This session explains the basics of Agile development. The session explains the Agile SDLC approach and compares it with traditional models. Further, the session describes the values provided by Agile manifesto and lists the principles of Agile. Finally, the session explains different Agile methodologies and presents a case study to understand the Agile approach in the software development.

In this Session, you will learn to:

- Describe Agile development and its evolution
- Explain Agile SDLC
- Differentiate between Agile and other traditional SDLC models
- Explain the values behind the Agile manifesto
- Describe the key features of Agile software development
- Describe the most popular Agile software development methodologies
- List the applications of Agile software development
- Describe a case study on Agile



6.1 Introduction

The traditional software process models, such as waterfall model assumes that each requirement of the project must be identified before the actual design and implementation of the project begins. Failure of clear understanding of requirements means ending up developing a piece of software that has less relevance with the actual need of the customer. For example, designing a weak user interface with less interactivity may add no value to the business needs.

Consider if all the project requirements are clearly understood by the team at the beginning of the project, but the time taken to create the software project is increased, may make the final product irrelevant, due to changing business needs. Such a scenario leads to a company spending time and money on a project which is of no use to it.

Thus, to overcome some of the constraints seen in traditional process models, Agile development methods are followed by businesses today. Agile is the solution for all the eager business communities asking for a lightweight, fast, and easy to understand development processes. Some of the fast growing businesses, such as mobile app development industry and Web-based applications accessed on Internet, whose needs are changing drastically can follow Agile approach to reduce both development cost and time.

In the words of Goldman (1995), agility in business terms is defined as: "Agility is dynamic, context-specific, aggressively change-embracing, and growth-oriented". It is not about improving efficiency, cutting costs, or battening down the business hatches to ride out fearsome competitive 'storms.' It is about succeeding and about winning i.e. about succeeding in emerging competitive areas, and about winning profits, market share, and customers in the very centre of the competitive storms many companies now fear.

Agile in software development terminology is a style of adapting process model that combines philosophy and a set of development guidelines to ensure fast delivery of software with more simplicity in overall development of the project.

6.2 Evolution of Agile

Agile methodologies are made up of several management, customer, and engineering practices, many of which have been around for decades. These practices put together, guides teams through the process of rapidly planning and delivering finished software products.

Even though many of the practices associated with Agile development have been around for years now, the average software development team yet to embrace many of its principles and practices. The average software team does not iterate, does not deliver software incrementally, neither does it practice continuous planning nor automate testing. During the last several years, this trend seems to be rapidly changing, now the Agile practices have been combined in a way that can more easily be understood and adopted.

Agile development methodologies provide opportunities to continuously assess the direction of a project throughout the development life cycle. This is accomplished through periodic iterations, at the end of which teams must have a product increment ready. Agile methodology is described as 'iterative' and 'incremental' in view of the repeated short work cycles and the functional product they yield. When using the waterfall model, development teams have only one chance to get each aspect of a project right.

In an Agile paradigm, each aspect of development is continually revisited throughout the life cycle. The team stops and re-evaluates the direction of a project periodically, and if required changes the direction.

The 'inspect-and-adapt' approach in Agile development reduces to a great extent both development costs and time to market. With this model, as teams can develop software and gather requirements simultaneously, the 'analysis paralysis' phenomenon is less likely to deter a team from advancing. Also, because the work cycle is limited to two weeks, it gives stakeholders repeated opportunities to improve the releases. Agile methodology empowers teams to continuously reassess their release to optimize its value throughout development, thereby allowing them to be as competitive as possible in the marketplace. Using Agile methodology, one protects a product's critical market relevance and also ensures a team's work never goes unreleased.

6.3 Agile SDLC

Figure 6.1 shows the Agile SDLC approach in software development.

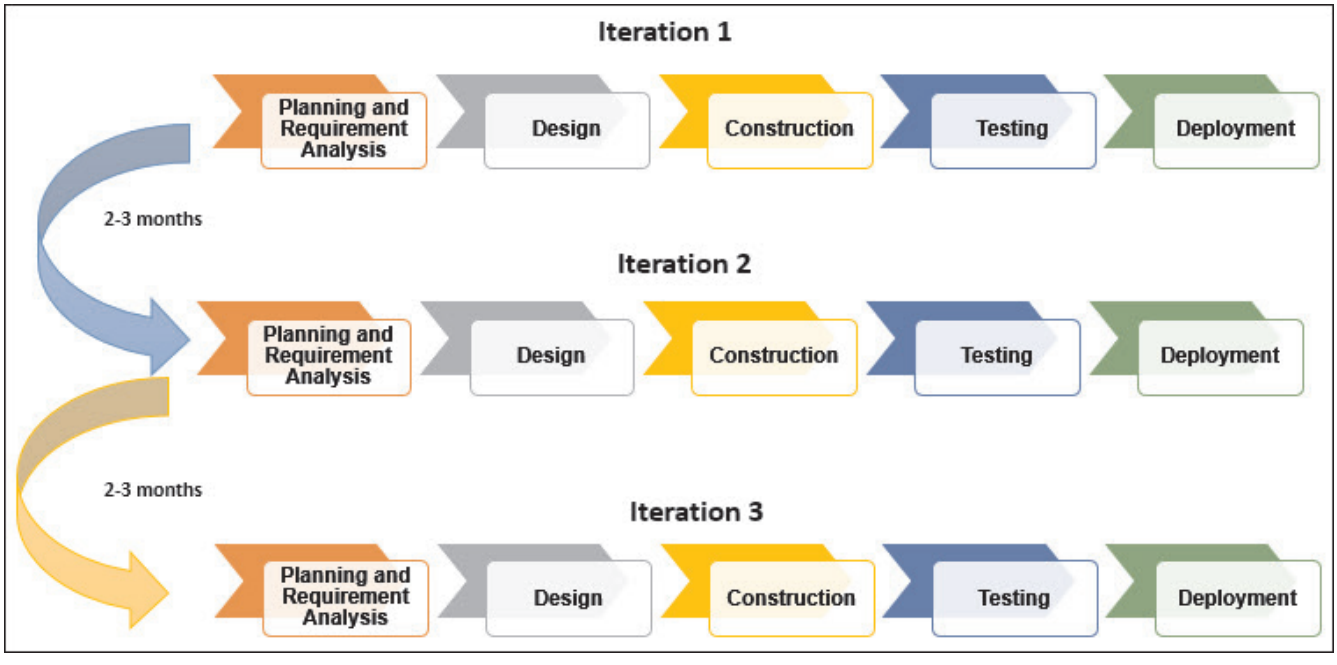


Figure 6.1: Agile SDLC

Agile SDLC development follows the principle of increments and iterations to deliver software products. This method breaks tasks into small increments with minimal planning. Iterations are of short time frames known as time-boxes which are of certain duration.

Iterative approach is taken to build software that can be delivered in each iteration. Each iteration adds or increments the software with the features laid by the customer.

The final product created by following Agile SDLC approach provided all the working features in the project.

6.3.1 Key Features of Agile SDLC

The key features of Agile software development are:

- **Iterative**: The entire application is divided into incremental units, each of which is called an 'iteration'. Each iteration is a mini-increment of the functionality and is built on top of the previous iteration. The development time of each of the iterations is small, fixed, and strictly followed.

- **Active customer involvement**: There is lot of client involvement and face-to-face interaction involved in Agile development. Each iteration is tested and approved by client. The feedback obtained from the client is implemented in successive iterations; thus reducing the risk and ensuring higher client satisfaction.

- **Feature driven**: In Agile projects, more stress is laid on providing the required features in the application. The 80/20 rule is applied to decide the 20% features which would be used 80% of the time.

- **Fixed time**: There is a fixed time span for each of the iteration in which it is delivered.

- **Priority based delivery**: Features are prioritized based on client requirement, development risk and so on. Features with high priority are developed first. The project priorities are re-evaluated after each iteration.

- **Adaptive**: Agile methodology is very adaptive. Hence, the developed application is able to accommodate the inflow of new requirements throughout its development with ease. The goal is to adapt to the changing needs, and not to remove the uncertainty in the very beginning.

- **Empowered teams**: The project teams in an Agile environment are usually small and hence the interactivity and communication level is generally high. As the entire team is actively involved, it is empowered to take decisions. As a result, there is no separate team to manage the project.

- **People centric**: In an Agile project, there is more emphasis on using suitably skilled people to do the development than on following the processes. Documentation and other non-development activities are reduced and more time is dedicated to development and testing.

- **Rapid development**: Agile projects use light-weight development technologies to complete the development rapidly.

- **Discipline**: Agile projects involve a lot of team and self-discipline as the deliverables have to be rapidly and accurately delivered. This requires highly organized and skilled team members.

- **Simplicity**: In Agile projects, the emphasis is on being open to change and keeping things as simple as possible.

6.3.2 Comparing Agile with Traditional SDLC Models

Agile is based on adaptive software development methods whereas traditional SDLC process models are based on predictive approach. Predictive teams plan well in advance and have a complete schedule of the tasks and features to be delivered during the product life cycle. These teams depend entirely on the planning and requirement analysis done at the beginning of project. Any changes that come up in the later stages go through strict prioritization and change control management stages.

Agile projects use an adaptive approach which means that there is no advance planning and there is clarity only on the features that need to be developed. There is feature driven development and the team adapts to the changing product requirements dynamically. The release iterations ensure frequent testing of the product, which minimizes the risk of any major failures in future.

Open communication, customer interaction, and minimum documentation are the typical features of Agile development environment. Agile teams work in close collaboration with each other and are mostly located in the same geographical location. The differences between traditional software development and Agile are given in table 6.1.

Parameter	Traditional	Agile
Requirements	Fixed	Evolve during the life cycle
Time and People	May vary	Fixed
Customer Involvement	Initially and at the end	During
Negotiable factors	Estimates	Schedule
Testing	At the end of development	In iterations
Feedback	After	During
Concentration on	Processes and Reviews	Working software
Focus	Plan-driven	Value-driven
Stages	Requirements, Design, Code, Test, Feedback	Adaptive

Table 6.1: Agile vs. Traditional Software Development

6.4 Agile Manifesto

The evolution of Agile software processes is an attempt to deal with the issues introduced by unpredictable and rapidly changing markets. The Agile manifesto was emerged at a ski lodge in Utah in 2001. The manifesto established a common set of values and principles for all Agile methodologies.

The four values provided by Agile manifesto, necessary for high-performing teams, are listed in figure 6.2.

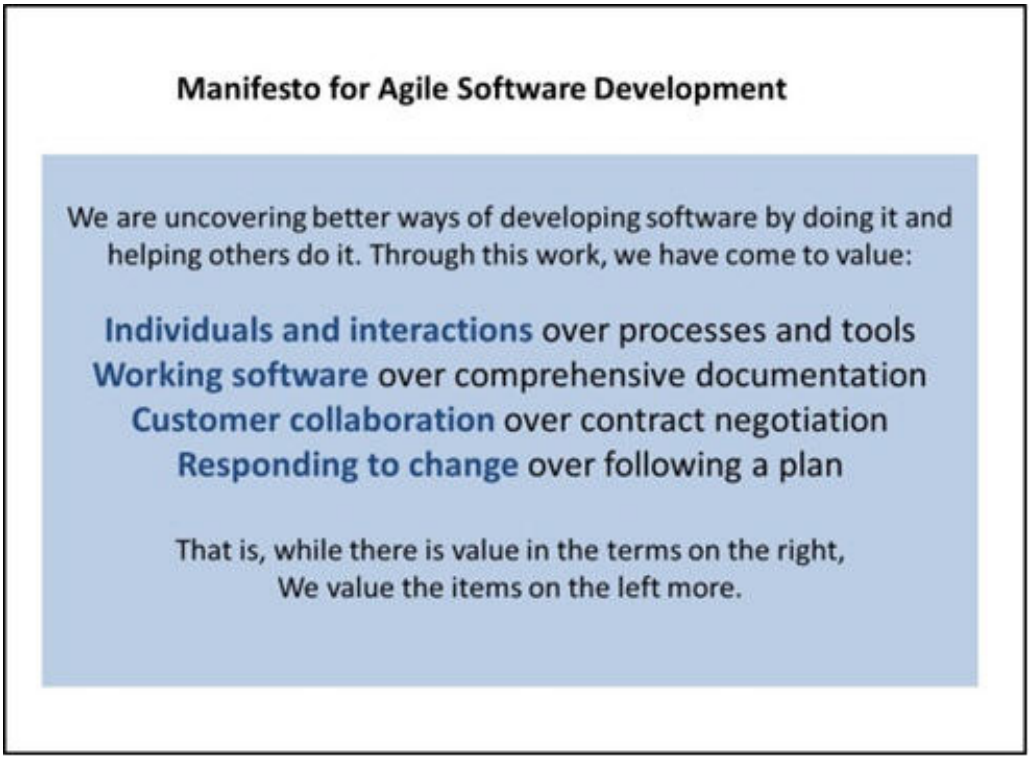


Figure 6.2: Agile Manifesto

6.4.1 Individuals and Interactions Over Processes and Tools

This is the first core value provided by Agile manifesto. It states that the communication between the members of the team must be valued than involvement of processes and tools in project development. In other words, Agile approach values people over processes.

When there are no communication gaps in a team, it can perform better. Thus, Agile methodologies rely on periodic inspect-and-adapt cycles. The inspect-and-adapt cycle can range from few minutes to few hours with continuous integration, to every day with a daily standup meeting, to each iteration with a review. Just increasing the frequency of feedback and communication is not enough to eliminate communication problems. The inspect-and-adapt cycles work well only when team members exhibit several key behaviors:

- Respect for each individual in the team.
- Truth in all communications.
- Transparency of all data, decisions, and actions.
- Trust that each one of them in the team will show their involvement and contribute to their fullest.

➔ Engagement in the team and working towards achieving the team's goals.

It is not as simple as it appears for the team members to exhibit this type of quality. To inculcate this quality, Agile management must provide a supportive environment, team coaches must promote their inclusion, and team members must display them. Only then the team can achieve in its full potential.

Most teams avoid truth, transparency, and trust because of past negative experiences from conflict or cultural norms. To reduce these tendencies, team members and the management must promote positive conflict. When teams employ positive conflict, not only do they encourage more productive behavior, but also work together and achieve several other benefits such as:

➔ The team uses process improvements to identify bottlenecks in the organization to tackle them and then, to systematically eradicate them in priority order.

➔ Discussing conflicting ideas leads to innovation.

➔ Resolution of conflicting agendas happens when teams align and work around common goals and discuss their potential conflicts and concerns.

➔ Commitment to work together happens when people agree on common goals and then put in all their efforts to improve both personally and as a team.

Agile methodologies promote commitment and self-organization by encouraging team members to select items from a prioritized work list, manage their own work, and improve their work practices. These practices make up the foundation of self-organization, which is the driving force for achieving results.

6.4.2 Working Software Over Comprehensive Documentation

One of the biggest differences that Agile development brings is working software. All of the Agile methodologies stress on delivering small increments of working software to the customer at fixed intervals.

Working software is more effective and attract the customers or clients ease of understanding, rather than presenting documents in each interval or increment.

6.4.3 Customer Collaboration Over Contract Negotiation

The rate of success of the software assignments worldwide have more than doubled, over the past two decades. These advancements are the fruits of smaller projects and more frequent deliveries, which allowed customers to provide acknowledgement on working software at frequent intervals.

Getting the customer involved in the software development process is a very important step that leads to the success of projects. Agile methodologies encourage this value by arranging to have a customer representative work along with the development team. Industry data shows that, it is through daily customer collaboration that Agile projects worldwide, have more than twice the success rate of traditional projects on an average. Now that Agile methodologies realize the value of customer engagement, they have a place specifically meant for the customer representative on their development teams.

6.4.4 Responding to Change Over Following a Plan

To create products that provide business value and satisfy customers, teams must respond to change. Industry reports predict that more than 60 percent of product or project requisites change during the software development stage. In widespread projects, even when they meet within estimated budget, on time, and with all the features, customers are not satisfied because the end product is not exactly what they wanted. 'Humphrey's Law' states that customers never know what they require until they see working software. If customers see working software only at the end of the project, it will be too late to make changes to incorporate their feedback. Agile methodologies seek customer feedback all throughout the project so that teams can incorporate feedback and accommodate new requirements as the software product is being developed.

All Agile methodologies have built-in processes to alter their plans periodically based on feedback from the customer or customer representative. These plans are designed to deliver the highest business value first. As 20 percent of the features hold 80 percent of the value, well-run Agile projects tend to finish early. As a result, developers enjoy their work more and customers are happier. The basis of Agile methodologies is that there must be change in order to succeed. For this, there are processes, such as reviews that are designed to reconsider priorities regularly based on business value and customer feedback.

Note -

Agile coding method is used by 60-80% of software developers today. This is a big leap from the 1% in the year 2001.

This is because Agile offers:

- ➔ Enhanced quality
- ➔ More opportunities for improvements along the course of the project
- ➔ Improved customer satisfaction
- ➔ Better alignment between developers and clients
- ➔ Improved time to market

6.5 Principles of Agile

The Agile manifesto consists of the following 12 principles:

1. Customer satisfaction is of highest priority. This is achieved by delivery working software at frequent intervals.
2. The requirements can be changed or added at any point of time. These changes are even welcomed at the later stage of development.

Concepts

3. Working software is frequently delivered at fixed time-boxes, from a couple of weeks to a couple of months.
4. Customers/Clients actively show their involvement throughout the project along with the developers.
5. The project members are appreciated and motivated by rewards and recognized for the work. They work in a supporting environment.
6. Face to face discussions are held to easy convey the needs.
7. Working software is used to measure progress rate.
8. Agile processes advocate sustainable development. The clients, developers, and customers/end users should to maintain a constant pace indefinitely.
9. Continuous governance to technical excellence and simple design brings out agility.
10. Simplicity—maintaining the ease to understand.
11. The best architectures, requisites, and designs arise from self-organizing teams.
12. At fixed intervals, the team members reflect on how to become more effective, then retunes and matches its behavior accordingly.

6.6 Agile Methodologies

Some popular Agile software development methodologies also referred as Agile process models are as follows:

➔ **Extreme Programming (XP)**

Extreme programming is a software development methodology designed to enhance software quality and responsiveness with changing customer requirements. This methodology promotes frequent releases in short iterations which improves productivity and enables new customer requirements to be accommodated. This method derives its name from the concept that the successful elements of traditional software engineering practices are taken to 'extreme' levels in this methodology.

➔ **Scrum**

Scrum is an iterative and incremental Agile software development framework for managing software projects. A key principle of Scrum is that it recognizes that customer requirements change during the project and that problems which are not expected, cannot be solved in a planned manner. Scrum follows an empirical approach accepting that problems cannot be fully understood or defined, and focusing on maximising the ability of the team to deliver quickly and respond to new requirements.

➔ **Adaptive Software Development (ASD)**

Continuous adaptation is the foundation of this development methodology. ASD is used in business

transactions involving considerable risks. It uses continues team cooperation and learning loops to develop the application.

➔ **Feature Driven Development (FDD)**

The main goal of this methodology is to deliver working software periodically in a fixed intervals. Milestones are identified and progress is tracked based on achievement of milestone.

➔ **Lean Software Development**

Lean software development is based on the following lean manufacturing principles:

- Eliminate waste. Waste is defined as anything that does not add value to customer.
- Strengthen learning by doing and testing things rather than documenting and implementing them.
- Make decisions as late as possible thus keeping all the options open as long as possible.
- Deliver as fast as possible.
- Strengthening the team by appreciating and rewarding the people.
- Build purity in the work by keeping the code easy to understand and avoiding repetitions.
- Working towards the end product. This way, the problems can be dealt at application and higher levels, that takes care of the whole application rather than investigating problems at minute levels.

➔ **Agile Unified Process (AUP)**

AUP is a simplified version of Ration Unified Process (RUP). It focuses on building up a model, implementing, testing, deploying, followed by configuration management. The target is on covering the high priority tasks rather than investing time on all possible things.

The AUP is based on the following principles:

- **Your staff knows what they're doing**: Team members aren't interested on detailed and lengthy process documentation, but they need some high-level guidelines and/or mentoring at frequent time intervals.
- **Simplicity**: Everything is described in simple, easy to understand terms in a handful of pages, keeping no/less complexity.
- **Agility**: The Agile UP is based on fundamental principles of the Agile Alliance.
- **Focus on high-value activities**: The focus is on the activities which have high priorities.
- **Tool independence**: Tools that are best suited for the work is used. In brings in the picture of ease.
- **You'll want to tailor this product to meet your own needs**: There is no need to invest on a special tool or take a course to learn it. The AUP product can be easily tailored via a HTML editing tool.

➔ **Crystal**

The crystal group of methodologies is people-centric and emphasizes on enhancing the work of people. These methodologies begin with baby tasks and then grow them up into giant ones. The aerial activities are reduced and battles are redirected to developmental tasks. Tracking and iteration are the other important features.

The main focus of Crystal method of development is on people and not processes or artifacts. Crystal is mainly applied to teams of up to 6-8 members working on systems that are not life-critical. This method focuses on efficiency and habitability as components of project safety.

➔ **Dynamic Systems Development Methods (DSDM)**

DSDM is based on Rapid Application Development (RAD). DSDM is mainly used for projects which have a rigid schedule and budget. DSDM is based on the Pareto principle-80/20 rule which says that 80% of the business benefit comes from 20% of the system requirements. Therefore, the DSDM process starts implementing the first 20% of system requirements, in order to meet 80% of the business needs. It then progresses to cover the rest of the requirements.

6.7 Application of Agile Development

Agile development methods are used in projects with the following characteristics:

➔ Incremental and iterative approach is used in modeling.

➔ Changes in requirements are acted upon and there is no requirement freezing.

➔ Customers are actively involved in modeling.

➔ Prioritization of tasks is done by the key stakeholders and the team works on the requirement of the highest priority.

➔ The team members actively participate in the project.

➔ The content of the model is given more importance than the representation or format.

Agile methods are not used under the following circumstances:

➔ Agile is not appropriate for projects having ample time for development.

➔ Simple projects with repeated ideas need not use Agile.

➔ Customers have limited or no involvement in the modeling or development phases.

➔ Documents are to be prepared at each stage of the project and have to be signed off by the key stakeholders.

- ➔ After a stage is completed and the associated documents are handed over the next team starts work on the project.

- ➔ Case tool is used to specify the design of the software, but not to generate it.

6.8 Case Study

The need for fast-paced software development, like that required by Internet, start-up companies provided the impetus for more flexible software development models, that later came to be labelled as Agile.

To demonstrate these series of events, let us take the example of two start-up companies, A and B, each with their own software development models. Although they were required to use a certified process to provide assurance to their investors, they were not keen on adopting a time-intensive bureaucratic model. To make their processes leaner and more efficient, both companies decided to go off the beaten track.

Company A decided to adopt a team approach to software development. The entire team would use whiteboard to brainstorm and solve programming challenges together. Even though the team took some time to get used to whiteboarding, it turned out to be more productive than using traditional methods of Computer-Aided Software Engineering (CASE) tools. **Company A** learning was that although whiteboarding was more challenging than programming individually using well-known tools, it did not limit them unlike a programming language. The team members were able to express their ideas easily and quickly. **Company A** also took the decision to use open source software where possible to avoid developing already existing software.

Note - Whiteboarding is the term used to describe the placing of shared documents related to an ongoing discussion on participants' on-screen whiteboard, if the sharing is done electronically. Otherwise, it refers to using an actual whiteboard during team meetings and brainstorming sessions.

Company B decided to hire a chief software architect to come up with the design. The architect talked to each team member to get their inputs and designed the software on her own. On completing the design, she published the results for all the team members to view it and give their feedback. He incorporated their suggestions and improvements in the next design cycle to produce better results.

The common aspect of both these approaches is that they were highly interactive development processes. Both companies realized that process-wide documentation was not really helping them and in fact hampered their speed of development. They learned to separate design from documentation.

Question: Which company do you think had better results?

Answer: Both.

Company A came up with their design faster as people worked parallely. As the development team participated in the design process, their design found wider acceptance and considered to be more robust.

Company B succeeded in lowering costs the most as only one person, the architect was involved in the

Concepts

design process. However, as only one person was in charge, this approach could lead to bottle-necks while implementing projects.

The design of both companies could be scaled-up easily to meet organisational requirements. They also managed to meet the requirements of a Rational Unified Process (RUP).

Note - RUP, developed by IBM is a software development process. RUP divides the development process into four phases. Each of these phases include the steps in a traditional software development process, namely, business modeling, analysis and design, implementation, testing, and deployment. The four phases are:

- **Inception** – The project idea is defined and its feasibility is decided.
- **Elaboration** – The software's design is evaluated to ascertain the resources required. Project costs are also worked out.
- **Construction** – The project is executed, the software is designed, developed, and tested.
- **Transition** – After the initial release, end user feedback is taken and adjustments are made to the final product.

The learning from both these approaches are listed:

- Developers are more important than processes, it is important to recognize the ideas and approaches of the development team for coming up with creative software solutions quickly.
- The team should comprises technically sound experienced professionals.
- Documentation is not as vital as previously believed. Although some amount of documentation is essential to any software development process, it should not be allowed to weigh down the process itself.
- As the focus shifts away from documentation, the need for greater communication arises automatically. This in turn translates into more effective problem-solving.
- While there are different modeling tools available, software design benefits from a more interactive approach involving multiple people.
- Developers may need to use more than one tool for software development.
- A comprehensive design is not necessary at the beginning of the project. An iterative model allows developers to introduce improvements and fine-tune design in successive cycles. This means that work need not stop because of an incomplete design and can run parallel to the design process. All this cuts down the overall time requirement of a project.
- Using different sources of solutions and ideas, such as using open source, reduces the effort spent on development, while making sure the focus is on innovation and not reinventing the wheel.

6.9 Check Your Progress

1. What is the expansion for DSDM?

(A)	Dynamic Software Development Method
(B)	Dynamic Systems Development Method
(C)	Distributed System Development Method
(D)	Distributed Software Development Method

2. What does Humphrey's Law state?

(A)	Customers never know what they want, until they see working software.
(B)	80% of consequences stem from 20% of the causes.
(C)	The value of a system grows as approximately the square of the number of users of the system.
(D)	Any piece of software reflects the organizational structure that produced it.

3. Which of the following statements are true regarding the principles of Agile?

(A)	Customer satisfaction is of highest priority.
(B)	The project members are appreciated and motivated by rewards and recognized for the work.
(C)	Working software is used to measure progress rate.
(D)	The best architectures, requisites, and designs arise from individuals in the teams.

(A)	A, B, and C	(C)	C and D
(B)	A, C, and D	(D)	Only D

4. Which of the following manifesto values business rather than following a plan?

(A)	Individuals and Interaction	(C)	Customer Collaboration
(B)	Working Software	(D)	Responding to Change

5. Which of the following are the key features of Agile SDLC?

(A)	Iterative
(B)	Feature Driven
(C)	Sequential
(D)	People Centric

(A)	A, C, and D	(C)	C and D
(B)	A, B, and D	(D)	Only D

6.9.1 Answers

1.	B
2.	A
3.	A
4.	D
5.	B



Summary

- Agile software development is a software development methodology based on iterations increments in software development process.
- The keys of the Agile manifesto are:
  - Individuals and interactions over processes and tools
  - Working software over comprehensive documentation
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan
- The features of Agile SDLC are:
  - Iteration
  - Active customer involvement
  - Feature-driven
  - Fixed time
  - Priority based delivery
  - Adaptive
  - Empowered teams
  - People centric
  - Rapid development
  - Discipline
  - Simplicity



Summary

➔ Some of the most popular Agile software development methodologies are:

- Extreme Programming
- Scrum
- Adaptive Software Development
- Dynamic System Development Methods
- Feature Driven Development
- Lean Software Development
- Agile Unified Process
- Crystal

➔ The learning from earlier efforts to make software development leaner, led to newer approaches that later came to define Agile philosophy.

➔ Documentation is not as vital as previously believed. Although some amount of documentation is essential to any software development process, it should not be allowed to weigh down the process itself.

➔ Agile is not appropriate for projects having ample time for development.

Session - 7

Agile Software Development - II

Welcome to the Session, **Agile Software Development - II**.

This session begins with describing the phases in Agile development. The session then explains Agile artifacts and list the advantages and disadvantages of Agile software development. Finally, the session describes various tools and mechanisms used in Agile software development.

In this Session, you will learn to:

- Describe the phases in Agile development
- Describe Agile artifacts
- List the advantages and disadvantages of Agile software development
- Describe the various Agile tools and mechanisms



7.1 Phases in Agile Development

Similar to the traditional process model such as the waterfall model, the Agile model also has several phases. Though at a surface level, the Agile phases may look similar to a waterfall model SDLC, there are actual several differences. The key difference is that all the Agile phases incorporate collaboration, iteration, and an incremental approach.

In Agile development, the features of the product to be developed are first identified. New features identified in the course of the life cycle are planned across sprints. A sprint is the most basic unit of development and is defined as a fixed period of time during which a specific task has to be completed and made ready for review. The duration of each sprint depends on the feature to be developed. Each sprint goes through all the phases.

Figure 7.1 represents an overview of the phases in Agile development. The actual implementation may differ in the real world. The Requirements Analysis phase, for example, may consist of three sub-stages, Conception, Initiation, and Analysis where the project is first conceptualized, initiated, and then, finally, analysis begins. The basic tasks in each phase are almost similar to the waterfall model, however every phase is iterative and incremental.

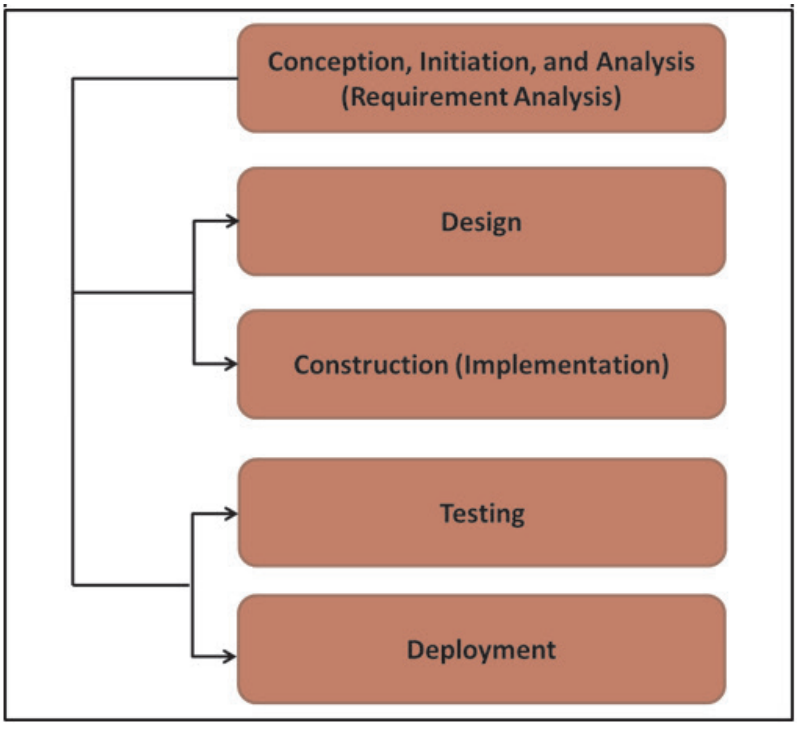


Figure 7.1: Phases in Agile Development

The **Conception Phase**, sometimes also called as Iteration -1, helps to define the problem or business opportunity, that leads to identify and assess a feasible strategy for the project.

The next phase, **Initiation**, takes place during the first one week or so and is sometimes called Iteration 0. In this phase, the project is initiated through a feasibility study and working with stakeholders to determine the initial requirements. **Analysis** phase involves performing a requirement analysis. **Design** phase involves creating design strategies, to model the system and **Implementation** phase begins the

Concepts

actual development of the project. **Testing** will evaluate and test the completed project on various aspects and factors and **Deployment** is the final stage of launching the project.

The basic principle of these phases is similar to that of waterfall model. The key difference is each piece of the development work (either individually or as a group) can be iterative and developed, tested, and released. In traditional approaches, a stakeholder sign-off is needed after completion of each phase. This, in turn, needs all the requirements to be completed in that specific phase. In Agile development, only an approval from stakeholders for each requirement can be considered in order to push the development to the next phase. This means that in Agile development, each operational team (such as business analysts, technical architects, developers, testers, and so on) must participate collaboratively.

7.2 Agile Artifacts

In software engineering, an artifact is defined as any form of portrayal denoting intermediate or end product of the development process. Some example of artifacts include requirements specifications, source code (programs), project plans, and so on.

Artifacts are created for a specific purpose and used accordingly to develop software.

In order to measure a project's progress, Agile project teams often use the following main artifacts or deliverables:

- **Product vision statement**: Comprises a short overview statement that indicates how a product supports an organization's strategies. The key purpose of the vision statement is to express the goals or outcomes of the product.

- **Product backlog**: Comprises a list of tasks for the project, ordered by priority. Some of the ways in which Agile project teams may use this term are as follows:
  - Tasks that may be considered in the next iteration's plan form part of the backlog
  - Tasks that are certain to be worked on in the next iteration may be part of the backlog
  - Tasks assigned to the current iteration, but are currently not being worked upon are also part of the backlog

- **Product roadmap**: Comprises a roadmap for the product requirements, with an approximate time frame for the development of those requirements.

- **Release plan**: Comprises an evolving timetable that describes the features to be delivered in upcoming releases.

- **Sprint backlog**: Comprises a list of tasks to be completed within the current sprint. The tasks in the sprint backlog must be ranked in the desired order of implementation. The ranking indicates the urgency of the task and dependencies (if any) that exist between tasks.

- **Increment**: Indicates the product functionality at the end of each sprint.

Figure 7.2 shows the differences between traditional and Agile documentation.

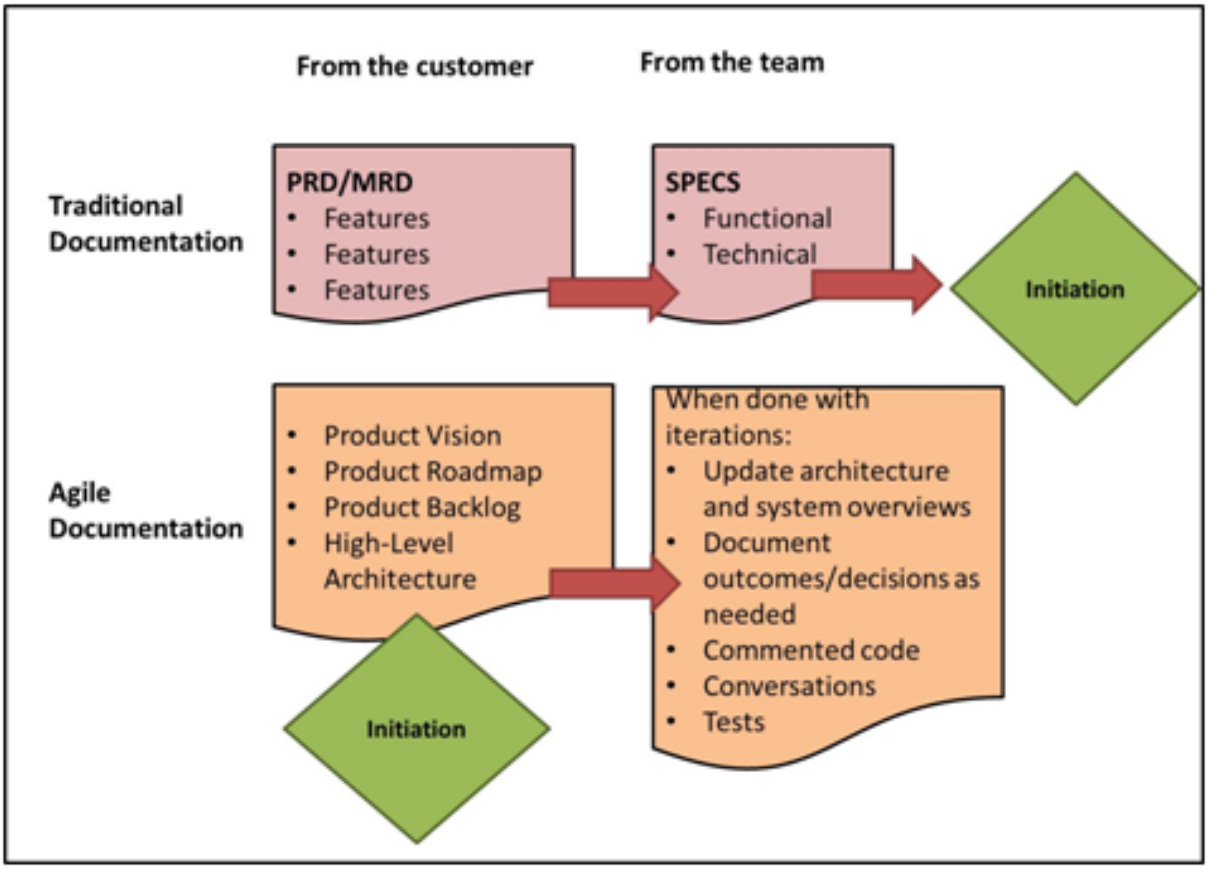


Figure 7.2: Traditional vs. Agile Documentation

7.3 Agile Advantages and Disadvantages

The advantages and disadvantages of the Agile software development are listed in table 7.1.

Advantages	Disadvantages
Very realistic in its approach.	Not suitable for handling complex dependencies
Promotes teamwork and cross training.	Requires skilled manpower.
Functionality can be developed quickly and demonstrated.	Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
Minimum resource requirements.	High individual dependency leads to minimum documentation. Lack of documentation might lead to difficulty in knowledge transfer to new team members.
Applicable for fixed or changing requirements.	Heavily dependent on customer interaction for collecting changing requirements
Delivers partial working solutions early in the cycle.	Partial solutions may not cover all the requirements.

Concepts

Advantages	Disadvantages
Suitable for regularly changing environments.	Requires strong change management system.
Minimal rules.	Adhoc rules may have to be made.
Development and delivery happens in parallel within an overall planned context.	Invariably requires an overall plan.
Easy to manage.	Risk of sustainability, maintainability, and extensibility.

Table 7.1: Agile Advantages and Disadvantages

7.4 Agile Tools and Mechanisms

The major Agile tools and mechanisms used today are as follows:

- Velocity
- Burndown and Burnup charts
- Informative Workspace
- Big Visible Charts
- Hand-Drawn Charts
- Process Improvement Charts
- Task Boards

- **Velocity**

  Velocity is measured by the quantity of work that the team members together can accomplish in one iteration. Velocity is used to measure the time taken for a team to deliver upcoming outcomes based on its prior performance. It also measures tasks in hours or an equivalent unit. This is ideal for an Agile team where work is comprehensively completed after each iteration. During the first iteration, the team has no velocity, so an approximate assumption is made as to how much work is likely to be accomplished. With the velocity calculated in the previous iteration, the amount of work that can be accomplished in the further iterations are estimated.

  Some of the most common units used for estimating work and determining velocity are Story points used in Scrum and craft units used in eXtreme Programming (XP). The team determines its velocity by calculating the average amount of work completed till date and the number of iterations completed. If a team size changes, such as new members added to a team or members are removed from a team, the velocity must be recalculated.

Consider a few simple examples to understand velocity.

- **Example A**: A team has completed 20 stories during a two-week sprint and each story was worth three story points, then the velocity of the team can be calculated as 60 story points per sprint.
- **Example B**: A team regularly completes an average of 60 units of effort in a two-week sprint has a velocity of 60 units of effort per sprint, or is simply considered as just 60.
- **Example C**: The product backlog of an Agile team has 400 story points. The team is averaging 40 story points per sprint. In this case, the team will require 10 more sprints to complete work.

The data used to calculate a team's velocity is depicted on the project burndown and burnup charts.

➔ Big Visible Charts – Burndown, Burnup, and Other Charts

Big visible charts are big charts placed near the Agile team workspaces that convey the progress of the project team in various ways. It is not just the team members who can make use of this, but also the stakeholders who can get a grip on the team's progress. Pictorial representations help in clear understanding and help in easy analysis. Thus, these charts aid the team in working effectively towards the overall progress in developing the software. Big visible charts implement the essential Agile values of transparency and communication. The goal of a big visible chart is to display information so clearly that it is seen even from across the room. It communicates the work to be done or the flow of work.

- **Burndown Charts**

  A burndown chart is the one of the most important tracking tools for Agile projects. It tracks the work remaining on the project and the possibility of the deadline being met. The vertical axis measures the work remaining and the horizontal axis marks the iterations. The planned end date of the project is marked in a burndown chart. The progress of each iteration is marked on the chart and is projected forward to see if the end date is likely to be met.

  Though its application and usage may vary, plotting a burndown using effort remaining is the most useful and efficient way of using burndown charts.

  Burndown can be tracked at either the sprint level or the release level. Sprint burndowns are often tracked using effort remaining whereas release burn-down are tracked using story points.

In figure 7.3, it can be observed that four iterations are now over. The dotted line suggests that the project has fallen behind schedule. Even though, there was good progress after the first iteration, things changed when the second iteration was completed. Ideally, the project manager should have taken some action then. Thus, it helps to track the work done and pending tasks.

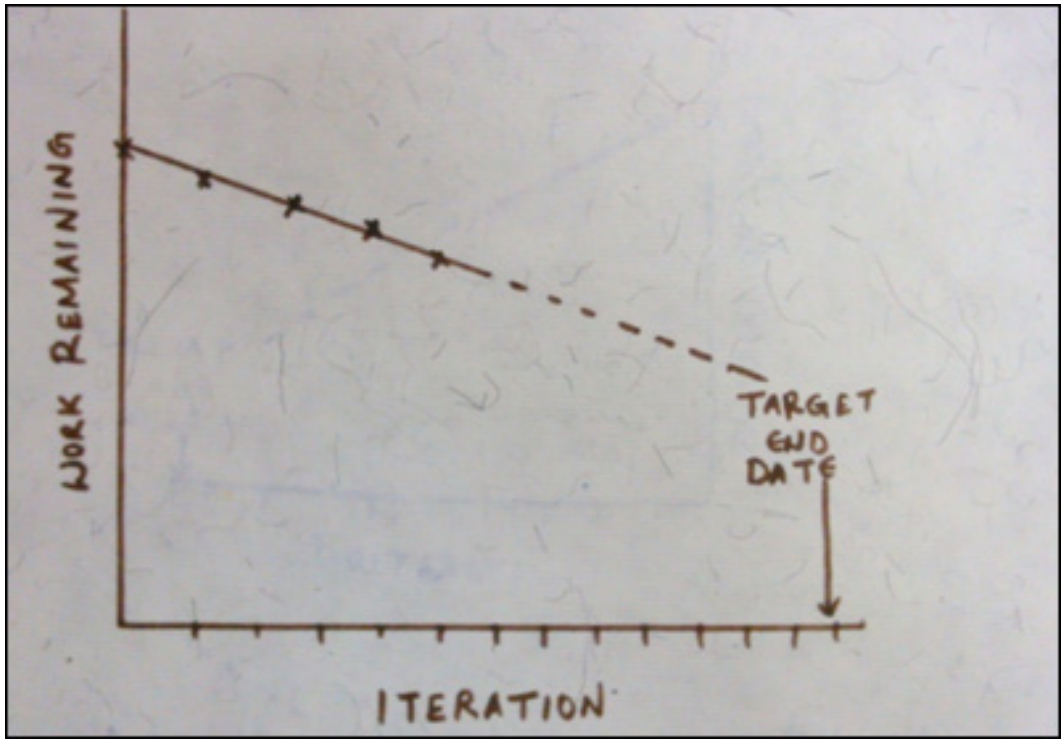


Figure 7.3: Burndown Chart

Burndown charts are among the most common sprint tracking mechanisms used by Agile practitioners.

- **Burnup Charts**

A burnup chart tracks the amount of work done. It has more information than a burndown chart because it shows the quantity of work in the whole project. A burnup chart shows changing goal or scope of the project, which is difficult to show in the burndown chart.

Figure 7.4 shows the burnup chart.

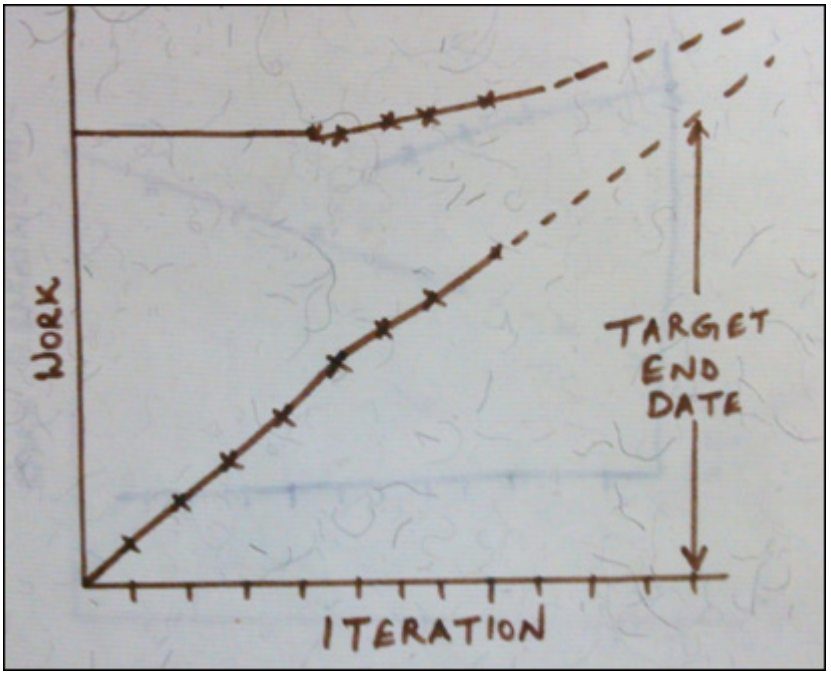


Figure 7.4: Burnup Chart

In the burnup chart, as shown in figure 7.4, the bottom line represents the work done and the top line, the total scope. Dotted lines show projections. The scope has remained steady for the first iteration, but has increased around the fifth iteration. This would have been acceptable if it had not continued to increase after that. By observing the projections, you can see that at the current rate of scope creep, it is unlikely that the project will meet the target deadline. The project may not even make it to the end if the scope continues to remain steady.

- **Process Improvement Charts**

  Process improvement charts are a type of big visible charts that measure specific issues that the team would like to improve upon. These charts stay only as long as they are necessary.

  The process improvement charts are created and maintained as a team responsibility and are kept up-to-date. Each team member updates his own status. Some charts have information updated on daily basis based on the day's progress; this is updated by a particular team member. The main goal of these charts is process improvement. People usually tend to improve their status when the team shows progress towards a mutual goal.

- **Other Charts**

  The iteration and release planning boards are other examples of big visible charts. The team calendar is another useful status chart that display important dates and iteration numbers. They also give information as to when a team member is not available in office, along with their contact information.

➔ Informative Workspace

An informative workspace is not a tool, but rather a practice. It is a workspace that has feedback mechanisms incorporated to help the people who are working in it. As a workspace is the main hub of the development effort, an informative workspace has to be created by arranging it with the information necessary to head for the project. The important feature of an informative workspace is that it advertises information into the room. An informative workspace makes the team members understand the phase of the project by just walking into the room. This can be done by blending the physical and virtual elements of a work environment. For example, visual displays and electronic devices that are connected to automated processes can be part of such an environment. It conveys status information without asking for it and interrupting the team members and thus, helps to improve the trust of the stakeholder.

Big visible charts described earlier are part of informative workspaces.

➔ Hand-Drawn Charts

The main benefit of the informative workspace is that the information displayed is constantly visible from everywhere in the room. However, for computerized charts to meet that criterion, projectors or plasma screens have to be installed everywhere, which is difficult and expensive. In such scenarios, hand-drawn charts are a solution. As the information in the charts constantly change, it is easier to use flip charts and whiteboards to draw a chart than creating one on a computer.

➔ Task Board

A task board represents at a minimum, three columns, 'to-do', 'in-progress', and 'done', and is used for organizing a team's work. It mostly displays the work to be performed during the current iteration. Some teams also include their backlog as a column on the task board. Distributed teams mostly use online task board applications but, ideally, the task board consists of note cards or sticky notes affixed to a wall. The task board illustrate different forms of work such as user stories or just tasks.

7.5 Check Your Progress

1. What is a product roadmap?

(A)	A high-level time table
(B)	A quick summary of the goals
(C)	A high-level view of product requirements
(D)	The tasks associated with the current sprint

2. A release planning board is an example of a _____ chart.

(A)	Big visible chart	(C)	Burndown chart
(B)	Process improvement chart	(D)	Burnup chart

3. Which of the following charts have three basic columns namely, 'to do', 'in progress', and 'done'?

(A)	Hand-drawn chart	(C)	Big visible chart
(B)	Task board	(D)	Process improvement chart

4. _____ comprises an evolving timetable that describes the features to be delivered in upcoming releases.

(A)	Product roadmap	(C)	Product backlog
(B)	Release plan	(D)	Increment

5. Which of the following is not a tool, but rather a practice?

(A)	Informative Workspace	(C)	Velocity
(B)	Hand-Drawn Charts	(D)	Scrum

7.5.1 Answers

1.	C
2.	A
3.	B
4.	B
5.	A



Summary

- In Agile development, all phases incorporate collaboration, iteration, and an incremental approach.
- New features identified in the course of the life cycle of an Agile project are planned across sprints.
- A sprint is the most basic unit of development and is defined as a fixed period of time during which a specific task has to be completed and made ready for review.
- The duration of each sprint depends on the feature to be developed. Each sprint goes through all the phases.
- in Agile development, each operational team (such as business analysts, technical architects, developers, testers, and so on) must participate collaboratively.
- Some of the artifacts of Agile projects include Product vision statement, Product backlog, Product roadmap, and so on.
- The major tools and mechanisms used in Agile software development are Velocity, Informative Workspace, Big Visible Charts, Hand-Drawn Charts, Task Boards, and so on.

Session - 8

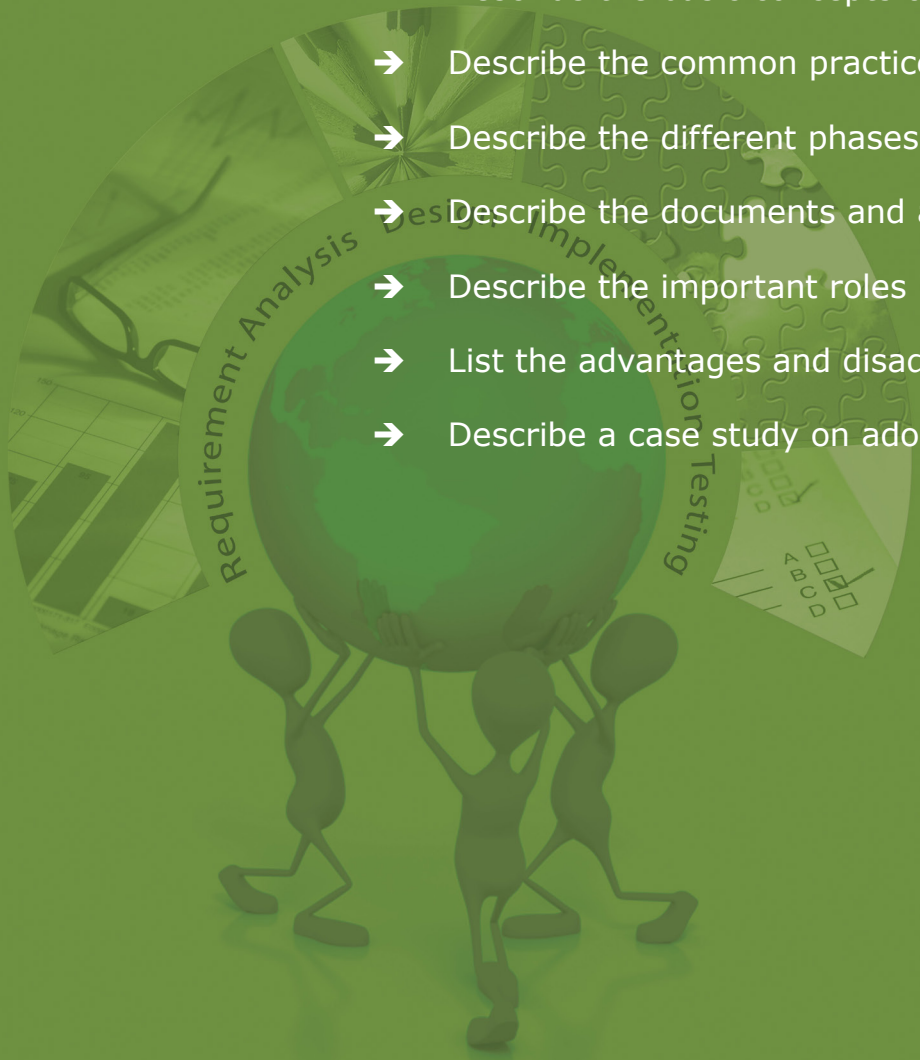
Extreme Programming

Welcome to the Session, **Extreme Programming**.

This session explains the basic concepts and practices of Extreme Programming (XP). The session explains the different phases in Agile XP process model. The session also discusses about different users involved in the XP model. Finally, the session lists the advantages and disadvantages of XP and presents a case study on shifting to XP practices for software development.

In this Session, you will learn to:

- Describe the basic concepts of XP
- Describe the common practices in XP
- Describe the different phases in XP
- Describe the documents and artifacts used in XP
- Describe the important roles in an XP team
- List the advantages and disadvantages of XP process
- Describe a case study on adopting XP practices



8.1 Introduction

Extreme Programming (XP) is one of the many popular Agile processes. It concentrates on software development rather than software project management. The first XP project was started on March 6, 1996. XP was designed in such a way that organizations could either adopt the whole methodology or in parts. This process has been proven to be very successful at many companies worldwide.

The main focus of XP is on customer satisfaction. This process delivers the software as and when it is required rather than delivering everything in bulk, at the end. The advantage of XP is that it empowers developers to handle changes in customer requirements confidently, even late in the life cycle.

XPalso emphasizes teamwork. Customers, developers, and managers coordinate as a team and work together as equal partners. This methodology enables teams to become highly productive by implementing a simple and effective environment. The team organizes itself around the issues and efficiently solves them.

XP methodology brings together the whole team with the help of simple practices. It also provides feedback which enables the team to see where they are and helps them to adapt practices to their unique situation. In XP, every individual contributor to the project is an indispensable part of the whole team. The customer is also an integral part of the team who sits along with the team and works with them daily.

The primary practices of XP are as follows:

- **Sit together**: The whole development team sits and operates from one single open space.
- **Cross-functional team**: XP utilizes a cross-functional team consisting of everyone who is essential for the success of the product.
- **Informative workspace**: XP encourages an informative workplace by placing informative charts around the workspace so that team members and stakeholders get an idea about the status and progress of the project.
- **Energized team**: XP teams are not encouraged to work overtime. The idea behind this is to keep the quality of the code high and the programmers happy.
- **Pair programming**: This refers to the practice where two programmers work together at one computer, working on the same design of the production software.
- **User-stories**: The team write short statements of customer-visible functionality required in the product. The customer prioritizes the stories and the developers estimate it.
- **Weekly cycle**: A meeting is held at the beginning of each week to review the progress of the project. The customer decides the stories that are to be implemented that week and divides it into tasks. By the end of the week, acceptance test cases for these stories should be ready for demonstration to the customer. The feedback from this drives the next weekly cycle.

- **Quarterly cycle**: The whole team picks a theme or themes for the quarter. Themes enable the team reflect on the bigger picture. This business value is delivered at the end of the quarter.

- **Slack**: In each iteration, a few of the lower-priority tasks are planned, so that they can be dropped if the team lags in their work. This way, the most important functionality is still delivered to the customer.

- **Ten-minute build**: The project structure is designed in a way that facilitates frequent testing. This allows tests to be run within ten minutes. This practice promotes regular testing.

- **Test-first programming**: All stories have at least one preferably automated acceptance test. The user story is considered to be fulfilled when it passes the acceptance test. Automated unit tests are also incrementally written using the Test-Driven Development (TDD) practice. TDD is a software development process that relies on the repetition of a very short development cycle. First, the developer writes an automated test case that defines a desired improvement or new function, then produces the minimum amount of code to pass that test, and finally refactors the new code to acceptable standards.

- **Continuous integration**: Programmers check in to the completed code and the associated tests several times a day. Code is only checked in if all associated unit tests are passed.

- **Incremental design**: Rather than designing the whole system in advance, it is designed every day using the experience gained till the previous day. Refactoring is done to improve the design of previously-written code. Teams with strong unit tests can safely experiment with refactoring as a safety net is in place.

8.2 Values of XP

XP has well-defined practices and guidelines. The first step in the process is collecting short use cases which can be put on an index card. The short use cases or stories as they are called, must be testable, estimable, and business-oriented. The customer prioritizes the stories and selects the most valuable one. This group makes up an iteration and is coded first. Coding is done in pairs, where two people code on one machine, and an iteration is typically one to two weeks long. Once the coding is complete, the code is tested and put into production.

XP methodology is based on five values:

- Communication
- Feedback
- Simplicity
- Respect
- Courage

- **Communication**: XP has an oral communication culture and its practices encourage interaction. Communication is given importance based on the observation that issues in most projects occur due to lack of communication.

- **Feedback**: The customer provides feedback to the development team at the end of each iteration and release. This feedback drives the next iteration. Design and implementation feedback loops are also built into the methodology.

- **Simplicity**: An important aspect of this value is to design and code only what is in the current requirements and develop the simplest product that meets the customer's requirements.

- **Respect**: Team members are to respect each other as valued team members. They must also respect the customers' expertise.

- **Courage**: The other values such as transparency about project progress and estimates enable the team to be courageous in its performance and decision.

8.3 XP Process Activities

The life cycle of XP consists of phases:

- Exploration
- Planning
- Iterations to Release
- Productionizing
- Maintenance
- Death

Figure 8.1 shows the different phases in XP.

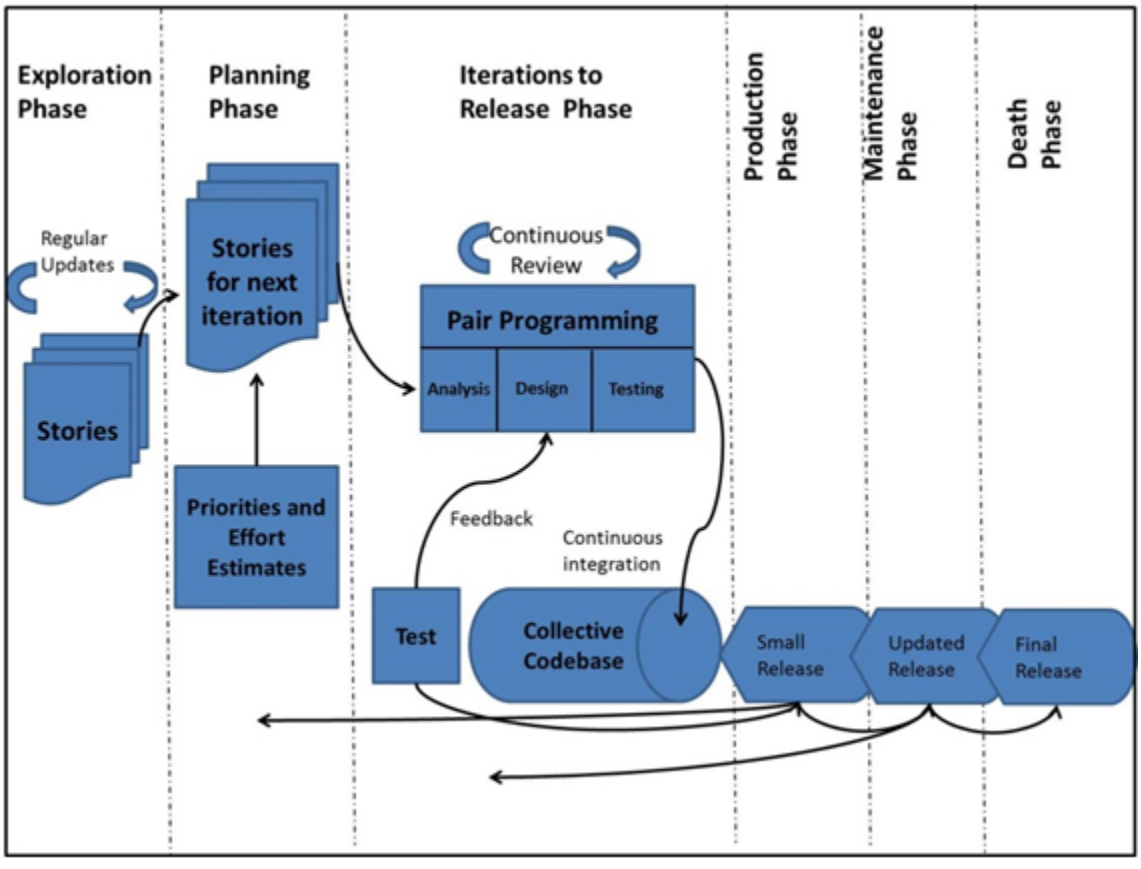


Figure 8.1: Phases in XP

➔ **Exploration Phase**: In this phase, the customers write the story cards that they desire to include in the first release. The story cards describe the features to be added in the program. Meanwhile, the project team familiarizes themselves with the project, including the technology, tools, and practices that are to be used. The team builds a prototype of the system in order to check the technology and the architectural capabilities. The duration of the exploration phase depends on how familiar the programmers are with the technology and takes anywhere between a few weeks to a few months. paradigm

➔ **Planning Phase**: This phase usually takes a couple of days. In this phase, the stories are prioritized and contents of the first release is decided. The schedule is drawn after the programmers estimate the effort required for each story. The scheduled duration of the first release normally does not exceed two months.

➔ **Iterations to Release Phase**: This phase includes several iterations of the systems before the first release. The schedule fixed in the planning phase is divided into several iterations each of which takes between one to four weeks to implement. The first iteration creates a prototype of the whole system. The customer decides and selects the stories for each iteration, and creates the functional tests to be run at the end of each iteration. The system is ready for production at the end of the last iteration.

- **Productionizing Phase**: In this phase, the system goes through extra testing and performance checks before being released to the customer. New changes can still be suggested by the customer and the decision to include it in the current release has to be taken. The duration of the iterations are shortened in this phase. The ideas and suggestions which were postponed are documented for implementation during the maintenance phase.

- **Maintenance Phase**: After productionizing the first release for customer use, the XP project must keep the system under development running and also produce new iterations. In order to do this, customer support tasks are also taken up during this phase. Thus, the development velocity decreases once the system is in production. In order to compensate for this deceleration, changes in the team structure may be required and additional people may have to be included in the team.

- **Death Phase**: In this phase, the customer requirements are met and no additional stories to be implemented. As there are no more changes to the architecture, code, or design, the necessary system documentation is done. Death also occurs if it gets too expensive for further development or if the system is not delivering the desired output.

8.4 XP Documents and Artifacts

Rather than documents, XP relies mostly on oral communication, the code, and tacit knowledge. While oral communication works for a small team, bigger teams require documentation to be in place. This is also applicable in case of systems which by nature, are high-risk and are audited for legal, engineering reasons, and/or software consistencies. To deal with such issues, the following artifacts have to be more formally managed, recorded/preserved, and regularly re-visited, as part of a formal and traceable XP process.

- **User story cards**: These consist of paper index cards which contain short requirement descriptions. The user story cards do not contain a full requirement statement but are a commitment for further conversation between the customer and the developer. During this conversation, the customer and the developer come to an understanding of the requirement to be fulfilled. Developer resource estimates and customer priority are also included in the card.

- **Task list**: Tasks represent visible features of a user story. The task list is a listing of the tasks for the user stories to be developed for an iteration. The resource estimate for a user story should not exceed the iteration duration. Programmers are not assigned tasks but rather volunteer for it.

- **CR Cards**: CRC stands for Class-Responsibility-Collaboration. The responsibilities and collaborators of classes are identified during a design brainstorming session involving multiple developers and are recorded on paper index cards. This serves as a basis for software design.

- **Customer Acceptance Tests**: The customer develops Customer Acceptance Tests which are textual descriptions and automated test cases. Successful conclusion of the test cases denotes the completion of a user story and the confirmation of customer requirements.

- **Visible Wall Graphs**: Posting progress graphs in team work area encourages communication and accountability. The graphs usually depict the number of stories completed and test cases passed.

8.5 Roles in an XP Team

The important roles in an XP team are as follows:

- **Manager**: The manager owns the team and is responsible for the issues faced by the team. He or she forms the team, gathers resources, manages people and problems, and coordinates with external groups.

- **Coach**: The coach teaches the team about the XP process, intervenes in case of issues, and monitors the process. The coach is basically a programmer and not a manager and is also responsible for the process as a whole.

- **Tracker**: The tracker traces the estimates made by the team and provides feedback in order to improve future estimates. The tracker also creates visible wall graphs by collecting user stories and acceptance test case progress from the developers. The tracker is a programmer, and not a manager or customer.

- **Programmer**: A programmer writes tests, design, and code. They also refactor, identify and estimate tasks and stories.

- **Tester**: A tester helps customers write and develop tests. They also run functional tests, broadcast test results, and maintain testing tools. A tester can also be a programmer.

- **Customer**: The customer writes stories and acceptance tests. They also decide which stories are to be prioritized a release and for an iteration. As customer collaboration is essential for the success of the project, a representative is selected from within the development organization. Occasionally, some customers also are involved in the team.

8.6 Advantages and Disadvantages of XP

The advantages of XP are as follows:

- **Robustness**: The main advantage of XP is its simplicity. The XP design is similar to a jigsaw puzzle where developers work on many small pieces or iterations. Combining the iterations at the end gives the final product. Regular testing at the development stage ensures detection of all errors. This approach creates working software faster with minimal defects. In this manner, the coding blocks created by the team, pass through validation tests that are customer-approved and therefore, once successfully completed are exactly what the customer requires.

  One advantage of this approach is that cost estimates are based on software features instead of developer activity. This allows the customers to make decisions about what has to be included,

depending on the budget. By prioritizing important requirements first, the customer obtains maximum value with the least amount spent. This results in a trade-off on the marginal increase in product utility with the cost to incorporate additional features. This approach also allows both the customer and the user to terminate the development at any point and still have valuable and functional code, even if incomplete.

- **Resilience**: Requirements in software projects keep changing either because of the emergence of new business opportunities or simply because the partial data made available at the initial requirement-gathering phase. Changed requirements readily are adopted by XP, through collection of user stories before beginning each iteration and by obtaining feedback throughout the course of iterations.

- **Cost Savings**: XP cuts short unproductive activities to reduce costs. It allows developers to focus on coding instead of spending time on unnecessary paperwork and meetings. It also takes away the requirement for separate testers.

  The cost of making changes increases as the software advances in its life cycle. Traditional programming methods make changes at the end of the life cycle, whereas XP allows changes at the development stage.

- **Lesser Risks**: XP reduces the risks related to programming drastically. Traditional programming methods depend a lot on critical members in the team. A task split into modules and each team is made responsible for their module. By distributing the risk, the dependency on any one architect, individual coder, or project manager is significantly reduced.

- **Employee Satisfaction**: The methodology of XP, with its value-driven approach encourages fixed work time with little scope for overtime. This helps increase employee satisfaction and retention. The breakdown of the project into subcomponents prevents accumulation of work to be completed before a tight deadline.

The disadvantages of XP are as follows:

- XP is a difficult process to apply in a practical scenario. As it takes discipline to follow this approach, it is difficult to convince developers to adopt this methodology and practice it. The customers also may not like the idea of too much involvement in the project. The management may have trouble getting used to the practice as it changes dynamically during the project life cycle.

- XP is code centric and not design centric development. The lack of a design concept in XP may not affect small programs much, but can be problematic when programs are large or when there are too many people associated with the project.

- Quality planning helps adequately trained teams produce high-quality products. XP does not measure or plan the quality aspect of development.

- Software projects today are large and complex by nature and are hard to design using incremental method that XP practices.

- XP emphasizes on refactoring during software development process. Too much refactoring can be a waste of time rather than being productive.

- Traditional programming enables testers to find defects easily because it is well documented. XP programming is not structured and it is difficult for testers to find errors by just looking at the screen.

8.7 Case Study

WDSGlobal is the leading global provider of knowledge-based services to mobile operators, manufacturers, application providers, and sales channels. In 2004, WDSGlobal brought together three independent development regions to form a development team that operates around the clock. This case study describes the challenges faced by the team in this environment, the lessons learned, and how issues such as global continuous integration, conflicting priorities, and cultural differences were resolved across regions.

Background

WDSGlobal has ten years of experience in the mobile market place. Each step in the mobile product life cycle is aided by the services provided by WDS Global. It includes settling any problems in the pre-launch stage to assisting with specialized customer support.

All the efforts that are put in the development environment is dedicated to the refinement of WDSGlobal's mobile configuration platform, which is used on the Sony Ericsson, Nokia, Siemens, and Vodafone Websites. As the mobile industry innovates frequently, the demand for WDSGlobal services has reached are markable level, placing a heavy burden on the development teams.

One Global Team

The company created three regions – the Americas, Asia, and Europe, in order to meet the requirements of its customers across the globe. It was discovered that the regions had some very common and some different requirements. The company had to locate its developers in order to meet a market's specific requirements of a particular region and then share the knowledge appropriately.

Till the end of 2003, Web-based tools were developed and deployed using XP and Java, as Application Programming Interfaces (APIs) by the UK team. In each region, a non-XP team uses these internal APIs to deliver to localized requirements with the help of non-Java technologies. This process created a lot of unnecessary duplication in the Websites and led to costly maintenance for the different technologies used in each region.

At the beginning of 2004, it was proposed to combine all the development teams into one global team which uses XP and Java. This global team was to share the same code base to minimize duplication and reduce maintenance costs.

➔ **Hurdles**

The location of the development team led to a number of challenges. One of these was that the teams did not have frequent communication with each other previously. There were also differences in the time zone, culture, and technical backgrounds.

- **Time Zone**: There was a minimum overlapping of time between the working hours of the three regions. Communication with each other outside their normal working hours impacted personal schedules and family commitments.
- **Cultural Differences**: Even though the members of all regions spoke English, many misunderstandings arose because the groups did not really share a higher-level common language. There were also a number of issues caused by unrecognized cultural difference which frequently led to confusions and disagreements between regions.
- **Technical Background**: The programming styles varied greatly among the regions. For example, the members of the UK team had a background in object-oriented programming and had some experience with Extreme Programming. The U.S. and Singapore developers preferred working in an academically correct manner. They did not have any experience with unit testing or refactoring. On the other hand, there were some other members who were not familiar with the XP process or object-oriented programming. They focused on getting work done quickly and learned new tools and techniques when required.

8.7.1 Transition to XP

WDSGlobal faced a number of technical, ownership, and training-related challenges during the transition to a global team.

The Source Control System (CVS) used by the UK team was not practically feasible for remote use. It took up to 40 minutes to synchronize any changes from a remote region. This led to the selection of Perforce, a new source control system. Local proxies had to be installed to deal with the performance problems. Two more build machines had to be created as the UK build machine was not sufficient enough for all regions. There were no pairing machines in the U.S. and Singapore regions, so new pairing machines and desks were purchased and installed according to the same specification as those in the UK. In order to accommodate the cube-free structure and pairing stations, both regions had to restructure the office space.

Given that the UK team had created the original core back-end segment of the system, they had difficulty sharing ownership with the other regions. While the new processes and technologies were welcomed by some of the U.S. and Singapore developers, others were overwhelmed and could not adapt to the large shared code base and the new XP practices.

Some of the changes done were as follows:

➔ **Regional Coaches**: An XP coach was hired in each region as the non-XP regions did not have any prior experience in Agile methodologies or Object-Oriented Programming (OOP). The coaches trained the local team in XP, and object-oriented programming with Java. The coaches also helped local business units to adjust to the new development process. In due course, the coaches

Concepts

recruited more people with Agile and OOP experience in the region. They also conducted weekly coach meetings which helped bring the teams together.

- **Boot Camp**: In an effort to develop mutual trust, the original UK XP team and developers from the non-UK region spent several weeks working together the UK office. By working alongside each other, the developers were able to build initial trust relationships and share common work practices. This helped a great deal in focusing their energies as a team towards solving problems and producing innovative solutions.

- **Rotating Guru**: The non-UK regions required help in setting up infrastructure, initial training, and mentoring. To assist with this, a senior member of the original XP team was sent to those particular regions. This provided the UK developers, an opportunity to identify and analyze the difficulties the other teams dealt with in developing remotely from the UK servers.

8.7.2 Problems Faced During Transition

- **Preparing the Business**: Although, WDSGlobal trained the developers in XP methodologies, it did not adequately prepare the business in each region for a dramatic change in the process. WDSGlobal did not have the necessary resources to train business units and developers.

- **Switchover**: In April 2004, all the development teams were using XP as the development process and Java as the technical platform and were virtually one team. The non-UK regions had the most trouble during the switchover as they had no prior exposure to XP and OOP. As a result, some were not able to make the transition. It would have been a better approach to have a more gradual transition by formally training the non-UK regional developers on Java and OOP, pairing them with Java developers for three months and then teaching them XP practices.

- **Inadequate Infrastructure**: The new distributed development environment was too large for the current global infrastructure to cope with. The network bandwidth did not support the large amounts of data and traffic through the network between the regions. The U.S. and Singapore developers experienced long delays during file copies. The continuous integration builds across servers located in different regions was a time-consuming process. These concerns were addressed in a step-by-step manner – by using Perforce and its caching proxies as the source control system and increasing the network bandwidth.

- **Continuous distributed development**: The standardization of values and principles across the board, leads to the success of an XP team. This includes a shared knowledge of refactoring, coding standards, and test-driven development. However, the interpretation of the particulars of these principles and values differ from person to person. In a non-distributed XP team, common ground is achieved through individual adjustments and communication among team members. In a distributed team environment, as team members are located in different countries and time zones and come from different cultural backgrounds, they do not enjoy constant communication.

8.7.3 Solution to Problems

The global team found it difficult to maintain a common process vision and values. Everyone in the team had to trust each other as equal team members in order to share the same code base and system. Without proximity, the trust relationship worsened over time.

In a non-distributed XP team, when new ideas and technologies are introduced, the team comes to an agreement before the team proceeds. With the virtually combined, yet distributed team, it was tough to introduce new technologies or architectural changes.

➔ **Daily Handovers**: Communication within the team is crucial in XP. In a distributed team, communication is of utmost importance. Daily handovers took place, when one workday ended and another began, amongst the three regions. Using Seattle time as the base, the handovers were done in the following manner:

- 9 a.m. - The UK region hands over the system to the U.S.
- 5 p.m. - The U.S. region hands over to Singapore.
- The Singapore region then hands over to UK at the end of their workday.

Initially, the daily handovers were mainly daily status updates. The objectives of the handovers gradually improved toward sharing daily learnings, collectively developing design ideas, and identifying and address cross-regional issues.

➔ **Face-to-face Communication**: The use of video conferencing for daily handovers and customer requirement gathering sessions as face-to-face communication is considered much more effective than e-mail or phone conversations.

➔ **Shared Common Environment**: In the distributed team, all pairing machines are installed in the same way to help minimize configuration and adjustment time when development pairs rotate to different machines. The configurations are checked into the shared source control system, so all developers receive configuration changes automatically when they synchronize with the system as they begin each workday.

➔ **Remote Pairing**: The close interaction within a non-distributed XP team often allows the team to form a group mind. These happen not just through pairing session experience, but also through informal discussions at coffee breaks or lunches. This shared context based on prior knowledge and experience helps each team member understand the others better when a complex idea or design is being discussed. This is much more difficult to achieve in a distributed team. Frequent remote pairing sessions where two developers from different regions would pair using Virtual Network Computing (VNC), teleconferencing, and the Integrated Development Environment (IDE) were introduced to help create a group mind across regions. This often helped improve collaboration and sharing, avoid misunderstandings, and encourage shared memory between the pairs and, through them, the teams.

- **Round-the-World Program**: Specific local market conditions influence the decisions made by a team within a region. These decisions may not be understood by other regions. The trust relationship across regions eventually breaks down, if misunderstandings are not sorted out, and collaboration becomes impossible. The round-the-world program which rotates members to another region for several weeks was introduced as a solution to this problem. Longer visits of four to six weeks are most effective, though this may not always be possible due to personal constraints or funding limitations.

- **Shared Stories**: The regions were encouraged to work together in an around-the-clock manner for stories that applied across all regions. This encouraged conversation and cooperation which in turn helped build common values and mutual trust and improved the relationship between the regions.

- **Putting Out Fires**: If in case an emergency production issue could not be addressed during one region's working hours, it was handed over to the next region until it was fixed. This approach forced the regions to communicate detailed designs and strengthened the trust level.

- **Coach-to-Coach-Level Communication**: It was essential for the coaches to maintain a constant level of communication and synchronization. Coaches talked to each other after the daily handovers, and all coaches met weekly through teleconference to ensure that any general or cross-regional issues were addressed and resolved. The coaches met in person every quarter, for two weeks to address more complex issues and conduct long-term planning for the quarter.

- **XP Principles and Practices**: XP principles and practices helped the distributed team to maintain a certain level of common values and knowledge. As XP encouraged open communication and collaboration, it increased communication across all regions. The XP core principles, such as refactoring and Test-Driven Development (TDD), allow the team to share a common language. However, as these principles and practices may not always understood in the way across regions, regular discussion are required to cultivate a common understanding.

8.7.4 Outcomes of Adopted Solution

- **Balancing the Teams**: In the distributed team environment, the region which had the most number of developers contributed more to the architecture and design of the project and the other regions were forced to accept. This deterred cooperation among the teams. It was important for each region to have the same number of developers and skill level.

- **Introducing Process Changes**: Rules and predefined processes ensures that the team members know what to expect from each other. When one region introduced high-level process changes without discussing it with the other regions, it caused confusion. The collaboration and trust relationship between regions suffered if these kinds of incidents were left unresolved. The team decided that prior agreement from all the coaches was required in order to introduce a major process change.

- **Introducing Innovation**: Introducing architecture changes or new technologies were the most difficult part of the distributed team structure. Usually, this is introduced in one region and then brought into the code base, where the other regions support it. A lack of consensus made the integration difficult. Getting feedback and approval from the whole team was tough and required time. Unfortunately, the team still has no perfect solution for this problem.

- **Allowing Process Flexibility**: Each region requires a certain amount of process flexibility due to its unique culture. This flexibility is needed to balance between the requirement for a global process and local adaptation.

- **Outside Forces**: The teams were managed by their local offices and were treated as local resources before they were combined into a global team. Most of the team members did not follow a formal development process. After becoming a global team, regional offices had to engage in planning and global story prioritization. As this called for an uninterrupted weekly plan, it was not well received by the local teams who worked in an ad-hoc fashion. The company gradually developed a process for building a consolidated monthly product backlog, known as the Company Program Plan.

- **Including the Business**: At the beginning of each week, the business managers worked with the team to plan and prioritize stories. Following a fixed plan for a whole week was initially tough for the team to accept as they were used to making changes on a daily basis. During the iteration, the coaches and business managers worked on planning and prioritizing new stories based on new business requirements.

- **Incorporating QA**: Prior to using XP, the team did not use automated user tests, and in some cases the new software was even released without customer signoff. As quality assurance is an essential part of the XP process, the team adopted a user acceptance testing framework, Selenium by Thought Works, and trained a non-technical business team to work with customers to automate user acceptance tests. This effort is still in the initial phase, but the team is already seeing positive results.

8.7.5 Conclusion

After a year, the company considers the transition to the globally distributed XP team to be successful. The team has proved that XP works for a globally scattered group developing around-the-clock with a shared codebase. With distributed XP, the company enjoys the advantages of an Agile process: the developers have direct contact with regional customers, and the business is able to accommodate changes in the local and global markets while producing and delivering quality products. Even though there is still much scope for improvement, the team is already on the right track towards success.

8.8 Check Your Progress

1. Which of the following people prioritize user stories?

(A)	Manager	(C)	Team
(B)	Customer	(D)	Developers

2. The _____ phase decides the priority order for the stories.

(A)	Exploration	(C)	Iterations to Release
(B)	Planning	(D)	Productionizing

3. Which of the following artifacts has textual descriptions and automated test cases developed by the customer?

(A)	CRC cards	(C)	Customer acceptance tests
(B)	Visible wall graphs	(D)	User story cards

4. The _____ teaches the XP process to the team and monitors whether it is being followed.

(A)	Coach	(C)	Tracker
(B)	Manager	(D)	Customer

5. XP processes concentrates on _____.

(A)	Software development	(C)	Requirement of developers
(B)	Project management	(D)	Coaches

8.8.1 Answers

1.	B
2.	B
3.	C
4.	A
5.	A



Summary

- XP concentrates on the development rather than managerial aspects of software projects. It stresses customer satisfaction and teamwork.
- In XP, every individual contributor to the project is an indispensable part of the whole team.
- XP is based on five values namely, Communication, Simplicity, Feedback, Courage, and Respect.
- The different phases in XP are namely, Exploration Phase, Planning Phase, Iterations to Release Phase, Productionizing Phase, Maintenance Phase, and Death Phase.
- The important documents and artifacts in XP include: User story cards, Task list, CRC cards, Customer acceptance tests, and Visible wall graphs.
- The important roles in an XP team are Manager, Coach, Programmer, Tester, and Customer.
- The advantage of XP processes are its robustness, resilience, cost savings, lesser risks, and employee satisfaction.

Tutor Chat

Onlinevarsity
your e-way to learning



are just a click away



To chat with a

login to **www.onlinevarsity.com**

Session - 9

Scrum

Welcome to the Session, **Scrum**.

This session explains the basic principles of Scrum, its characteristics, principles, and values. It also explains the Scrum process, life cycle, documents, and the roles in a Scrum team.

In this Session, you will learn to:

- Describe the characteristics, rules, principles, and values of Scrum
- Explain the Scrum life cycle and events
- Describe the different roles in a Scrum team
- Describe the documents and artifacts used in Scrum
- Explain the advantages of using Scrum



9.1 Introduction to Scrum

Scrum is an Agile framework that lets you create your own lightweight process for developing new products. Since the early 1990s, Scrum has been used to manage complex product development. It is not a process or a technique for building products, but is a framework within which various processes and techniques can be employed. Scrum makes it easier for development practices and product management to improve.

Scrum has been defined as "A framework within which people can address complex adaptive problems, while productively and creatively delivering products of the highest possible value."

9.1.1 Scrum Framework

Scrum is a team process that begins when stakeholders make a request for a product. The team builds the product incrementally in short sprints. A sprint is for fixed time period with a maximum duration of four weeks, but with preference towards a shorter duration. The Scrum team builds and delivers a product increment during each sprint. Each increment is a recognizable, visually enhanced, operating subset of the final product.

Scrum encourages transparency with the stakeholders and within the team. This leads to visible displays of plans and progress. Scrum binds together events, roles, and artifacts, managing the relationships, and communication between them.

Scrum includes three essential artifacts:

- **Product backlog**: List of requirements for the product, in the order in which they are to be built.
- **Sprint backlog**: Detailed plan for development during the next sprint.
- **Product increment**: An integrated high quality version of the product which is ready to be delivered at the request of the product owner.

9.1.2 Scrum Theory

Scrum is based on the empirical process control theory, or empiricism. This theory states that all the basis of all knowledge is experience derived from what is already known. Scrum uses an incremental, iterative approach to improve consistency and control risk.

Each implementation of empirical process control is guided by the following factors:

- **Transparency**: Important aspects of the process must be evident to the responsible people. Transparency requires significant aspects of the process to be defined by a common standard so that observers share a common understanding of what is being seen.
- **Inspection**: The team needs to inspect Scrum artifacts and progress towards a goal to identify unwelcome diversions. Inspections should not be too frequent that it gets in the way of work.

They are best performed by skilled inspectors at the point of work to identify unacceptable variances.

- **Adaptation**: If a process deviates from acceptable limits, it is adjusted quickly in order to prevent it from deviating any further.

Four formal opportunities for inspection and adaptation in Scrum are:

- Sprint Planning Meeting
- Daily Scrum
- Sprint Review
- Sprint Retrospective

9.1.3 Essence of Scrum

The essence of Scrum is:

- The team is given clearly defined goals.
- The team organizes itself around the work.
- The team routinely delivers the most valuable features.
- The team receives constant feedback from the customers.
- The team reviews the work done in order to improve on its way of working.
- The entire organization has visibility into the progress of the team.
- The team and management honestly communicate about progress and risks.

9.1.4 Rules and Characteristics of Scrum

Scrum follows some common rules:

- Always have a product that is ready enough to be despatched.
- Have a common language at a single development site.
- Continuously test the product as you build it.

Characteristics of Scrum are as follows:

- Breaks down large products into smaller manageable chunks.
- Enables the project to proceed systematically even when the team is not able to determine a complete product design at the beginning of the project.
- Facilitates large teams to divide work among multiple small teams developing projects in parallel, but synchronizing continuously, stabilizing in increments, and continuously finding and fixing issues.
- Facilitates competition based on customer feedback, and short development times by incorporating customer inputs, setting priorities, completing the most important requirements first, and then changing less important features.

9.2 Scrum Principles

Scrum is the most popular of the Agile frameworks. The Agile Manifesto values apply directly to Scrum. They are as follows:

- **Individuals and interactions over processes and tools**: Like all the Agile frameworks and methods, Scrum relies directly on trust between the individuals in the teams and their interaction. Teams take decisions on the course of action and then work towards their goals. They also identify the difficulties in their path and resolve the issues which come under their scope. Teams also sort out issues with other people in the organization who are outside their team. Thus, the primary focus of Scrum is team responsibility and autonomy.
- **Working software over comprehensive documentation**: The output of each sprint is a working, finished increment of the product. Even though the documentation for the analysis, design, and testing, will have to be done, it is the working software which is the most important output for the team and the organization.
- **Customer collaboration over contract negotiation**: The Scrum team's prime point of contact is the Scrum product owner with the end users of the product. The product owner is also a member of the Scrum team and works collaboratively with the team to make decisions. The product owner prioritizes the tasks and ensures that the product has the highest possible value all the time.
- **Responding to change over following a plan**: The Scrum process is designed in such a way that the information needed to make good decisions is easily available. The progress of the project is represented by a working product increment at the end of each sprint. The overall progress and the progress between sprint to sprint is clear. The backlog to be done is available for all to see. Problems and concerns are discussed openly and dealt with immediately. Scrum is a good option for teams that openly inspect the process and adapt their actions to the reality.

9.2.1 Scrum Values

The five values on which the Scrum team's process and principles are founded are:

- **Focus**: The team works well together, produces outstanding results, and delivers valuable items sooner since it focuses on only a few things at a time.
- **Courage**: The Scrum team always has a lot of resources at their disposal and feels supported always which gives them the courage to undertake bigger challenges.
- **Openness**: The Scrum team communicates to each other about their progress and the challenges that they face. Expressing concerns helps the team sort them out faster.
- **Commitment**: The Scrum team is very committed to success since they are in control of their process.
- **Respect**: As the team works together sharing successes and failures, the members come to respect each other and help each other become worthy of respect.

9.3 Scrum Process

The Scrum process consists of the following phases:

- Pregame
- Game
- Postgame

9.3.1 Pregame

Planning is the process of defining a new release based on currently known backlog with an estimate of its schedule and cost. When a new system is being developed, the planning phase consists of both conceptualization and analysis. For the enhancement of an existing system, the planning phase consists of limited analysis.

Steps in Planning:

- Preparation of a comprehensive backlog list.
- Definition of the functionality and delivery date of one or more releases.
- Prioritizing the functionalities according to importance.

- Mapping of product packets for backlog items in the selected release.
- Definition of project teams for the development of the new release.
- Assessment of risk and risk controls.
- Review and modification of backlog items.
- Estimation of release cost, including development, marketing, training, rollout, and collateral material.
- Verification of infrastructure and development tools.
- Confirmation of management approval and funding.

The Architecture phase includes system architecture modification and high level design.

Steps in Architecture:

- Review assigned backlog items.
- Identify changes necessary to implement backlog items.
- Analyze domain to develop, modify, or update domain models to reflect the new system context and requirements.
- Modify the system architecture to support new requirements.
- Identify issues in developing or implementing the changes.
- Design review meeting, with each team putting forth their strategies and plans to implement the backlog items.

9.3.2 Game

A development sprint is the development of new functionality, with respect to the variables of time, requirements, quality, cost, and competition. There are multiple iterative development sprints used in the evolution of the system.

- **Steps in Development Sprint**:

  In the development phase, development work is done in an iterative cycle. The management determines whether the time, competition, quality, or functionality are met and the iterations are completed. This approach is also known as Concurrent Engineering.

Development consists of the following processes:

- Meeting with teams to review release plans.
- Review and improvement of the standards that the product needs to adhere to.
- Plan iterative Sprints, until the product is ready for despatch.

Risk is continuously assessed and sufficient risk controls and responses are put in place. Each Sprint consists of the following steps:

➔ **Develop**: Defining changes needed for implementing the backlog requirements into packets, opening and analyzing them, designing, developing, implementing, testing, and documenting the changes. Development consists of discovery, invention, and implementation.

- **Wrap**: Closing the packets and creating an executable version of the changes to implement backlog requirements.
- **Review**: All involved teams meeting to present their progress, raising and sorting out issues, and updating the backlog list. Risk is also reviewed and appropriate action taken.
- **Adjust**: Including the information from the review meeting into appropriate packets, and including new properties as needed.

Each Sprint is followed by a review, where:

- The whole team and product management participate.
- Customers, sales, marketing, and others may participate.
- Functional and executable systems that include the objects assigned to that team and include the changes made to implement the backlog items are reviewed.
- New requirements may be introduced and assigned to teams.
- The time of the next review is decided based on progress and complexity.

9.3.3 Postgame

The preparation for release, final documentation, pre-release testing, and release happens during the closure phase.

The management team declare a release 'closed' when the variables of time, competition, requirements, cost, and quality meet the requirements. Closure includes tasks such as integration, system test, user documentation, training material preparation, and marketing material preparation.

9.4 Scrum Life Cycle and Events

Scrum uses time-boxed events such that each event has maximum duration fixed. This ensures that no more than the fixed amount of time is wasted in planning. Other than the sprints, each event in Scrum is an opportunity to inspect and modify something. These events are also specifically designed to enable inspection and transparency. The time boxes are as shown in figure 9.1.

The time boxes in Scrum are:

- Release Planning Meeting
- Sprint
- Sprint Planning Meeting
- Daily Scrum
- Sprint Review
- Sprint Retrospective

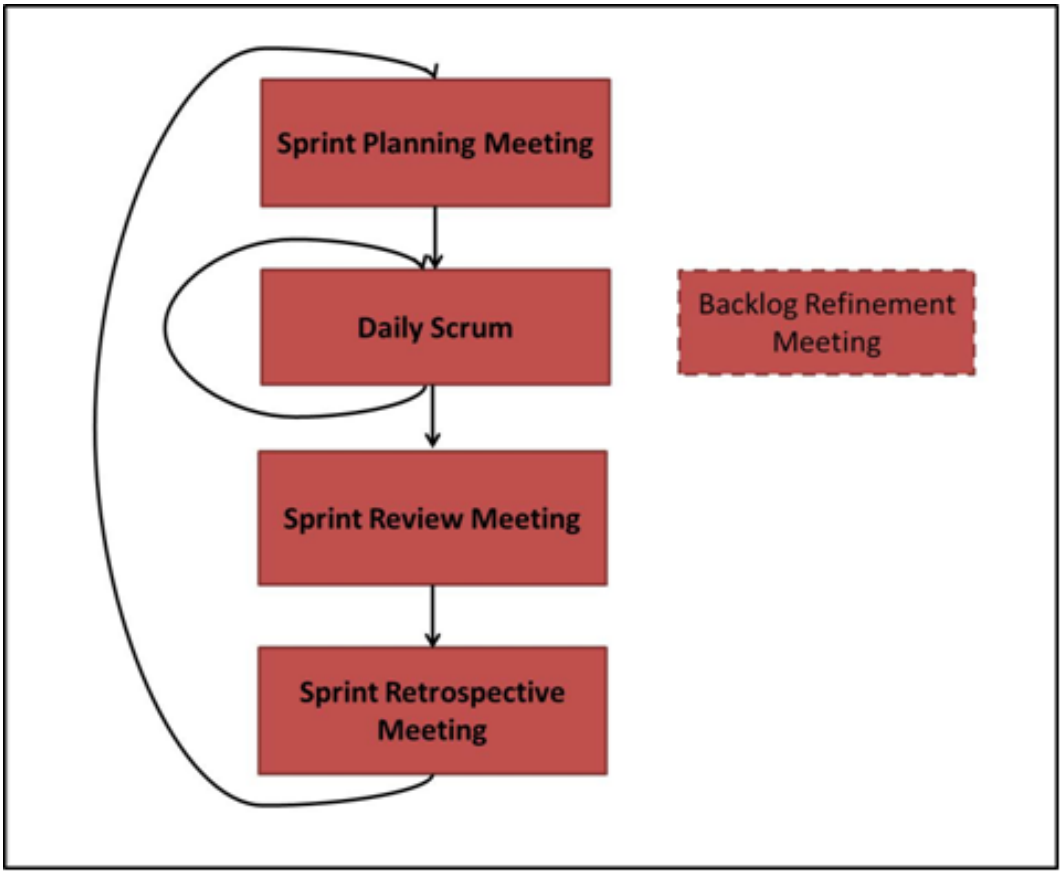


Figure 9.1: Events in Scrum

9.4.1 Release Planning Meeting

The release planning meeting establishes a plan and goals that can be understood and communicated by the Scrum teams and the rest of the organization. Release planning answers the questions:

- Which is the best possible way to turn the vision into a quality product?
- How can customer satisfaction be met or exceeded?

The release plan includes the following:

- Goal for the release
- The product backlog with the highest priority
- Major risk factors for the release
- Overall product functionality and features for the release
- Approximate delivery date and cost for the release

The organization then monitors the progress and alters the release plan on a Sprint-by-Sprint basis.

Starting with the most important and the riskiest, Scrum uses iterative development. Each Sprint creates an increment of the product where each increment is a potentially shippable portion of the entire product. The product is released when enough increments have been created for it to be of value.

Probable outcomes and overall goals are defined in the Scrum Release Planning. Estimation and prioritization of the product backlog for release is also done here.

9.4.2 Sprint

Sprint is a repeatable work cycle during which a useable and potentially shippable product increment is created. The duration of a sprint is usually one month or less. A new Sprint begins immediately after the conclusion of the previous Sprint. Sprints normally have similar durations throughout a project.

Sprints consist of:

- Sprint Planning Meeting
- Daily Scrums
- Development work
- Sprint Review

- Sprint Retrospective

During the course of a Sprint:

- The development team remains unchanged
- No changes that would affect the Sprint goal are made
- Quality goals remain unchanged
- Scope of the project may be clarified and worked out between the product owner and development team as understanding improves

Each Sprint is considered a project with a duration of no more than a month. The increment to be built, the design, and the plan to be followed, are all defined in a Sprint.

Sprints enable predictability by ensuring analysis and adaptation of progress towards a goal at least once every calendar month. Sprints also limit risk to one calendar month. When a Sprint duration exceeds one month, the definition of the product being developed changes, complexity arises, and the risk involved increases.

A Sprint can be cancelled if the Sprint goal is discarded. Only the product owner has the authority to cancel a Sprint before the Sprint duration is over. The goal may be discarded if the market or technology conditions change or if the company changes its direction. In general, a Sprint should be cancelled if it no longer makes sense under the circumstances. However, this rarely happens due to short duration of Sprints. When a Sprint is cancelled, any completed product backlog items are reviewed. The Product Owner typically accepts any work that is potentially releasable. The incomplete items on the product backlog are re-evaluated. Since, the work done on them depreciates quickly they must be re-estimated frequently. Sprint cancellations are uncommon since they consume a lot of resources, as a new Sprint will have to be started from the beginning.

9.4.3 Sprint Planning Meeting

The purpose of the Sprint Planning Meeting is to plan the work to be performed in the next Sprint. The entire Scrum team collaborates to create this plan. The Sprint Planning Meeting is time-boxed to eight hours for a one-month Sprint and is proportionately shorter for shorter Sprints. The Sprint Planning Meeting consists of two parts, and answer the following questions, respectively:

- What will be the deliverable in the upcoming Sprint?
- How will the work get done?

In the first question the product owner presents the prioritized product backlog items and the entire development team collaborates on understanding the work to be done.

The main topics of interest in this meeting are the product backlog, the previous increment, predicted capacity of the development team, and the past performance of the development team.

The development team selects the items to be developed from the product backlog since they are the best people to judge what they can accomplish.

After this is done, the Scrum team creates a Sprint Goal. The Sprint Goal is the objective to be met within the Sprint through the implementation of the product backlog, and it serves to provide guidance to the development team.

In the second question, after selecting the work to be done during the Sprint, the development team then decides how it will build this functionality into a completed product increment. The product backlog items selected for the particular Sprint along with the plan for delivering them is called the Sprint Backlog.

The development team begins by designing the system. Enough work is planned during the Sprint Planning Meeting and the team assesses the work that can be done in the upcoming Sprint. The planned work is divided into units of one day or less by the end of the meeting. Both during the Sprint Planning Meeting and throughout the Sprint, the development team self-organizes to carry out the work.

The Product Owner may be present during the second part of the Sprint Planning Meeting to clarify any issues with the chosen product backlog items. The development team may also renegotiate the Sprint backlog items with the product owner if it feels that it has too much or too little work. The development team may also invite technical or domain experts for their advice. By the end of the meeting, the development team should be able to explain to the product owner and the Scrum master how it intends to work as a selforganizing team to develop the increment and realize the Sprint goal.

9.4.4 Daily Scrum

The Daily Scrum is a 15 minutes time-boxed event for the development team to synchronize its activities. The team creates a plan for the current day by inspecting the work completed since the last Daily Scrum and estimating the work that can be finished before the next one. The Daily Scrum is held at the same place and time each day, in order to avoid complexity.

During the meeting, each member explains:

- The work completed since the last meeting.
- The work that will be done before the next meeting.
- The obstacles in the way, if any.

The Daily Scrum is used to assess the work completed in the Sprint backlog and the progress towards the Sprint Goal. On a daily basis, the development team should be able to explain how it intends to work together as a self-organizing team to realize the goal of creating the planned increment in the remaining duration of the Sprint, to the Product Owner and Scrum Master.

Even though the Scrum Master ensures that they have Daily Scrum and have it under the 15 minutes timebox, it is the responsibility of the development team to have it daily. Daily Scrums improve communication, eliminate the need for other meetings, identify and remove obstacles to development, highlight and promote quick decision-making, and improve the team's knowledge of the project.

9.4.5 Sprint Review

A Sprint Review is an informal meeting held at the end of the Sprint to review the product increment and modify the product backlog if required. For one-month Sprints, it is four- hours' time-boxed meeting. During the Sprint review, the Scrum team and stakeholders discuss the work that was done in the Sprint and decide the course of action for the next Sprint. The team presents the product increment to the stakeholders and gets their feedback. The product backlog is revised at the end of the Sprint Review. The product backlog may also be adjusted to accommodate new opportunities.

The Sprint Review includes the following elements:

- The product owner identifies the tasks done and the ones not done, if any.
- The team discusses the aspects that went well during the Sprint, the hurdles that were encountered, and how they were solved.
- The development team demonstrates the product increment and answers questions about it.
- The product owner discusses the status of the product and projects likely completion dates based on progress so far.
- The entire group collaborates and decides the activities that go into the next Sprint.

9.4.6 Sprint Retrospective

The Sprint Retrospective provides an opportunity for the Scrum team to review and create plans for improvement during the next Sprint. The Sprint Retrospective happens after the Sprint Review and prior to the next Sprint Planning Meeting. For one-month Sprints, the Sprint Retrospective is three-hours time-boxed meeting and is proportionately shorter for shorter Sprints.

The purpose of the Sprint Retrospective is to:

- Review the activities of the previous Sprint with regards to people, relationships, process, and tools.
- Identify the potential improvement areas and the major aspects that went well in the previous Sprint.
- Devise a plan for implementing process improvements.

The Scrum Master encourages the Scrum team to improve its development process and practices, within the Scrum process framework, to make it more effective for the next Sprint.

The Scrum team comes up with new techniques to improve the quality of the product and identifies improvement areas during each Sprint Retrospective. The Scrum life cycle is shown in figure 9.2.

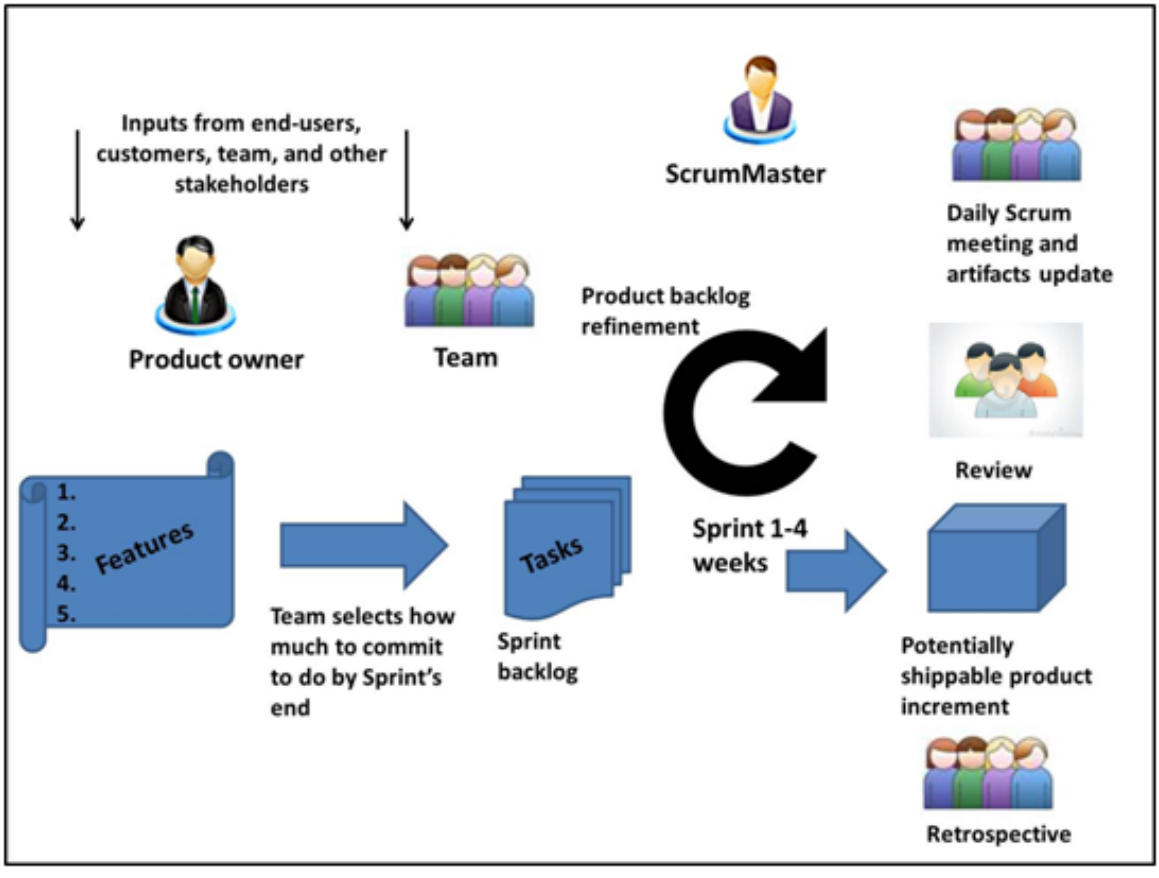


Figure 9.2: Scrum Life Cycle

9.5 Roles in Scrum

The Scrum team consists of:

- Product Owner
- Development Team
- Scrum Master

Scrum teams are cross-functional and self-organizing. Self-organizing teams choose how best to organize and accomplish their work. Cross-functional teams have all the competencies required to accomplish the work without depending on people outside the team. The team model in Scrum is improves productivity, flexibility, and creativity.

➔ **The Product Owner**: The product owner is the only person responsible for managing the product backlog. The product owner is also responsible for the work of the development team and maximizing the value of the product.

Product backlog management includes:

- Distinctly listing the product backlog items.
- Prioritizing the items in the product backlog in order to achieve best results.
- Ensuring that the product backlog is visible, transparent, and clear to all, and reflects the status of development.
- Ensuring that the development team understands the items in the product backlog.
- Ensuring the value of the work done by the development team.

The product owner remains accountable for the backlog management even if the management of the backlog is done by the development team. The product owner is a single person and not a committee. The team has to convince the product owner if they want to change the priority of a backlog item.

➔ **The Development Team**: The development team is responsible for developing and delivering a potentially releasable increment at the end of each Sprint. Organizations empower the development teams to organize and manage their own work. This improves the overall efficiency of the development team.

Development teams have the following characteristics:

- They are self-organizing.
- They are cross-functional and possess all the skills necessary to create a product increment.
- Regardless of the work performed by the person, all development team members are referred to as developers.
- Accountability is by the team as a whole even though individual team members may have specialized skills and areas of focus.
- Development teams are not further divided into sub-teams for testing or business analysis.

The ideal development team is big enough to complete significant work and small enough to be agile. Fewer than three development team members decreases interaction and may face skill constraints which might make it difficult for them to deliver the increments. Big teams require a lot of coordination and are too complex for an empirical process. The team count does not include the Product Owner and the Scrum Master.

➔ **The Scrum Master**: The Scrum Master is a servant-leader for the Scrum team. The responsibility of the Scrum Master is to ensure that the Scrum team follows the Scrum practices and rules.

The Scrum Master serves the Product Owner in several ways, including:

- Communicating vision, goals, and product backlog items to the development team with clarity
- Finding effective strategies for product backlog management
- Educating the Scrum Team to create clear and compact backlog items
- Practicing agility
- Promote Scrum events as needed or requested

The Scrum Master serves the development team in several ways, including:

- Teaching self-organization and cross-functionality to the development team
- Coaching and leading the development team in creating products of high-value
- Removing obstacles that hinder the progress of the team
- Facilitating Scrum events as needed or requested
- Coaching the development team in environments where Scrum has been newly introduced

The Scrum Master serves the organization in several ways, including:

- Leading and coaching the organization in Scrum
- Arrange for Scrum implementations within the organization
- Helping employees and stakeholders understand and work on Scrum
- Bringing about alterations that increases the productivity of the Scrum Team
- Working with other Scrum Masters to further promote the effectiveness of Scrum in the organization

9.6 Documents and Artifacts

Scrum artifacts represent work or value in different ways that are useful in providing transparency and opportunities for inspection and adaptation. Scrum artifacts are specifically designed to maximize transparency of vital information needed to ensure successful delivery of Scrum increments.

➔ **Product Backlog**: The product backlog is an ordered list of the requirements and the features to be included in the product. It is the single source of reference for any changes to be made to the product. The product backlog lists all features, functions, requirements, enhancements, and fixes that need to be made to the product in the future. The product backlog is ordered by value, risk, priority, and necessity. The main characteristics of product backlog items are their description, order, and estimate.

➔ The product owner is responsible for the product backlog, including its content, availability, and ordering.

The product backlog starts with only the initial and best-understood requirements. It evolves as the product and the environment in which it will be used evolves. The product backlog constantly changes according to product's need to be appropriate, competitive, and useful. The product backlog exists as long as a product exists.

A product backlog can be called a living artefact as requirements keep getting updated. Changes in requirements, market conditions, or technology may cause changes in the product backlog.

Product backlog grooming is the act of adding order, detail, and estimates to the items in the product backlog. This is done as a part-time activity during a Sprint and is an ongoing process in which the development team along with the Product Owner reviews and revises the items in the backlog. The Product Owner may influence the development team by helping understand and select trade-offs, however, the development team is responsible for all estimates. The items in the backlog can be updated at any time by the Product Owner or at the Product Owner's discretion.

The Product owner tracks the total work remaining to reach the goal at each Sprint Review and compares this amount with the work remaining at previous Sprint Reviews to assess progress. This information is made transparent to all stakeholders.

➔ **Sprint Backlog**: The Sprint backlog is the set of product backlog items that have been selected for the current Sprint along with a plan for developing and delivering the product increment and meeting the Sprint Goal. It is a forecast of the work that the development team plans to accomplish during the next Sprint.

The Sprint backlog is a representation of work that has to be done by the development team in order to meet the Sprint Goal. The development team modifies the Sprint backlog throughout the Sprint and new requirements are added to it. The estimated remaining work is updated as and when some work is completed. Unnecessary elements in the plan are removed. Only the development team can change its Sprint backlog during a Sprint.

The development team tracks the work remaining in the Sprint backlog on a daily basis and projects the possibility of achieving the Sprint Goal. The development team manages its progress this way. The time spent working on Sprint backlog items is not considered and the only variables of interest are the date and the remaining work.

The Sprint backlog is represented in figure 9.3.

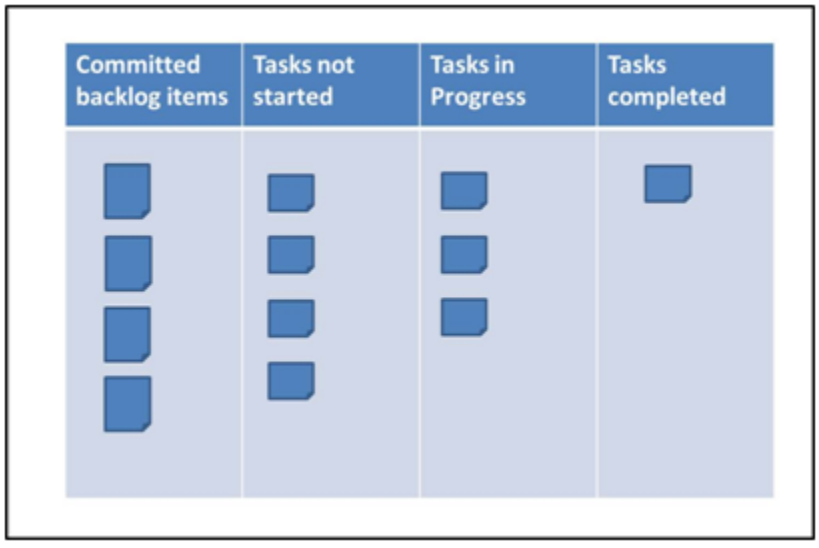


Figure 9.3: Sprint Backlog

➔ **Sprint Burndown Chart**: The team monitors its progress using the sprint burndown chart. It is a leading indicator of whether it will meet its goals at the end of the Sprint. The format requires teams to estimate on a daily basis the duration of each task in hours to chart the total remaining hours for the tasks not completed.

➔ **Product or Release Burndown chart**: The product burndown chart is also known as the release burndown chart and measures the team's velocity which is the rate of delivery of a stream of running, tested, features over time. As features vary in complexity, effort, and time, a scale known as story points is used to compare their size. The velocity of a team, who have worked together for a few sprints, can be established within a definable range. Product Owners use velocity to predict the output rate of the team for future deliverables. This results in credible release plans. The Product Owners thus use this chart to report progress, determine release dates, and predict release scope.

➔ **Impediment Backlog**: The impediment backlog is a current list of things that prevent the team from progressing or improving. A good Scrum Master will try to remove these obstacles within 24 hours of them being identified in order for the team to achieve best results.

9.7 Advantages of Scrum

Traditional development methodologies are only designed to respond to the unpredictability of the development and external environments at the beginning of the cycle. Many new approaches have limited flexibility to respond to changing requirements once the project begins. The Scrum methodology, on the other hand, is designed to be flexible throughout.

Scrum provides mechanisms for planning a product release and for managing the variables as the project progresses. This enables organizations to deliver the most appropriate releases by making appropriate changes at any point in time. Scrum methodology allows developers to devise the most innovative solutions, as the environment changes and more learning happens. Small, collaborative teams of developers are able to share knowledge about development processes.

9.8 Case Study

The success or failure of any project depends on how the Scrum process is adapted to address specific needs and circumstances. This case study describes how a large Scrum project which was once already scrapped under a traditional approach was successfully executed. This project included 20 man-years and 100,000+ lines of code and included developers from different countries. This case study provides lessons on project startup, importance of estimates, getting the right product owner, effective communication, testing, and documentation.

➔ Background

The Dutch railways provide transport for 1.2 million passengers daily, and are among the most heavily used in the world. Dutch Railways built a new information system to provide travelers with more accurate travel information, and which requires less manual intervention. As part of this program, they built the PUB (publish) system that centrally controls information displays and audio broadcast systems in all stations.

The PUB system was first attempted to be built using a traditional waterfall approach. The requirement specifications were handed over to the IT vendor who was expected to develop a fully built system not requiring much customer involvement. The project had to be cancelled after three years as the vendor failed to deliver a working system. The project was then handed over to another vendor who had to build the PUB system from scratch. The new vendor introduced an Agile approach using Scrum, which focused on open communication, close cooperation with the customer, and delivering working software in small increments.

➔ Initialization

The project was started with a project kick-off to set up everything that is required before start of the first sprint. This 3-week kick-off was done by a project manager, an architect, and a Scrum Master.

Selecting a Product Owner was a challenge as it was tough to find a single person that had the domain knowledge, time, and the mandate to prioritize requirements. Two business analysts were nominated to fill the product owner role. They could be made available and they had built up a lot of business knowledge because of their involvement in the project in the first attempt as well. This made it easier for them to be good proxy Product Owners to several groups of customer stakeholders. Major decisions involving scope and priorities were taken by a project manager chosen by Dutch Railways. This project manager did not have an in-depth knowledge about the requirements and was not available throughout.

This model worked well on the whole, but there were occasions when the project manager changed the priorities that had just been set in his absence during the planning meeting and the meeting had to be redone. Ideally, the person who has the final say about priorities should always be present at the sprint planning meetings.

As there was already an earlier attempt build the system, detailed requirements documentation for parts of the system was available. This documentation was however not suited for Agile planning and estimation because they were not grouped into small chunks that could make up a sprint. Also, the Scrum Master helped the Product Owners produce the initial product backlog containing estimated user stories for the first few iterations as they did not have prior experience in it.

The software was part of a bigger program that involved multiple related software systems which was used for building and installing the displays in the stations. Meeting deadlines was important to implement this elaborate effort. An overall picture of the planning had to be created. After a few iterations, this was addressed by making 'best-effort' estimates for the functional areas of the system. By then, the velocity was also determined and so it became easier to track and communicate progress using a release burndown chart. Thus, even if there is only little information available, any estimate is better than no estimate.

➔ Distributed Teams

The project started with a team of seven people working in two weeks iterations. Two Indian developers joined the project and worked at the customer site from the first sprint onwards for six weeks. They familiarized themselves with the application domain, the development team, and with key customer representatives.

In order to facilitate working together, they held a norming and chartering session with all team members. Practices like how to do pair programming, use of tools, core hours, quality targets, and so on were decided and recorded on a Wiki page. The agreements were shared by the whole team and practices were updated when there was a change in the agreements.

In the first iterations, the team built, tested and demonstrated user stories that formed the heart of the system. This customer was pleased with this especially because results were seen easier and much faster compared to the previous attempt. The customer was also impressed with the fact that they had more control over the project.

After a couple of iterations, the Indian developers returned home. Two Scrum teams of five developers each created in India and the Netherlands with a common tester. This was later modified to three development teams with three testers. Each Scrum team consisted of resources from both India and the Netherlands. This model was found to be very successful in maintaining good quality and high velocity.

Skype was used intensively using headsets, webcams, good microphones, and a large screen. This helped them work as one team from several locations. Skype was used for meetings that involved the whole team and to do one-on-one meetings as well. No specific investments were required for this since they were off the shelf components and free software.

Only, some enhancements were needed for the network facility in India using UPS to make sure that there was no discontinuity in the Skype sessions during power outages. Within the same location, pair-programming was followed.

Since the interactivity of pair programming required the coding pair to be present in the same location, regardless of tooling available, there were pairs in India and in the Netherlands.

The ScrumWorks tool was used since the teams were in different locations. This tool kept track of the progress of the Sprint and had details of who is working on what. ScrumWorks also proved very useful for to discuss the product backlog with the product owners.

There were some challenges involved in the implementation of this distributed model. According to Scrum, the product owner explains the user stories to the team and prioritizes the tasks for the sprint, during the first part of the sprint planning meeting. Since the product owners were not comfortable with English, the meeting was done in Dutch without involving the Indian team members. The second part of a planning meeting consisted of identifying and estimating tasks to implement user stories and was done in English using a Skype session, with the Indian developers, but without the product owners. Extra time had to be spent to communicate the contents of the first meeting to the Indian team mates. As the Sprint demos were held in Dutch, the Dutch team members wrote a newsletter about the demo in order to update the team members in India. This newsletter which was also distributed to the rest of the company turned out to be very popular.

➔ **Separate team focused on architecture**

The project was part of a chain of applications, and had to fit into the Railway's existing IT infrastructure. Even though the product owners possessed in-depth knowledge about the core functional requirements, their knowledge about requirements for security, logging, availability, and performance was limited. Since getting these requirements from the customer organization required meeting up with people from different departments, it was laborious and difficult. To deal with this issue, a separate team with architectural focus was created. This team specialized in getting the 'non-functional' requirements from the customer thus enabling the IT vendor to translate them to user stories for the backlog.

➔ **Documentation**

The customer requested for very extensive documentation in Dutch and which was compliant with the MIL documentation standard. Since this writing could only be done by Dutch resources, a technical writer who had experience with MIL standard and with writing user manuals was hired. This also allowed the developers and testers to concentrate on delivering working software. Even though this model required a lot of communication between the technical writer and the team, it was proved to be successful.

➔ **Requirements**

The Product Owners, who were business analysts, usually wrote extensive requirements documentation in Dutch.

As far as the process was concerned, the requirements were just the user stories on a backlog and the explanations of the Product Owners during the planning meeting. The Dutch Railways, however, required comprehensive requirements documentation. In order to synchronize this with the Scrum process, the requirements were translated into user stories by collaborating with the Product Owners.

This led to requirements being in two places: The requirements documents and the backlog. This lead to confusion when the requirements needed updates.

A lot of effort was required to ensure that the Product Owners took responsibility for the backlog and not only for the requirements documents.

➔ Testing

Automated testing was employed during the project. Even when the system expanded, only one tester was used for the eight-person Scrum team while maintaining high quality.

Automated tests were carried out on two levels: unit tests and acceptance tests. Bugs were usually found and fixed during the Sprint. This approach had another advantage. In this approach, testers created test cases before the user stories were implemented and hence were effective from the beginning of the Sprint.

➔ Outcome

Dutch Railways was satisfied with the output. The success factors were regularly discussed with them during the course of the project and they were satisfied with the outcome. Unfortunately, there were problems with other systems that were part of the nation-wide deployment.

Dutch Railways hired an external company to audit the software. Their conclusions were as follows:

- The maintainability of the system is very good.
- The quality of the source code is very high.

9.9 Check Your Progress

1. _____ is a detailed development plan for the next Sprint.

(A)	Product backlog	(C)	Product increment
(B)	Sprint backlog	(D)	Product burndown chart

2. The _____ is responsible for maximising the value of the product.

(A)	Product owner	(C)	Scrum master
(B)	Development team	(D)	End-user

3. The _____ leads and coaches the organization in its Scrum adoption.

(A)	Product owner	(C)	Project manager
(B)	Customer	(D)	Scrum master

4. The _____ starts with only the initial and best-understood requirements.

(A)	Product backlog	(C)	Product increment
(B)	Sprint backlog	(D)	Daily scrum

5. Which of the following are the phases in Scrum process?

(A)	Pregame
(B)	Game
(C)	Postgame
(D)	Intergame

(A)	A, C, and D	(C)	C and D
(B)	A, B, and C	(D)	Only D

9.9.1 Answers

1.	B
2.	A
3.	D
4.	A
5.	B



Summary

- Scrum is an Agile framework that lets you create your own lightweight process for developing new products.
- The process control in Scrum is based on three factors namely, Transparency, Inspection, and Adaptation.
- The five values on which the Scrum team's process and principles are Focus, Courage, Openness, Commitment, and Respect.
- The Scrum life cycle consists of three phases are namely, Pregame, Game, and Postgame.
- Scrum uses time-boxed events such that each event has maximum duration fixed. This ensures that no more than the fixed amount of time is wasted in planning.
- The Scrum team consists of Product Owner, Development Team, and Scrum Master.
- Scrum artifacts are specifically designed to maximize transparency of vital information needed to ensure successful delivery of Scrum increments.

Session - 10

Agile Process Models - I

Welcome to the Session, **Agile Process Models - I**.

This session explains different types of Agile models such as Adaptive Software Development (ASD), Dynamic System Development Method (DSDM), and crystal.

In this Session, you will learn to:

- Explain ASD model
- Explain the life cycle phases of ASD model
- Explain DSDM
- Explain the principles and techniques of DSDM model
- List and describe the different roles in DSDM model
- Explain crystal model
- Describe the core elements and properties of the crystal method



10.1 Adaptive Software Development (ASD)

Adaptive Software Development (ASD) is a new software development methodology that focuses on rapid creation, application, and evolution of software systems. It was derived from rapid application development and addresses the Internet economy. ASD is characterized by high speed and high change, which introduces a complexity that cannot be handled by traditional approaches. It underlies the Internet and the market of today's dot-com companies. Iterations in this model are called adaptive cycles.

10.2 ASD Life Cycle

The ASD life cycle has the following characteristics:

- Mission focused
- Feature-based
- Iterative
- Time-boxed
- Risk-driven
- Change-tolerant

The ASD life cycle focuses on results which are the application features or customer functionality that is to be developed during iteration. Features evolve over several iterations based on the feedback provided by the customer.

The adaptive life cycle is divided into three main phases:

- Speculate
- Collaborate
- Learn

The adaptive life cycle is shown in figure 10.1.

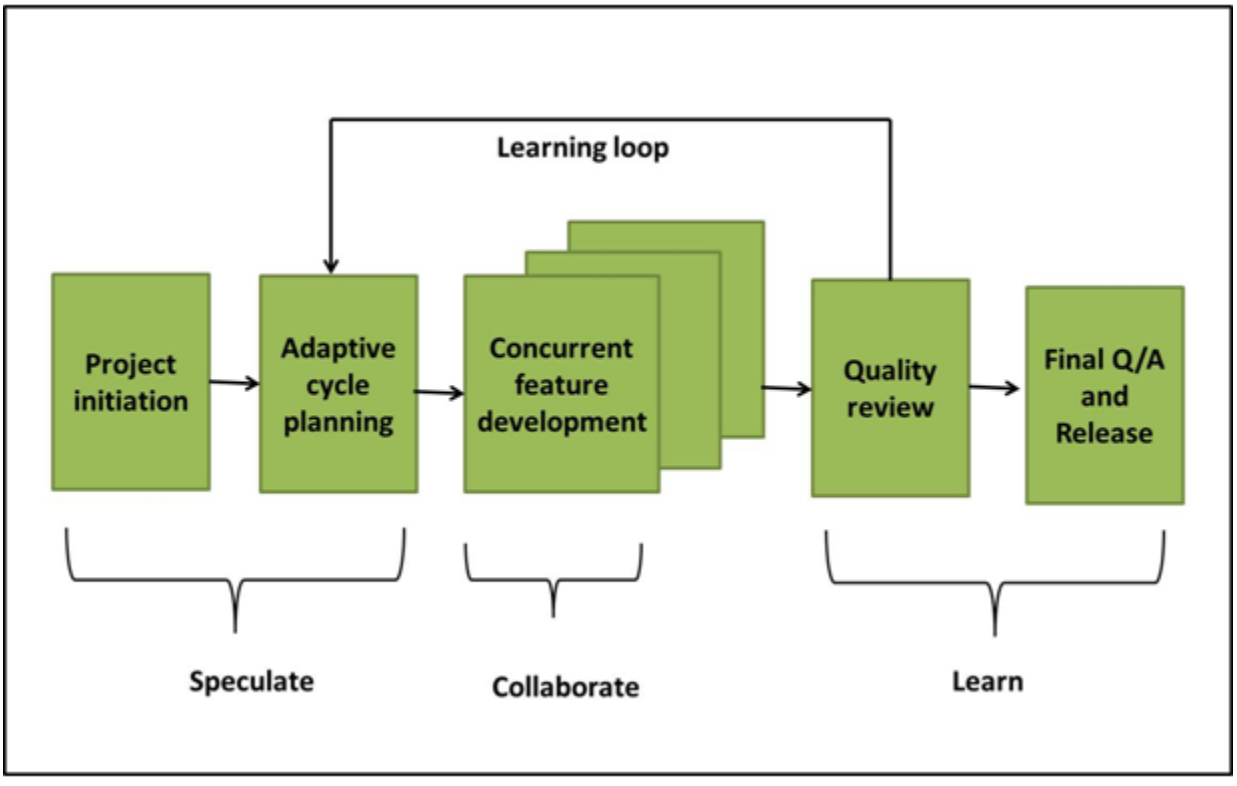


Figure 10.1: Adaptive Life Cycle

10.2.1 Speculate

There are five main steps in the Speculate phase. These steps can be revised several times during the initiation and planning phase.

Project initiation consists of:

- Setting project goals
- Recognizing obstacles
- Making initial scope and size estimates
- Analyzing project risks

Much of the project initiation data is gathered in the preliminary session as speed is a major consideration in ASD. Initiation is completed in two to five days for a small to medium project and in about two to three weeks for a larger project. During this session, requirements are collected in enough detail to identify features and form an architectural model.

Concepts

Next, the time-box for the project is fixed based on the scope, requirements, estimates, and resource availability from the data gathered during initiation.

The next step is to decide the number of iterations and fix the duration for each one. Individual iteration lengths depend on the project size. Iterations vary from four to eight weeks for a small to medium sized project, while others might require more than eight weeks. Projects work best with two-week iterations.

Similar to the Sprint Goal in Scrum, the team now develops a theme for each iteration. Each iteration makes the product visible by delivering a group of demonstrable features to the client.

Testing is an ongoing process and not an activity done at the end. Developers and customers assign the features that are to go into each iteration. The customer decides the priority of the features using estimates, risk, and dependency information from the development team. A spread sheet can be used as an effective tool for feature-based iteration planning. This type of planning done by the team rather than by the project manager, provides a better understanding of the project than a traditional task-based approach.

10.2.2 Collaborate

When the development team works on working software, project managers promote teamwork and other development activities in parallel. For projects in which teams work from the same physical location, collaboration consists mainly of informal chats during breaks and whiteboard scribbling. However, larger projects require collaboration tools and project manager interaction.

Collaboration includes the customer, the development team, outside consultants, and vendors and is based on trust and respect. Teams collaborate on business requirements, technical issues, and decision making.

10.2.3 Learn

In a waterfall development model, people are continually compelled to get things right in the first time itself. Each phase completion also discourages backtracking. Learning becomes difficult in such a case, as there is no time to experiment. Learning from mistakes and experimentation requires the team members to share partially completed code and artifacts early, in order to find errors, learn from them, and to reduce the amount of rework.

There are four important things to learn at the end of each iteration:

- ➔ Result quality from the customer's perspective
- ➔ Result quality from a technical perspective
- ➔ Operation of the development team and the practices used
- ➔ Status of the project

The first priority in adaptive projects is to get feedback from the customers. ASD recommends a customer

focus group for this which focuses on reviewing the application. These sessions explore a working model of the application and record customer change requests. Customers relate best to working software than extensive documentation or diagrams.

The second priority is technical quality. A standard practice for assessing the technical quality is periodic technical reviews. Although pair programming or code reviews are continuous, other reviews such as an overall technical architecture review can be conducted weekly or at the end of an iteration.

The third feedback for the team is the people and process review where the team monitors its own performance. Mini retrospectives at the end of each iteration help to determine what works and what doesn't work and the practices that have to be continued and the ones that can be discontinued. Retrospectives encourage teams to know more about themselves and about how they work together.

The fourth category for review is the status of the project. This leads to planning effort for the next iteration. In a waterfall cycle, the end of each major phase is marked by completed deliverables, whereas in a feature-based approach such as ASD, the end of each iteration is marked by completed features. The basic review questions are:

- Where is the project now?
- Where is it versus the plans?
- Where should it be?

As the plans are hypothetical, it is difficult to establish progress by measuring against them. The customer and the team should continually analyze what they have learned so far and if it changes their perspective on where they have to be.

10.3 Advantages and Disadvantages of ASD

The advantages and disadvantages of ASD are listed in table 10.1.

Advantages	Disadvantages
Iterative-incremental process	Not scalable
Well-worked-out process	Thorough monitoring of the project essential in order to preserve cooperation within and outside the team during the development
Based on structural, functional, and behavioral modeling of the system and the problem domain	Over-dependence on inter-human communication
Special focus on quality assessment and control	No clear-cut design effort
Component-based development	No specific models prescribed
Adaptive process	Physical configuration modeling not encouraged
Process continually improved according to experiences in previous iterations	Lacks formalism

Advantages	Disadvantages
Information gathering and decision making carried out through a number of sessions	
Focus on collaborative environment	
Test-based development	
Continuous integration	
Refactoring used for simplifying the code	
Parallel development of components encouraged thus speeding up the process	
Traceability to requirements through continuous validation and quality review	

Table 10.1: Advantages and Disadvantages of ASD

10.4 Dynamic System Development Method (DSDM)

Dynamic System Development Method (DSDM) is an Agile software development approach that provides a framework for developing and maintaining a system that meets rigid time constraints, through the use of incremental prototyping, in a controlled project management. DSDM being a RAD method, is dynamic. This method is used for systems which are to be developed in short duration and where all the requirements cannot be collected and fixed at the beginning of the project. Here, the design is prepared and developed for whatever requirements are initially known. In DSDM, the requirements evolve with time and the analysis, design, and development phases can overlap. DSDM is:

- ➔ Straightforward
- ➔ Expandable
- ➔ Simple framework based on best principles
- ➔ Not suited for all kinds of projects

10.4.1 Need of DSDM

The goal of DSDM is to deliver working systems to organizations in shorter period of time. A typical DSDM project delivers an operational system within six months. According to DSDM, most projects fail because of people issues rather than technology. Therefore, the main focus of this method is to guide people to work together in such a way that business goals are profitably accomplished.

Following the Agile approach, in DSDM, systems are built for focusing on the current requirements. Therefore, the resulting system is expected to better fit the requirements, be easier to test, and is more likely to be accepted by the customer.

10.4.2 DSDM Principles

There are nine underlying principles in DSDM which includes four foundations and five starting points. They are:

- Active user involvement is essential.
- Teams must be empowered to make decisions.
- The target is frequent product delivery.
- The essential criterion for acceptance of deliverables is the compliance to business purpose.
- Iterative and incremental development is necessary to reach an accurate business solution.
- All changes during development are reversible.
- Testing is integrated throughout the life cycle.
- Requirements are baselined at a high level.
- A cooperative and collaborative approach between all stakeholders is imperative.

10.4.3 Phases in DSDM

The phases in the DSDM model are applicable for both the development of new systems and for change of business processes. The core processes in the project are preceded by the pre-project phase and ends with the post-project phase. The different phases in DSDM are:

- Pre-Project Phase
- Feasibility Study
- Business Study
- Functional Model Iteration
- Design and Build Iteration
- Implementation
- Post-Project Phase

In DSDM, the iterative approach facilitates revisiting of previous steps. Therefore, the current step has to be completed only, just enough to move to the next step, as it can always be completed in a later iteration. According to this approach, the requirements are allowed to change, while the time and resources are fixed. This approach has proved to be useful under varying requirements and time constraints.

➔ **Feasibility Study**: The feasibility of the project for the use of DSDM is studied in this phase. The most important technique used in this phase is workshops. The following questions are answered as pre-requisites for the use of DSDM:

- Can this project meet the required business requirements?
- Is this project suited for the use of DSDM?
- What are the risks involved in this project?

The deliverables for this phase are the Feasibility Report and the Feasibility Prototype that are concerned with the feasibility of the project. The other deliverables are the Global Outline Plan, which is a plan for the remaining project and a Risk Log, which lists the risks that the project faces.

➔ **Business Study**: The business study is an extension of the feasibility study. This stage examines the business processes, the user groups involved, and their requirements. Workshops are very important in this phase too, where the proposed system is discussed by the different stakeholders. The requirements gathered from this session are prioritized and grouped into a requirement list. Based on this prioritization, a development plan is devised for the project. Time-boxing (A time-box usually consists of two to six weeks, within which planned tasks are completed) is used in the development of the plan and is essential in attaining the goals of DSDM, namely, being on time, budget, and guaranteeing quality.

The deliverables for this stage are:

- Business area definition that describes the context of the project within the company
- System architecture definition that provides an initial global architecture
- Development plan that defines the most important steps in the development process

The Risk Log is also brought up-to-date with the risks that are identified in this phase.

➔ **Functional Model Iteration**: In this phase, the requirements that were gathered in the previous phases are converted into a functional model. This model consists of both a functional prototype and models. The use of prototyping makes sure that there is good user involvement throughout the project. The prototype is then reviewed by different user groups. Testing is done in each iteration of DSDM in order to ensure quality.

Functional model iteration is an important part of testing. The functional model can be divided into the following sub-stages:

- **Identify functional prototype**: Identify the functionalities to be implemented in the prototype.
- **Agree on schedule**: Decide on the methods and schedules.
- **Create functional prototype**: Develop the prototype. Investigate, improve, and consolidate it with the combined functional prototype of previous iterations.
- **Review prototype**: The accuracy of the developed prototype is tested by the end-user and the feedback is used to generate the functional prototyping review document.

Functional Model and a Functional Prototype are the deliverables of this stage. Together, these represent the functionalities that can be realized in that particular iteration, ready for testing by users. The Requirements List is then updated, and the items that are completed are deleted. The remaining items are then reprioritized. The Risk Log is also updated after analyzing the risk of further development.

➔ **Design and Build Iteration**: The main focus of this stage is to integrate the functional components from the previous phase into one system. It also addressed the non-functional requirements of the system being built. Testing is an important ongoing activity in this stage too. The design and build iteration can be divided into four sub-stages:

- **Identify design prototype**: Identify the functional and non-functional requirements that are to be in the tested system.
- **Agree schedule**: Decide on the methods and schedules for the development of the requirements.
- **Create design prototype**: Create a system that can be handed over to the end-users. They investigate, improve, and consolidate the prototype of the current iteration with the previous iterations.

The main deliverables for this stage are:

- A design prototype that is tested by the end users at the end of the iteration. The tested system is then handed over to the next phase.
- User documentation

➔ **Implementation**: In the implementation phase, the tested system and the user documentation is handed over to the users. The future users are trained.

The implementation stage can be subdivided into four sub-stages:

- **End-user approval and guidelines**: End-users approve the tested system. Guidelines with respect to the implementation and system usage are created.
- **Train users**: Train future end-users on how to use the system.

- **Implement**: Implement the tested system at the end user's location.
- **Review**: Review the impact of the implemented system on the business and ensure whether the system meets all the goals which were initially fixed.

The DSDM phases are shown in figure 10.2.

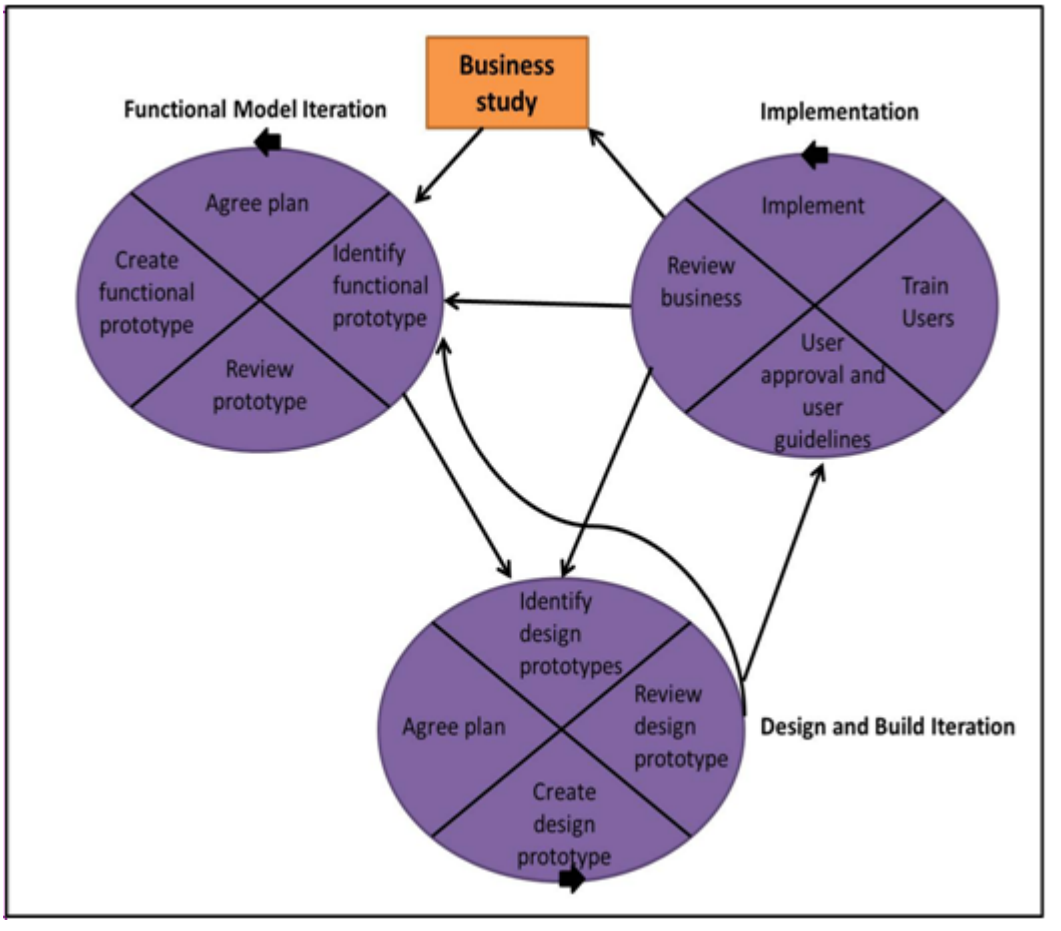


Figure 10.2: DSDM Phases

The DSDM phases and their activities and products are summarized in table 10.2.

Phase	Activity	Product
Pre-Project	Ensures that the right projects are started and setup correctly. Performs the initial project planning for the feasibility study.	No defined DSDM products
Feasibility Study	Checks the feasibility of the model for the current project. Defines the costs, the issues to be addressed, the costs, and technical feasibility.	Feasibility Report, Feasibility Prototype Outline Plan Risk Log

Concepts

Phase	Activity	Product
Business Study	Focuses on the affected business processes and their information requirements. **Deliverable**: Business Area Definition.	Business Area Definition Updated Risk Log System Architecture Definition Development Plan Prioritised Requirements List
Functional Model Iteration	Refines the business aspects of the system building on the high-level processing and information requirements identified in the business study.	Functional Model Review Records Non-Functional Requirements List Implementation Plan Risk Log Time-box Plans
Design and Build Iteration	The system is fine-tuned to high standards to be delivered to the users. **Deliverable**: the Tested System.	Time-box Plans Design Prototypes Design Prototyping Review Records Tested System, Design documentation, and supporting Test Records
Implementation	Transferring the system to the operational environment from the development environment. Training to the users. Deliverable: Increment Review Document.	Delivered System together with supporting build, delivery, and acceptance records Trained User Population Increment Review Document
Post-Project	Maintenance	User Documentation Post-Implementation Review Report

Table 10.2: DSDM Phases, Activities, and Products

10.4.4 Roles in DSDM

The different roles in the DSDM team are:

- **Executive Sponsor or Project Champion**: This is a person from the customer's organization who has the ultimate power to make decisions. The project champion is also responsible for funds and resources.
- **Visionary**: A visionary possesses the most accurate perception of the business objectives of the system and the project. The visionary initializes the project by finding all the essential requirements in advance. The visionary also supervises and makes sure that the development process is in the right track.
- **Ambassador User**: The ambassador user is a representative of the user community. He ensures that the developers receive enough amount of feedback from the user during the development process.
- **Project Manager**: The project manager manages the project in general. A project manager can be from the user community or IT staff.
- **Technical Coordinator**: The technical coordinator is responsible for designing the system architecture and controlling the technical quality in the project.
- **Team Leader**: The team leader ensures that the team works effectively as a whole.
- **Developer**: A developer interprets the system requirements and models it. A developer is also responsible for developing the deliverable codes and building the prototypes.
- **Tester**: A tester checks the code and ensures it is technically correct.
- **Scribe**: A scribe is responsible for collecting and recording the requirements, agreements, and decisions made in each workshop.
- **Facilitator**: A facilitator is responsible for managing the workshops progress and also facilitates preparation and communication.
- **Specialist Roles**: Business Architect, Quality Manager, System Integrator, and so on.

10.4.5 Techniques in DSDM

The following are the techniques used in DSDM:

➔ **Time-boxing**: DSDM used the time-boxing technique where milestones are used in the traditional methods. The time-box usually consists of two to six weeks, where the planned tasks are completed. A time-box:

- Can contain several tasks.
- Has to deliver a product at the end.
- Is subject to change.

➔ **MoSCoW Rules**: MoSCoW rules are the DSDM techniques used to weigh the importance of requirements. The rules are:

1. **Must have**: All features belonging to this group must be implemented. The system would not work if they are not delivered.
2. **Should have**: Features of this priority, although important, can be omitted if there are time constraints.
3. **Could have**: The features in this group add functional enhancements to the system and can easily be reassigned to a later time-box.
4. **Want to have**: These features are of little value and serve only a limited group of users.

➔ **Prototyping**: Evolutionary prototyping in DSDM encourages incremental development and frequent delivery. Critical functionality is developed first, so that difficulties are discovered and sorted out early and product is delivered early. The feedback is provided through a workshop.

The different prototypes in DSDM are:

- **Business Prototype**: Allows assessment of the evolving system.
- **Usability Prototype**: Checks the user interface.
- **Performance Prototype**: Ensures that the performance of the product and its ability to handle volume.
- **Capability Prototype**: Evaluates the capability of the product by exploring all possible options.

10.4.6 Key Factors in DSDM

The ten most important factors in DSDM as compiled by the DSDM consortium are:

- Acceptance of DSDM philosophy before starting work on the project.
- Decision making powers of users and developers.
- Responsibility of the senior user management to provide significant involvement of end-users.
- Incremental delivery.
- Approachability of end-users to developers.
- Stability of the team.
- Skills of development team.
- Size of the development team.
- Supportive commercial relationship.
- Development technology.

10.4.7 Advantages and Disadvantages of DSDM

The advantages and disadvantages of DSDM are summed up in table 10.3.

Advantages	Disadvantages
Results of development are directly visible	High cost of licensing
Basic functionality is delivered quickly and additional functionality is delivered at regular intervals	Relatively higher barrier to entry
Users can steer the direction of the project at any point	Cultural shift in organization
As the constant feedback is provided from the users, the system being developed meets all the user requirements	Very costly as the manpower need to be trained properly
Early indications of whether a project will work or not are available rather than unpleasant surprises at the end	It is not suitable for small organizations
Deliverables on time and budget	Not very user friendly as users need to have proper knowledge of the system to implement it. Implementation is not easy

Table 10.3: Advantages and Disadvantages of DSDM

10.5 Crystal

Crystal was introduced as a family of methodologies in 1998. These are lightweight methodologies and are based on the belief that different projects call for different methodologies. Continuous review and revision of the development process ensures flexibility of the process. In Crystal methodologies, projects are categorized according to the size of the project and the criticality of the system being developed.

According to Alistair Cockburn, the core Crystal philosophy is:

"Software development is usefully viewed as a cooperative game of invention and communication, with the primary goal of delivering useful, working software, and the secondary goal of setting up for the next game."

Two consequences of this philosophy are:

- Different projects require techniques.
- The amount of modeling and communication that people require to put in is just the amount they require to jointly move the game forward.

The Crystal family has two common rules:

- The project must use incremental development, with increments of four months or less.
- The team must hold pre and post-increment reflection workshops.

The two basic Crystal techniques are:

- Methodology tuning technique: This uses project interviews and team workshop to convert a base methodology to a starter methodology for the project.
- Reflection workshop technique: The workshops are conducted every couple of weeks duration to find the processes which are not working well. The strategy is developed to rectify such processes that can help the team.

Crystal methods are mainly focused on:

- People
- Interaction
- Community
- Skills
- Talents
- Communications

10.5.1 The Crystal Family

The Crystal family of methodologies uses different colors to express the 'weight' of each methodology. A methodology such as Crystal Clear, Crystal Orange, or Crystal Yellow is used if a project is a small one, and Crystal Diamond or Crystal Sapphire methods are used if the project is a critical one, where human life could be at risk. The Crystal family is as shown in figure 10.3.

The family is divided into different colors such as:

- Crystal Clear
- Crystal Yellow
- Crystal Orange
- Crystal Orange Web
- Crystal Red
- Crystal Maroon
- Crystal Diamond
- Crystal Sapphire

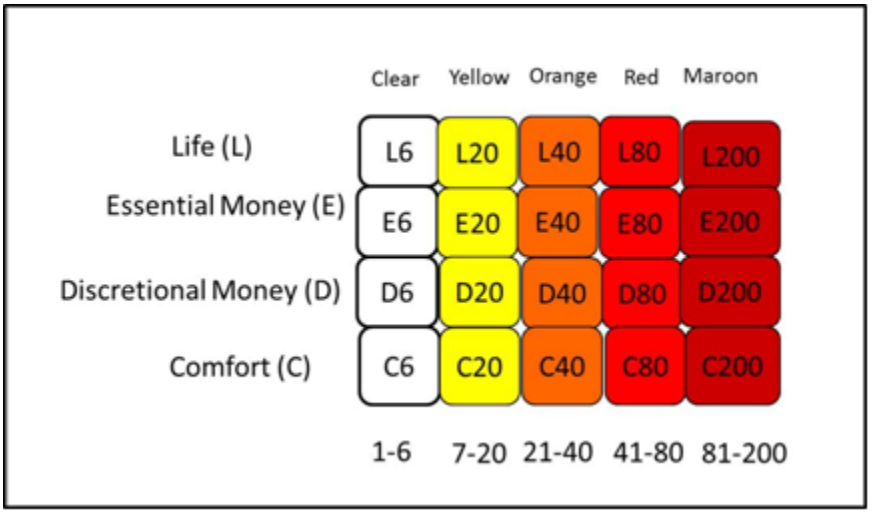


Figure 10.3: Crystal Family

Larger projects, requiring more coordination and communication, map to darker colors (clear, yellow, orange, red, and so on). Projects for systems that can cause more damage, need added hardness in the methodology, more validation, and verification rules. A quartz methodology is suited to a few developers creating an invoicing system. The same team controlling the movement of Boron rods in a nuclear reactor needs a diamond methodology, one calling for repeated checks on both the design and the implementation of their algorithms.

Concepts

Crystal methodologies are characterized by color, according to the number of people being coordinated: Clear is for collocated teams of eight or fewer, Yellow is for teams of 10-20 people, Orange is for 20-50 people, Red is for 50-100 people, and so on, through Maroon, Blue, and Violet.

There are seven prevailing common properties between all the members in the Crystal family. The more these properties are there in a project, the more likely it is to succeed. They are:

- Frequent delivery
- Reflective improvement
- Osmotic communication
- Trust
- Focus
- Easy access to expert users
- Technical environment with configuration management, frequent automated tests, and integration

- **Frequent delivery**: This is an crucial feature of Agile methodologies and implies the regular release of iterations of the software product. Designers and developers decide the features to be included in each release. The length of the project determines the release times.

  By releasing the product in iterations, stakeholders are be able to identify issues earlier in the project which helps to save a lot of time and effort in the later stages. Also, during the initial iteration releases, steps are taken to make redress the situation, in case the customers feel that the output is not upto their expectations. Sometimes, a number of iterations are gathered and delivered in a single release as it may not be feasible to release each iteration.

- **Reflective improvement**: Developers sometimes take a break from regular development and try to find new and better ways to improve their processes. Iterations help by providing feedback on whether the current process is working or not.

  'Reflection Workshop' meetings are encouraged each couple of weeks. These workshops help identify the processes that are and aren't profitable and help the team to improve their strategies.

- **Osmotic communication**: Osmotic communication is used in Crystal Clear and other smaller Crystal methodologies. Osmotic communication involves the team being together in a room and getting the information to flow around it. When it comes to larger teams where there is the possibility of distraction, close communication is preferred.

The team is encouraged to be in the same room for this to work. Else, there is a chance that a person loses his focus. This type of communication enables information to flows quickly throughout the team. Queries can be responded to instantly and any issues or misconceptions can be sorted out. By getting inputs from the others in the team, a developer can get an update on what the others are doing, get new ideas, and gain experience. Developers working together can help out each other with their work.

This method reduces communication overhead to a great extent and necessity for email updates and extra documentation is lowered. Another advantage is that, when the team discusses issues together, each member knows what the others are doing, and are able to take over their team-mate's part of the project if required.

➔ **Trust**: The people in the team must be able to speak up about the issues they face and trust each other. If someone is ridiculed whenever they ask a question or suggest an idea, there are fewer chances for them to speak up the next time.

➔ **Focus**: Focus in crystal refers to two things:

- Focusing on an individual task in a project till there is progress
- Focusing on the direction in which the project is heading

With individual tasks, the flow of progress is taken into account while considering distractions such as interruptions, meetings, and phone calls. This delays the completion of the current task as it can take a while to get back into the flow again. Crystal defines two rules for dealing with issues that are likely to interrupt focus. One is to fix a two hour period during which the developer is not to be interrupted. The other one is to assign a project to a developer for a minimum of two days before being switched to another project.

Definition of goals is important when it comes to focusing on the direction of the project. The goals of the project should be clearly defined and the developers should have precise knowledge of them. The project leader should prioritize the goals which allow developers to focus on particular areas.

➔ **Easy access to expert users**: This involves the developers working with a project expert, so that they can clarify queries and request for solutions to problems. The expert user should be an actual/real-life user. The more involved the expert, user is the better. There should be a minimum of a two hour meeting at least one time a week with the expert user and the provision to contact them on the phone.

➔ **Technical environment**: The idea behind a technical environment with automated tests, configuration management, and frequent integration is that, errors can be spotted easily when changes are made. Issues are less likely as this is done on a regular basis.

Checking-in code into a repository can help identify the code with errors and also help removing it by updating with the correct code. Automated testing and configuration management allow for faster unit testing and consequent resolution of problems.

10.5.2 Advantages and Disadvantages of Crystal

The advantages and disadvantages of the Crystal methodologies are given in table 10.4.

Advantages	Disadvantages
Iterative and incremental process	Limited scalability
Continuous integration	Limited applicability. Not suitable for developing highly critical systems
Iterative development managed by planning and reviewing	Over-dependence on inter-human communication
Methodologies used in a low-criticality project can be used for a high-criticality project too	Lack of a clear common process
Active user involvement	Seamlessness not addressed (Crystal Clear)
Flexible and configurable process	Traceability to requirements suffer as planning and development activities are not necessarily
requirements-based (Crystal Clear)	
Early and frequent releases	Design of the entire system is not performed as a team effort and is carried out by individual developers in the manner, they choose (Crystal Clear)
Specific work-products prescribed	Behavioral and functional modeling can be of low quality as details of many work products are left to the individual developers to decide (Crystal Clear)
Continuous validation	No formalism
Traceability to requirements (though limited) through continuous validation and quality reviews (Crystal Clear)	
Requirements evolve over time (Crystal Clear)	
Test-based development (Crystal clear)	

Table 10.4: Advantages and Disadvantages of the Crystal Methodology

10.6 Check Your Progress

1. What does ASD stand for _____.

(A)	Adaptive Software Development	(C)	Administrative Software Development
(B)	Adaptive Software Design	(D)	Adaptive System Development

2. _____ are the most important techniques used in the feasibility study in DSDM.

(A)	Reviewing	(C)	Designing
(B)	Workshops	(D)	Refactoring

3. Which one of the following is the role of the Ambassador user?

(A)	Initializing the project by finding the essential requirements early.
(B)	Has the ultimate power to make all decisions.
(C)	Ensure that the developers receive enough amount of feedback from the user during the development process.
(D)	Manages the projects.

4. In DSDM, a _____ checks the code and ensures it is technically correct.

(A)	Tester	(C)	Facilitator
(B)	Scribe	(D)	Developer

5. Which of the following are the correct crystal techniques?

(A)	Methodology tuning technique
(B)	Reflection workshop technique
(C)	Work collaboration technique
(D)	Time-based solution technique

(A)	A, B	(C)	C, D
(B)	B, C	(D)	A, C

10.6.1 Answers

1.	A
2.	B
3.	C
4.	A
5.	A



Summary

- ASD is a new software development methodology that focuses on rapid creation and application and evolution of software systems. It was derived from rapid application development and addresses the Internet economy.
- The ASD life cycle is divided into three main phases namely, Speculate, Collaborate, and Learn.
- DSDM is an Agile software development approach that provides a framework for building and maintaining a system that meets tight time constraints through the use of incremental prototyping in a controlled project management.
- The different phases in DSDM are namely, Pre-Project, Feasibility Study, Business Study, Functional Model Iteration, Design and Build Iteration, Implementation, and Post-Project.
- The techniques used in DSDM are Time-boxing, MoSCoW Rules, and Prototyping.
- Crystal was introduced as a family of methodologies in 1998. These are lightweight methodologies and are based on the belief that different projects call for different methodologies.
- There are seven prevailing common properties between all the members in the Crystal family. The more these properties are there in a project, the more likely it is to succeed.

Onlinevarsity
your e-way to learning



Visit the
Frequently Asked Questions
section @

www.onlinevarsity.com

Session - 11

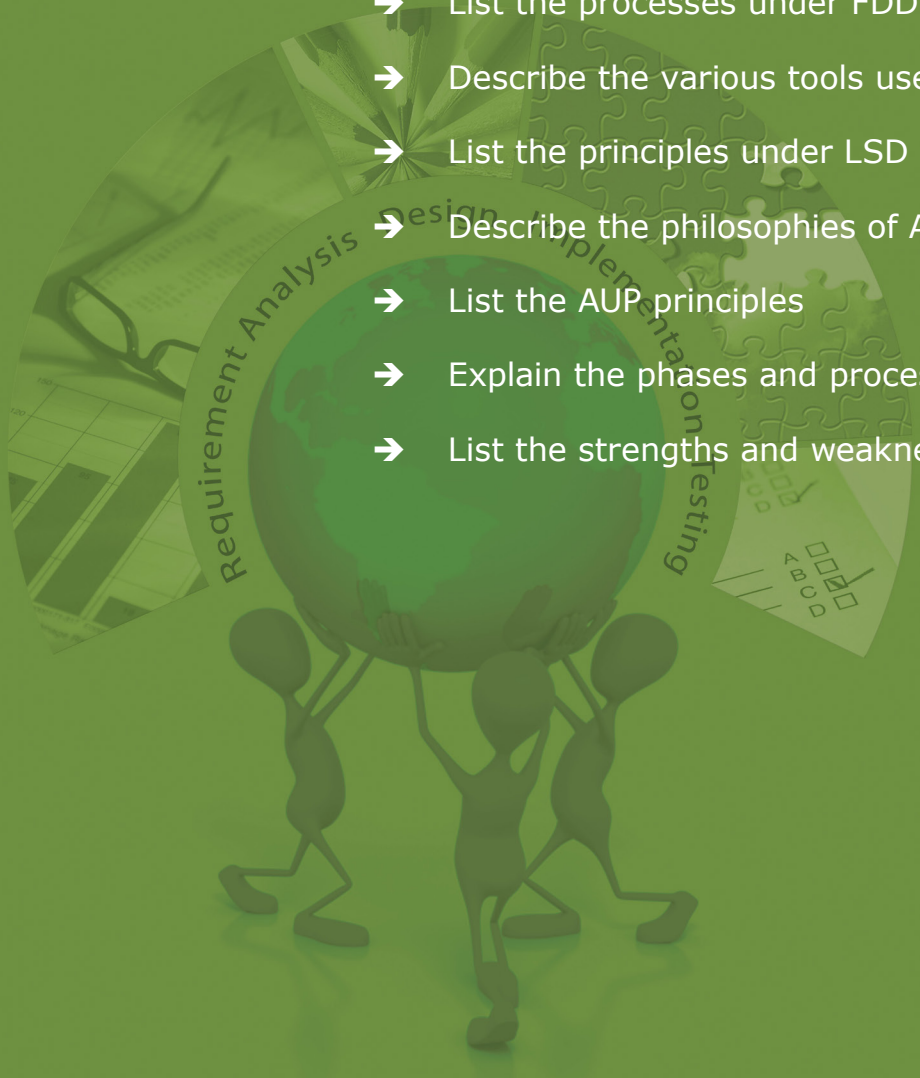
Agile Process Models - II

Welcome to the Session, **Agile Process Models - II**.

This session explains the methodologies – Feature Driven Development (FDD) and Lean Software Development (LSD). It also describes the Agile Unified Process (AUP).

In this Session, you will learn to:

- Describe FDD
- Describe different FDD practices and principles
- List the processes under FDD
- Describe the various tools used under LSD
- List the principles under LSD
- Describe the philosophies of AUP
- List the AUP principles
- Explain the phases and processes under AUP
- List the strengths and weaknesses of AUP



11.1 Feature Driven Development (FDD)

FDD is a type of software development methodology, ideal for use by big teams, who undertake assignments involving object-oriented technology. The methodology description includes certain instructions regarding the tasks which are to be carried out and which roles should be assigned for them.

Corporations that are looking forward to switch to an iterative approach from a phase-based approach, without foregoing task and role assignments would benefit from FDD.

Given that FDD is a short-iteration process:

- ➔ It starts with creating an overall model shape
- ➔ Once a model shape is created, a two week series of 'design by feature, build by feature' iterations is launched

The main goal is to create features which should be useful in the eyes of the client. This is challenging as a majority of the iterative processes are lengthy and do not feature on the clients' priorities; for example, 'build the accounting system' and 'build the persistence layer' iterations.

FDD is a highly adaptive software development process, which:

- ➔ Is highly and short iterative
- ➔ Pays attention to quality, in all the steps
- ➔ Provides working results at the end of each step
- ➔ Is favored by managers, developers, and users

11.1.1 FDD – Practices

FDD is a process which is a combination of a number of best practices. Some of the practices are explained as follows:

- ➔ **Domain Object Modeling**

  This best practice involves creating class diagrams which show the important objects and their types in a specific problem area and how they relate to one another. Since the problem gets divided into the important objects/classes, it becomes easier to find solutions. Finally, once all the classes are put together, they form the solution to the larger problem. There are several techniques under Domain Object Modeling, amongst which 'Modeling in Color' is considered to be the best.

➔ UML in Color

Regular UML with color-coded classes defines the UML in Color process. Classes are classified under different color categories. This makes it easier for the user to understand the dynamics of the problem domain. Do note that the auxiliary classes and interfaces are not assigned any colors.

Each color has a different meaning:

- **Yellow**: Denotes the role assumed by a person in an organization. A user can have many different roles. For example, in an online auction, a user can participate as a buyer, seller, or a system administrator.
- **Blue**: Used for description, often similar to catalogues. This description helps classify or label an object, providing extensive details. An example would be a car dealership brochure.
- **Green**: Stands for a party, place or thing, with something unique or particular identifiable about it. People, organizations, and objects can be used as examples for the Green category.
- **Pink**: Refers to a point in time or an interval of time, which can be traced. The sale of an item, such as a car, can be traced as per the time-stamp on the bill; therefore, it falls under the Pink category.

This concept has been illustrated in figure 11.1.

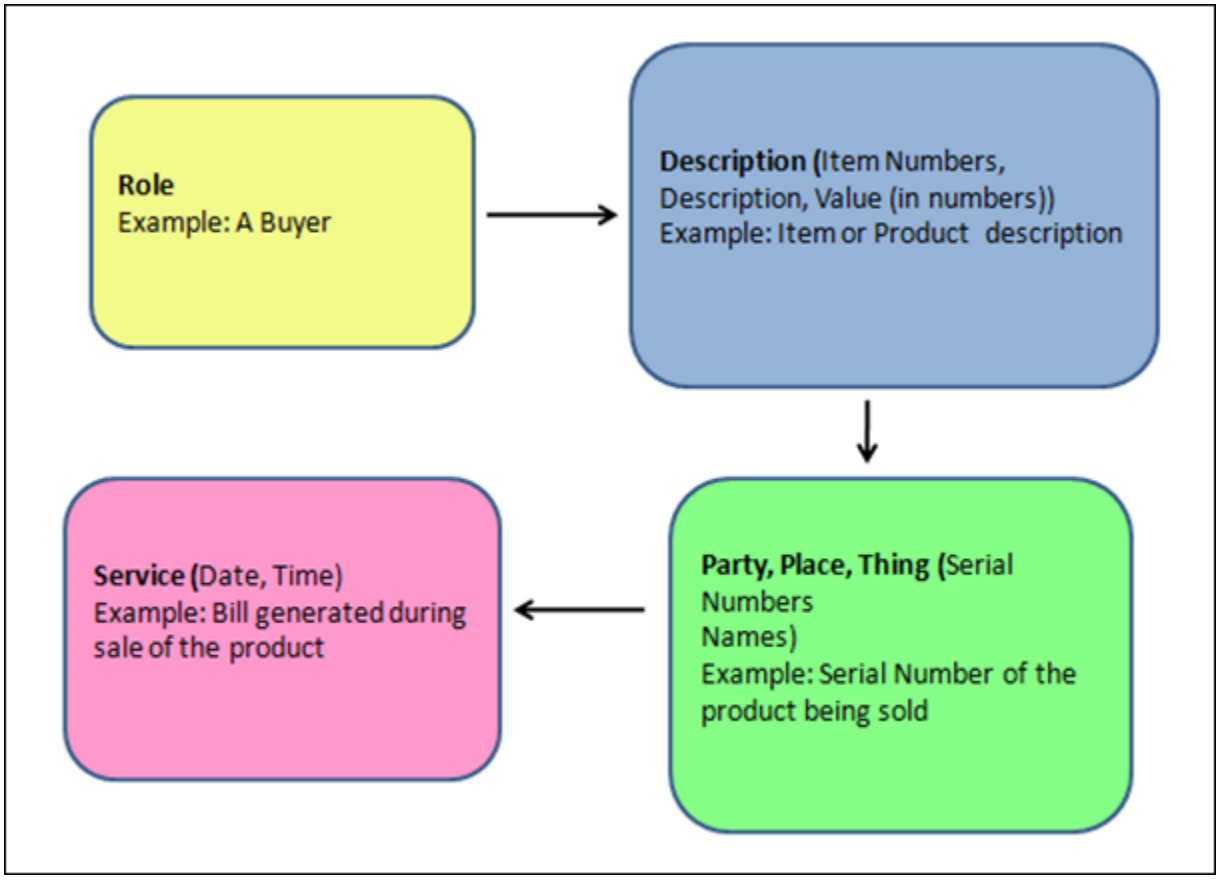


Figure 11.1: UML in Color

➔ Developing by Feature

A complex function, which spans over two weeks, is broken down into smaller functions, which can be completed within two weeks. The smaller functions are called Features. In a business system, a feature can be traced to an activity which forms a part of a business process.

The feature naming template is:

<action>the <result>< by | for | of | to |><a (n)><object>

Some examples of Features are as follows:

- Calculation of [action] the total [result] of a sale [object]
- Evaluation of [action] the fulfillment timeliness [result] of a sale [object]
- Calculation of [action] the total purchases made [result] by a customer [object]

A Feature Set is a group of business-related features. An example for a Feature Set is as follows:

- Making a product sale
- A major feature Set
- <object>Management

➔ Class (Code) Ownership

Class code ownership represents the person who has the ultimate responsibility for the contents of a class. It could also be a role, which is assigned ownership of the contents of a class. FDD uses individual ownership, a concept where developers are given ownership of a set of classes, based on the domain object model.

➔ Feature Team

Since the implementation of a feature could involve multiple classes, it could have multiple class owners. In such a scenario, the Feature owner will be required to co-ordinate the efforts of the multiple developers.

➔ Inspections

This practice aims at publicizing good practices, conventions, and development culture. As it is a combination of several people within a team, it helps generate better ideas, without fear of embarrassment on any individual's part. The best known defect detection techniques are applied and the resulting opportunities are leveraged, to promote best practices.

➔ Regular Build Schedule

At regular points of time, a system is built using the source code of the completed features as well as

the components and libraries it is dependent on. This way, a system is always ready for exhibition.

➔ **Configuration Management**

The main purpose of this best practice is the identification of the latest versions of the completed source code files. This also provides historical data relating to information artifacts in a project.

➔ **Progress Reporting**

Frequent reporting is an essential part of the process as it helps identify the status of completion of the project. Reporting should be regular, apt, and accurate, spanning all levels. It should also consider the internal and external factors of the project for reporting purposes.

11.1.2 FDD Process

There are five processes under FDD are as follows:

1. Develop an overall model
2. Build feature list
3. Plan by feature
4. Design by feature
5. Build by feature

The five FDD processes are shown in figure 11.2.

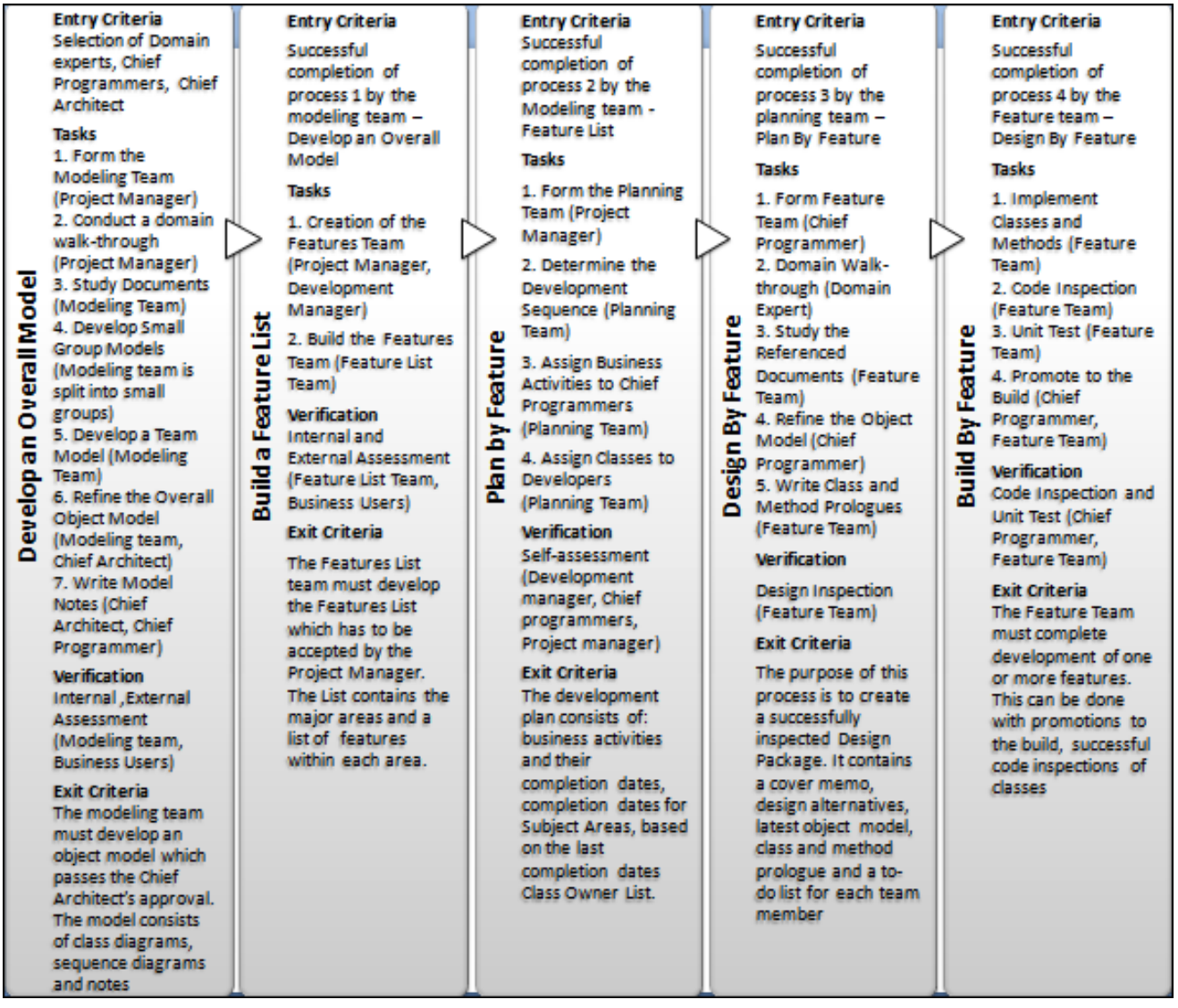


Figure 11.2: FDD Processes

Concepts

Table 11.1 shows five processes used under Feature Driven Development.

	Process #1: Develop an Overall Model	Process #2: Build A feature List	Process #3: Plan by Feature	Process #4: Design by Feature	Process #5: Build by Feature
Entry Criteria	The client collects all the requirements and decides what he really needs.	Selection of Domain Experts, Chief Programmers, and the Chief Architect has been completed.	Completion of the 'Build a Features List' process.	Completion of the Planning process.	Completion of the Design by Feature process—the conclusion is based on the successful inspection of the design package.
Tasks	Form the Modeling Team Domain Walk-through Study Documents Develop the Model Refine the Overall Object Model Write Model Notes	Form the Features List Team Build Features List	Form the Planning Team Determine the Development Sequence Allocate Business Activities to Chief Programmers Assign Classes to Developers	Form Feature Team Domain Walk-through Study the Referenced Documents Refine the Object Model Write Class and Method Prologues	Implement Classes and Methods Code Inspection Unit Test Promote to the Build
Verification	Internal or self-assessment is achieved by the active participation of domain experts.	Two types of assessments are conducted – an internal and an external assessment.	A self-assessment is conducted by the development manager, chief programmers,	The chief programmer decides whether to have a design inspection with the feature-team or any other project members.	The verification of the output of this process can be traced to a successful code inspection and the successful completion of unit test.

Concepts

	Process #1: Develop an Overall Model	Process #2: Build A feature List	Process #3: Plan by Feature	Process #4: Design by Feature	Process #5: Build by Feature
Verification	On an as needed basis, external assessment is made by referring back to the business (users) for ratification or clarification of issues that affects the model.	The internal assessment is performed by the domain experts. An external assessment involves confirmation on the issues that affect the features list, by the Business-end or Users.	and the project manager as planning is a team activity.	Once accepted, a 'To-Do' list is generated per affected class. This requires all the members of the team to add their tasks to the calendar task list. The Chief Programmer must collate all the changes and upload it on the change control system from the shared Feature Team Area.	The details of the unit test as well as the code inspection have been described earlier.
Exit Criteria	Class diagrams – It ¬is a pictorial representation of the different classes in a domain, their connections to one another and their limitations. A Class is basically a template for creating objects, which describes	A list of the Subject Areas. The set of Business Activities within each Subject Areas. The Features which satisfy the criteria fixed in each Business Activity.	A schedule of business activities and their completion dates. The chief programmers assigned to each business activity.	A Cover Sheet or memo, which is a comprehensive guide of the Design package, which can independently stand for review.	Successful code inspection of classes and/or methods. Classes promoted to the Build. The completion of a client-valued function, otherwise known as Feature.

Concepts

	Process #1: Develop an Overall Model	Process #2: Build A feature List	Process #3: Plan by Feature	Process #4: Design by Feature	Process #5: Build by Feature
	the attributes and methods operating on data as an encapsulated unit. The class diagram provides information about placement of methods and attributes. Sequence Diagrams, if applicable. Notes substantiating the selection of a particular model shape, along with comparisons with alternative models.		Assignment of completion dates for Subject Areas, which are based on the last completion dates of the relevant business activities. The Class Owner List –a catalogue of classes and developers who own them.	The documents and all related confirmation memos along with supporting documentation, which are referenced requirements. The Sequence Diagram. The Design alternatives. The new/ updated classes, methods, and attributes of the Object Model. The <your tool> output generated for the class and method prologues produced or altered due to this design. Calendar/To-Do task-list entries each team member with regard to the action items on affected classes.	

Table 11.1: Feature Driven Development

Concepts

11.1.3 Roles in FDD

The most significant part of a project is people, followed by process and technology. Six key roles are defined by FDD, and some more roles are implied as follows:

- **Project Manager**: Serves as the administrative head of the project. The Project Manager would be in-charge of reporting, budgeting, procuring required headcount in addition to overheads management (managing equipment, space, resources, and so on).
- **Chief Architect**: Responsible for the overall design of the system. This includes conducting the workshop design sessions and team management as design involves team collaboration and problem-solving. In addition to this, the Chief Programmer is required to possess excellent technical skills, modeling skills, and facilitation skills.
- **Development Manager**: Manages the day-to-day development activities. This role requires a fair amount of facilitation and problem-solving, when it comes to dealing with team issues, in case the Chief Programmers are unable to do so.
- **Chief Programmers**: The Chief Programmers are developers who have extensive experience of the entire software development life cycle. They contribute to the high-level analysis and project design activities. Their responsibilities include leading the teams through design, development, and low-level analysis of the new software's features. The teams are populated by three to six people.
- **Class Owners**: They are Developers who design, code, test, and record the features required by the new software system. They work as members of small development teams and are supervised by a Chief Programmer.
- **Domain Experts**: They represent the knowledge database. They can be users, sponsors, business analysts or a combination of these roles. Developers depend on domain experts to deliver the correct system. Good verbal, written, and presentation skills are pre-requisites for a Domain Expert. It is their knowledge and participation that is important for the success of the system being built.

Supporting Roles are as follows:

- **Release Manager**: Reports directly to the Project Manager and is in-charge of the progress reports and has other responsibilities. The progress reports are sent by the Chief Programmers.
- **Language Guru**: An expert in a programming language or a specific technology. In projects where a particular programming language or technology is used for the first time, the role of a language guru is vital.
- **Build Engineer**: The Build Engineer is in-charge of setting up the regular build process, maintaining it, and running it as well.
- **Tool Smith**: Since the development team, test team, and the data conversion teams require for creating small development tools, a Tool Smith is charged with creating the necessary tools.

➔ **System Administrator:** Responsible for configuration, management, and troubleshooting any servers and network of workstations specific to the project team.

11.1.4 Need of FDD

FDD is useful because it shows that in a project that is of an iterative and incremental type, the attention can be on domain-modeling. It also explains what the agile-like methodologies can scale. Most essentially, FDD demonstrates how planning and a clear understanding of the domain at the beginning of the project, helps in drafting a plan to see it through completion. This also helps work around 'analysis and design paralyses'.

FDD is usually used for large projects. However, it is an iterative and incremental approach. The method is useful for a large team. The large team can be split into smaller, feature-focused teams. Earlier approaches divided a team based on skill-sets or roles.

11.2 Lean Software Development (LSD)

LSD is a movement dedicated to reducing errors and wasted time and for maximizing education and efficiency. It is originally derived from the IT and Manufacturing industries and is now used largely in the programming industry.

LSD is the application of Lean Thinking to the software development process. Organizations which are truly lean respond in a highly disciplined and rapid manner to market demand. This gives them a strong advantage over their competition, which focuses more on predicting the future of market demand. Likewise, Lean Software Development is a drive to create software that can promptly adapt to the changes in its domain. It exercises high discipline along with high responsiveness to change. This was derived after observation of extreme types of systems – from unplanned software development to heavy process discipline. Neither was able to sustain, as one system could not scale large complex systems while the other proved unresponsive to change.

A high-level overview of LSD is shown in figure 11.3.

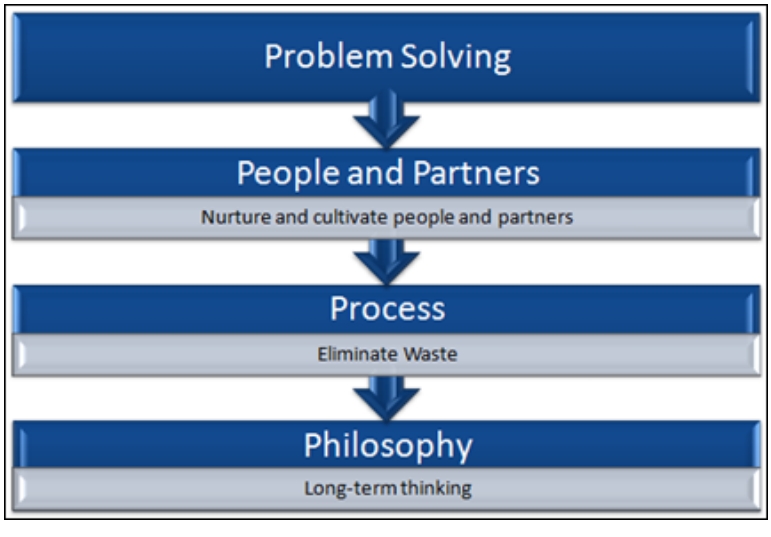


Figure 11.3: Lean Software Development

11.3 Principles of LSD

There are seven key principles that form the backbone of Lean Software Development. They are as follows:

- Waste elimination
- Encouraging learning
- Late decision-making
- Fast delivery
- Team empowerment
- Built-in integrity
- Vision of the big picture

11.3.1 Waste Elimination

Anything that fails to bring any additional value to the customer can be defined as 'Waste'. The concept of waste covers a large area – from time to unnecessary codes as well as functions and requirements that do not make sense.

In the world of software development, waste can be defined as any effort that is a non-value-add which could potentially impair the final product. A typical example for this is following the traditional approach of formulating a plan at the beginning of a project, listing detailed requirements. This proves to a potential waste of time and effort as all the variables cannot be identified in the early stages. If a development team continues with this approach, it is very likely that many unnecessary features will be created. In addition to that it is possible that the team commits to incorrect software architecture.

Under 'manufacturing', seven types of wastes have been identified:

- Defects
- Overproduction
- Transportation
- Waiting
- Inventory
- Motion
- Processing (extra steps)

The seven types of wastes listed under software development are as follows:

- Defects
- Unnecessary features
- Handing off work
- Waiting
- Incomplete work
- Information gathering
- Processing (extra steps)

Several tools have been designed to eliminate the waste. Some of them are as follows:

- **Eliminate Waste by Seeing Waste (Tool #1)**

  An example is as follows:

  Inventory (partially done work in software development): Inventory has the risk of becoming obsolete on many counts. Whether it is useful or not is determined only after the software system is actually in production. Also, its role in solving business problems is debatable, especially if the system is considered redundant.

- **Eliminate Waste by Value Stream Mapping (Tool #2)**

  Value Stream Mapping is an intelligent way of detecting waste in the software development process as it focuses on the value-to-client products, rather than organizations, assets, technologies, procedures, and career graphs.

Figure 11.4 is a pictorial representation of the Value Stream Mapping process.

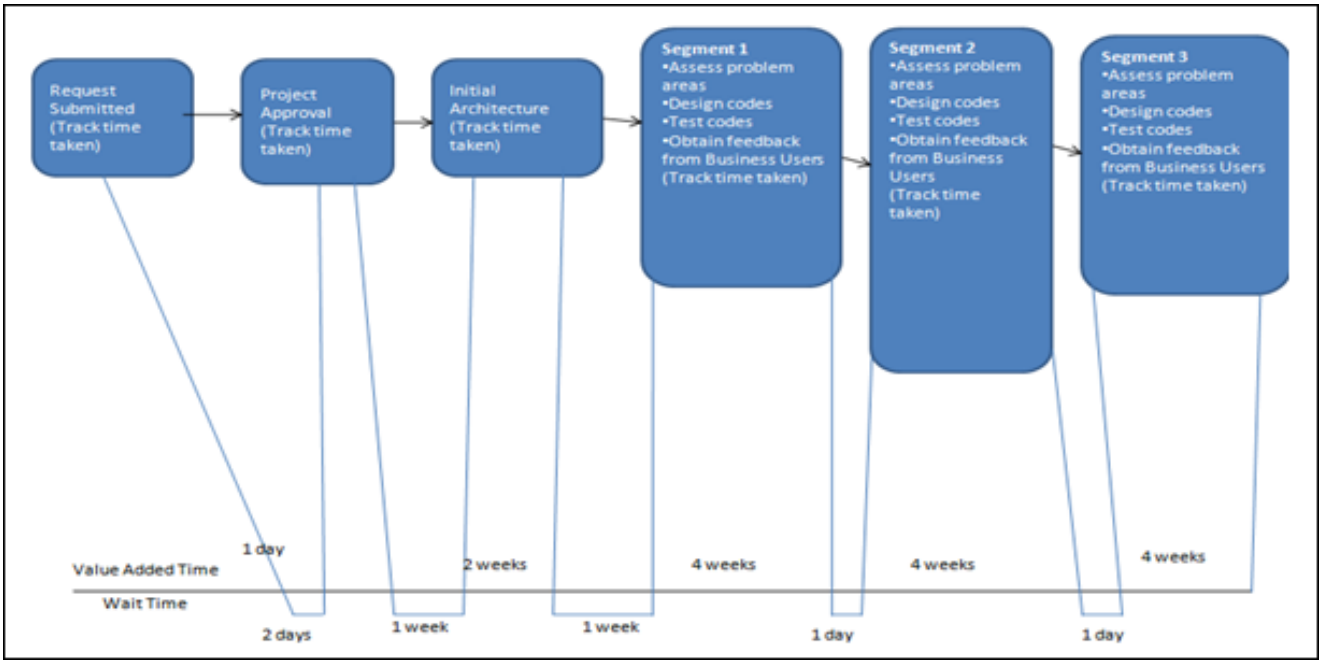


Figure 11.4: Value Stream Mapping

11.3.2 Encouraging Learning

Provide all the tools required to have all the resources, working on a project, from comprehensive code reviews to documentation. Create a Wiki database along with sufficient commenting, deliver training sessions, and hold group meetings. Understanding their tasks will help the resources deliver better and faster results. A method called pair positioning can be implemented where two programmers team up at the same station and work on separate tasks. In this manner, the lesser experienced programmer can tap into the knowledge of the experienced programmer, increasing this rate of progress in the project.

➔ **Encouraging Learning with Feedback (Tool #3)**

When a problem arises, the first task is to ensure that the feedback loops in position. Once they are in place, raise the regularity of the feedback loops in the problem areas. Encouraging and increasing feedback is the most effective way to deal with problematic software development projects and environments.

➔ **Encouraging Learning with Iterations (Tool #4)**

The universal starting point for all agile software development approaches is iterations. Short feedback loops enhance control. Iterations are points of synchronization, where the team and the users see what has been accomplished.

Concepts

Therefore, it can be said that iterations induce decisions (refer to figures 11.5 and 11.6).

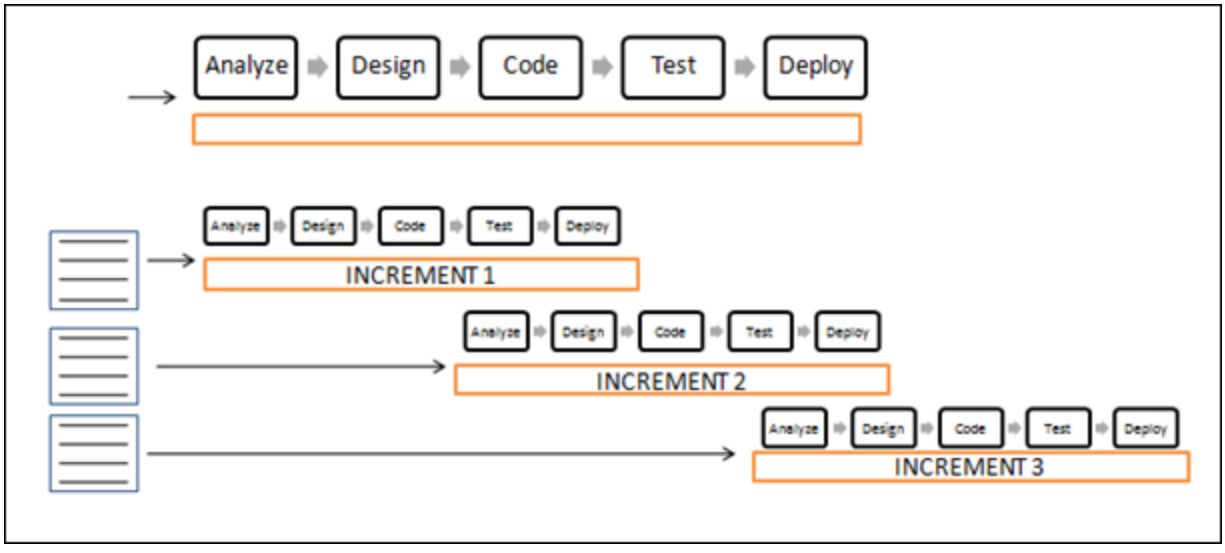


Figure 11.5: Incremental Approach

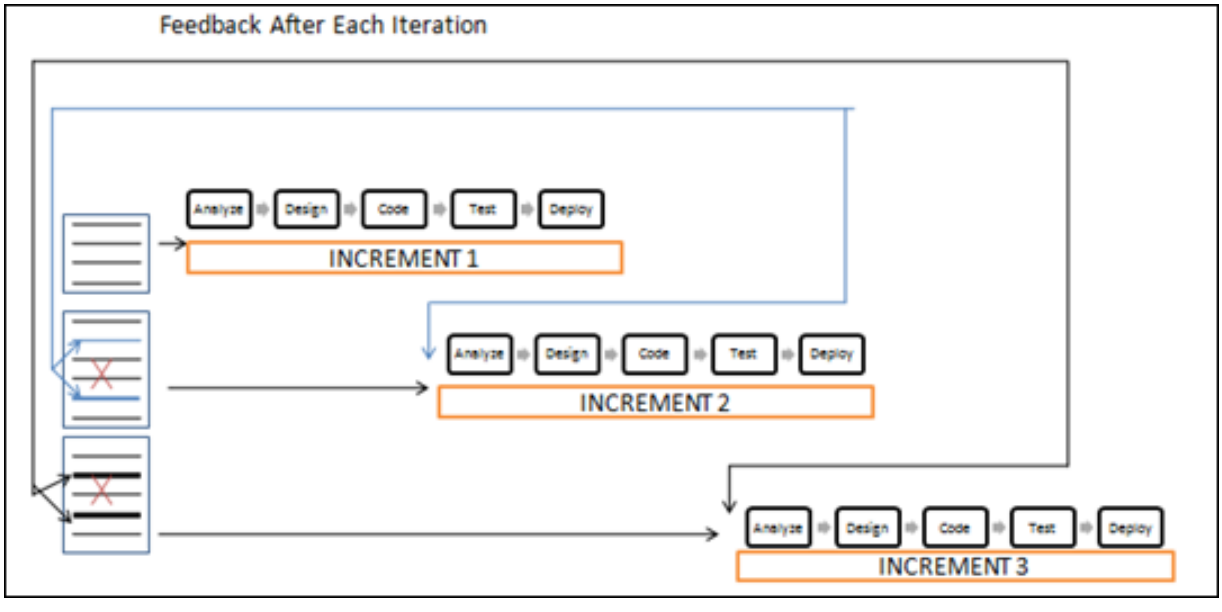


Figure 11.6: Iterative and Incremental Approach

➔ **Encouraging Learning with Synchronization (Tool #5)**

Synchronization is essential when several individuals are working on a single project. It is a fundamental need for any complex development process. Synchronization has to be done at regular time intervals, depending on the need and team size. For example, each team should integrate or synchronize on a daily basis, by checking into a local repository and multiple teams must synchronize on a weekly basis and check into the central repository. The more frequent the build, more feedback they provide. It would be smart to automate builds and build tests.

This has been illustrated in figure 11.7.

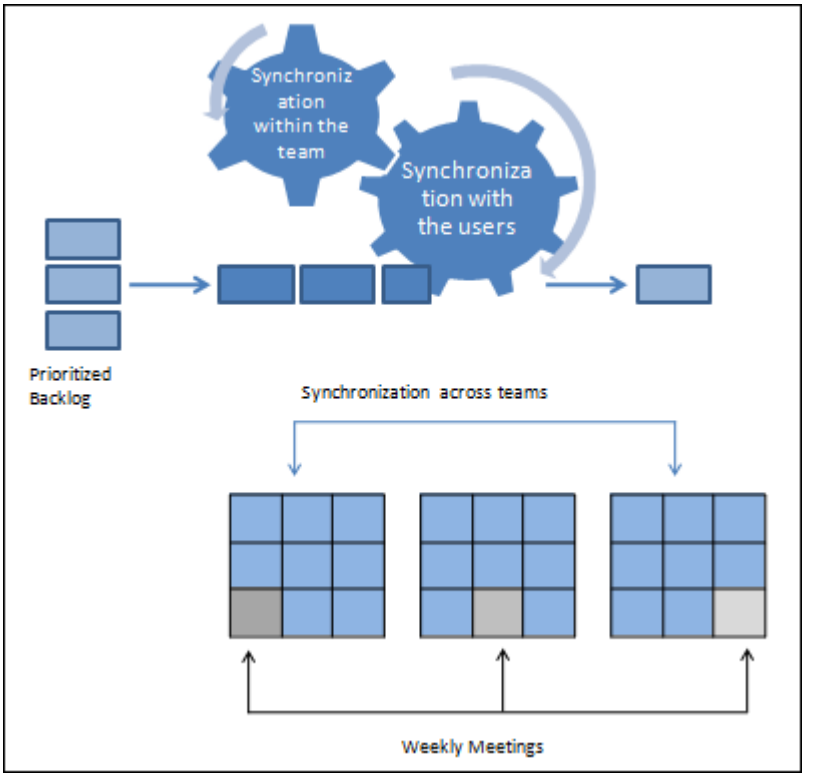


Figure 11.7: Learning with Synchronization

➔ **Encouraging Learning with Set-Based Development (Tool #6)**

In set-based development, communication is not about choices, but limitations. This is regarded to be a powerful form of communication, requiring significantly less data to convey far more information. This also delays making choices until they are absolutely necessary.

By developing multiple options and sharing the constraints, solutions can be sought. When faced with a difficult problem, follow these steps:

- Develop a set of alternative solutions to the problem.
- Test the alternate solutions.
- Merge the best features of the solutions or choose one of the alternatives.

V 1.0 © Aptech Limited

Set-based developments can bring about efficient and faster solutions, as shown in figure 11.8.

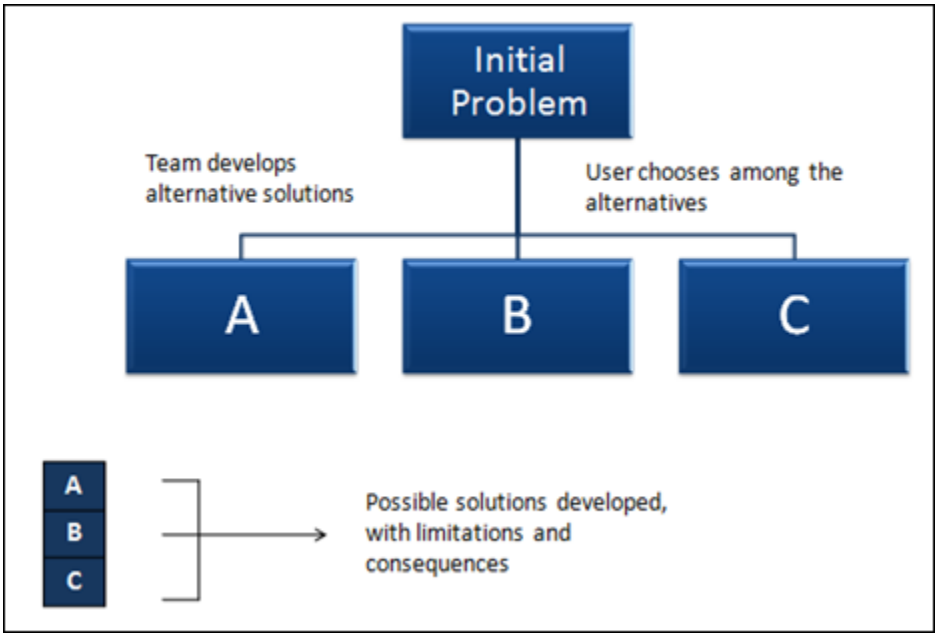


Figure 11.8: Learning with Set-based Development

11.3.3 Late Decision-Making

This process refers to postponing decision-making for as long as possible, especially decisions that will be difficult or impossible to take back. This gives the programming team opportunities to keep their options open, by gathering information and brainstorm longer. Delaying the process ensures that the correct choice is made when it comes to the deployment of a product. It helps maintain flexibility within the code until decisions are finalized.

➔ **Decide as Late as Possible with Options Thinking (Tool #7)**

Giving users options allows them to make fact-based decisions based on learning rather than speculation. Commitment of design at a very early stage, compromises on learning, aggravates the impact of defects. It also limits the usefulness of the product and increases the cost of change. Drafting elaborate or detailed plans at the beginning of a project could turn out to be redundant. While planning is essential, making decisions with too many variables is best avoided. However, caution must be used while employing this approach.

➔ **Decide as Late as Possible with the Last Responsible Moment (Tool #8)**

Synchronized development helps delay commitment until the last responsible moment, where the possibility of failing to make a decision, eliminates an important alternative. Sharing partially complete work, increases the length of the feedback loop, causing irreversible decisions to be made sooner than necessary. By going through short, repetitive, exploratory cycles, better work, and minimal waste can be achieved.

Changes can be accommodated by the use of components with interfaces, abstractions, avoid repetition, separate concerns, encapsulate variation, and avoid extra features. Ascertain the critical aspects in the domain and also sense when decision have to be made.

➔ **Decide as Late as Possible with Making Decisions (Tool #9)**

Intuitive decision-making is preferred as opposed to rational decision-making, as a more mature approach, resulting in good decisions. The following points explore the reasons:

- Rational decision making involves breaking a problem down, removing the context, applying analytical techniques, and exposing the process and its results for discussion. Do note that there is an inherent danger of intentionally ignoring the instincts of experienced people. It is doubtful that rational analysis will be able to detect high-stakes mistakes.
- It is more vital to develop people with the expertise to make wise decisions than it is to develop decision-making processes that purportedly think for people.

An example will help explain this concept better. Marines are expected to plan, but not make predictions. They rely on their understanding of the situation and the assessment of the strengths and weaknesses of both sides. Based on their understanding, they consider boundary conditions and alternative approaches. They encourage varied inputs and settle on a plan with a 110% chance of success. In this manner, Marines manage the project by intent and end result. They convey the tasks to be accomplished and the reasons why, leaving the remaining details with them. They learn to strike a balance between discipline and creativity, empowerment and hierarchy, plans and improvisation, rapid action, and careful analysis.

11.3.4 Fast Delivery

Though a speedy delivery is one of the top priorities of programmers, it is not always possible. A fast delivery also translates to lesser time for changes suggested by users and managers. A conducive environment, where teams can communicate effectively, is key to producing faster and more efficient results.

➔ **Deliver as Fast as Possible with Pull Systems (Tool #10)**

Clarity must be given to each person on how and what they should contribute to the business. The Command and Control method, where resources are told what to do and the Self-Organization method, where a setting is created, so that the resources can figure it out for themselves, are two ways of dealing with it. However, the Self-Organization method works best in the current environment. For effective self-organization, methods for local signaling and commitment must be developed to coordinate work. The slightest variability can render fine-grained assignments in a complex environment. Visual control or Management by Sight is a mechanism where everyone is able to view the process, the problems, and the progress of the project.

➔ **Deliver as Fast as Possible with Queuing Theory (Tool #11)**

The fundamental measurement of a queue is cycle time. This is based on the premise that when in a queue, one wants the shortest possible time-cycle. The cycle-time can be classified as the following:

- **Steady rate of arrival**: When arrival of demand is spread out to match the capacity of the system, queues, and therefore cycle times, will be shortened. One way to control the rate of work arrival is to release small packages of work.
- **Steady rate of service**: The easiest way to remove variability in the processing time is to increase the number of servers that process work in a single queue. Small work packages will allow parallel processing of the small jobs by multiple teams. Therefore, in case one is stalled by a problem, the rest of the project can proceed without delay.
- **Slack**: Short cycle times are not possible if resources are overloaded. Full utilization provides no value to the overall value stream, but is often detrimental.

➔ **Deliver as Fast as Possible – Cost of Delay (Tool #12)**

A number of benefits can be gained through rapid development. The following exercise shows how they can be used:

- Create a simple economic model of a new product for the next few years; for example: a Profit and Loss account.
- From the marketing team, get a rough estimate of the effect of delays on sales volumes and market share.
- The result of the given step will illustrate the effect delays can have on profits.
- The cost of delay can be very high if the delay stands for 'loss of early high pricing' or 'long-term loss of market share'.
- Keep the model relevant and simple to get the buy-in from everyone, by taking help from the Accounts team.
- Economic models could help you justify the cost of reducing cycle time, eliminating bottlenecks, and purchasing tools that will allow faster deliveries. They can also help drive development decisions.
- By presenting the economic model, the team is empowered to think beyond their individual deliverables and ascertain what is important for the business. Since the frame of reference is the same, decisions and suggestions can be derived from the same platform, reducing the probability of variables.

11.3.5 Team Empowerment

Lean team management requires the managers to ensure the programmers are not overloaded. They should also be able to establish their own reasonable workflow. The motivation of the team is key to

producing the best results. One way of motivating the team is involving them in the decision-making process.

➔ **Empower Team by Self-Determination (Tool #13)**

Self-determination works well as a concept, however, practical application and the ability to sustain the growth over time, is especially difficult.

➔ **Empower Team with Motivation (Tool #14)**

Motivation is vital to performance. Some vision statements from William McKnight, a leader of 3M from 1930s through the 1950s are as follows:

- "Hire good people, and leave them alone."
- "If you put fences around people, you get sheep. Give people the room they need."
- "Encourage, don't nitpick. Let people run with an idea."
- "Give it a try - and quick!"

Self- motivation calls for a feeling of belonging, safety, competence along with a sense of progress.

➔ **Empower Team with Leadership (Tool #15)**

Respected Leaders: Usually behind a great team, is the hand of a great leader. The leader works with the team on the initial concepts, sets the pace of progress and determines how decisions are made.

- **Cope with Change**:
  - Set Direction
  - Align People
  - Enable Motivation
- **Cope with Complexity**:
  - Plan and Budget
  - Organize and Staff
  - Track and Control

➔ **Master developers**: Includes chief programmers, architects, and system engineers. Some reasons why master developers make good leaders are as follows:

- In large systems, a small team of exceptional designers come forward to take up primary responsibility for the design. Exceptional designers are singled out for leadership based on their superior knowledge.

- They are familiar with the application domain and the technology, and therefore, serve as experts in providing technical guidance.
- Their understanding of both users as well as the developers puts them in a unique position. They recognize system's constraints, interactions, unstated requirements, exception conditions, and likely direction of change.
- They have a bird's eye-view of the process, but their in-depth knowledge helps in identifying problems at the micro-level. This helps developers and engineers, as they can benefit from their expertise.
- They form the heart of the team, as they are responsible for communication and leadership. Additionally, they assist the team in making good decisions, rapid progress, and developing high-quality software.

➔ **Empower the Team with Expertise (Tool #16)**

Though a great deal of information can be stored in books and other mediums, they cannot replace oral knowledge-sharing, a medium for tacit knowledge. Some of the details that are shared in this mode are as follows:

- Standards are developed by a group of experts, as and when it is required by the program team.
- It is recommended to program teams to work with existing standards than develop new standards.
- Foster communities of expertise, where standards are lacking or haphazard, helps overcome sloppy work, as (new) standards will be developed.
- Developers appreciate reasonable standards, especially if they have helped in developing them and in keeping them relevant.

11.3.6 Built-In Integrity

The main aim of the team should be to produce high-quality code. As discussed earlier, there are several ways of doing so – pair-programming, continual testing, creating small modules of code, and automation.

➔ **Build Integrity In – Perceived Integrity (Tool #17)**

Perceived integrity is when a system appears to know what the user wants of it next. When put in the context of a software development process, the following points are important to note:

- Companies that want to achieve perceived integrity should aim to put the vision of the users before themselves as well as the technical team. Chief engineers have added to their engineering and leadership skills the ability to understand the target customer base and create a vision.
- Though customers know what they want from a design, it is not possible for them to

determine everything that should be a part of the design. This can prove to be a problem when situations and the users' perception change.

- Domain models must be both understood and directly usable by the user and by the developers actually writing the code. Through joint modeling, the understanding of both - the team as well as the users will be the same. This is applicable, when it comes to domain issues as well as implementation. This method is known as Model-Driven Design.
- Software that reflects domain model is more adaptable to market changes than software with significantly different internal structures chosen for technical reasons.
- There is usually a gap between the design on paper and in actuality. The best way to bridge this gap is to deliver tests along with the code, a high-level overview model that was created at the beginning of the development phase.

➔ Build Integrity In – Conceptual Integrity (Tool #18)

Conceptual integrity is the principle that you can tell that a design is part of the same overall design, irrespective of where you look into it from. Some of the salient features of this tool are as follows:

- When the components work well together, the system architecture will be able to balance flexibility, maintainability, efficiency, and responsiveness.
- Conceptual product integrity can be achieved by maximizing the effectiveness of communication mechanisms. These mechanisms are developed among the engineering groups.
- Use of existing parts removes many degrees of freedom and reduces the complexity and requirement for communication.
- The use of integrated problem solving is recommended to ensure a smooth flow of technical information. This can be executed in the following ways:
  - Understanding and solving the problem at the same time, not sequentially.
  - Release of information as and when possible, instead of holding details until all the information is available.
  - Transmission of information in small batches, at regular time intervals, rather than sending a large transmission.
  - Two-way information flow.
  - Encouraging face-to-face interactions as opposed to communicating through documents.
- Forecasting can lead to a significant loss of time and effort, in an uncertain/changing environment. It is advisable to take the breadth-first approach and get the basics right. Once this is fixed, encourage discussion, and brainstorming to generate ideas. Post this phase, plan on regular refactoring to keep the architecture healthy.

➔ Build Integrity In - with Refactoring (Tool #19)

Refactoring is based on the premise that complex problems require simple solutions; however, to find simple solutions takes more than one attempt. Engineers start with something that works, learn from its weaknesses, and improve the design.

Conceptual Integrity can be maintained by refactoring when a system begins to lose the following characteristics:

- Simplicity
- Clarity
- User-friendliness
- No repetition

Good design evolves over the life of a system. Poor code does not get better by being ignored. In such cases, it is important to:

- Stop the line of action or code development when something does not seem right.
- First, take the time to dig for the root-cause of the problem and fix it from the base, then proceed towards the next development.
- It is vital that too much time is not spend on non-value add activities.
- Refactoring is not cost-intensive, but pays, like advertising.

➔ Build Integrity In - with Testing (Tool #20)

Programming has increased in scale, from a point where programming was done one module at a time, to programming entire capabilities and feature. This has blurred the lines between unit, system, and integration tests. One of these tests would be developer tests – tests that ensure the code does what the developer intended it to do. Acceptance tests involve tests that ensure systems do what the customers expect of them. These tests are usually scheduled at the end of the development stage. These can also be called customer tests.

In a situation where the time provided is not sufficient, reallocate resource from requirements documentation to writing consumer tests. Tests should be automated as much as possible and run as part of the daily build. The tests conducted should be accurate and comprehensive. They should be put under version control and automated and under continuous process improvements. By maintaining and running all-inclusive tests throughout the life cycle of the system, you can guarantee that the system can be safely repaired and refactored throughout its life-span.

If the tests are clear and well-organized, they serve as a resource. They provide a clear understanding of how the system works, from the user's point of view as well as the developer's point of view.

11.3.7 Vision of the Big Picture

The Lean development teams ought to view the project as a whole, not as separate segments. The entire project should be kept in mind while setting goals and mini-goals. If not, codes will be written haphazardly, which gives way to bugs.

➔ **See the Whole with Measurements (Tool #21)**

Optimization of every single task can prove to be harmful to the project. It is difficult to measure the work that is qualitative as there is no standard for it, especially when it is unique and ambiguous. In such cases, it is best not to sub-optimize – a practice where parts of an important qualitative piece of work is optimized as it cannot be measured accurately as a whole. The best way to ensure measurement of all tasks, is aggregation, where the measurement is moved one level up, not down.

Information measurements, which are an aggregate of the performance of a team, should be used instead of individual performance data. It is not ideal to track defects per developer, as less than 20% of all defects are under the developer's control. The remaining percentage of defects is caused by existing systems and procedures, which fall under the management's purview. It is important to get to the root of the defect, which can be achieved by encouraging the collaboration of the entire organization, in rooting out the defects.

➔ **See the Whole – Contracts (Tool #22)**

Project Managers control four variables – time, costs, quality, and scope. Of the four components, time, costs, and quality can be fixed. While drafting the initial plan, prioritize features and keep the scope negotiable. In this manner, you can deliver high priority features first. You can possibly present to the user, most of the business value long before their wish list is completed. As per a study by the Standish Group, up to 45% of the delivered features are not used. Barry Boehm and Philip Papaccio noted that the best way to develop low cost, high-quality software is to write less code. Rigid control of the scope tends to expand it. Overall cost saving can be achieved by working with the customer by means of a form of optional scope contract.

These seven principles guide the software development process to achieve maximum efficiency while offering maximum value to the user. Merging smart workflow design and the right attitude produces high-quality results, taking into account the important parameters.

11.4 Lean in Software Life-cycle

The definition and concept of 'Lean' differs in the various disciplines in software development. Some of the important aspects are as follows:

- ➔ It is important to add customer value and to create only those artifacts that helps in other steps of the process.
- ➔ It is appropriate to adopt practices of others – Poppendiecks.
- ➔ Create practices in line with principles that can be customized as required.

The following section summarizes the major points for the different disciplines involved in the software life-cycle:

➔ **Requirement engineering**

Middleton and Sutton argue that traditional requirement engineering should be rethought as a search for custom values. The team must focus on building the right thing, rather than building a thing right. The Kano Model categorizes product attributes as 'must-be', often not explicitly told by the customer, performance factors, and attractive factors in the form of exciting features. Definite guidelines are given which specify ways to provide express requirements in an effective manner. The user can understand this better and it can be used effectively in subsequent steps.

Middleton and Sutton argue for a tabular driven requirement representation called Software Cost Reduction method (SCR), appropriate for real-time systems. Coplien and Bjornvig argue for used cases, more suitable in more (human) interacting software. Poppendiecks also refers to such specification techniques. All of them stress the importance of Domain Modeling as a tool to understand the customer space and build high integrity products which are easy to maintain.

➔ **Architecture**

Middleton and Sutton refer to Parnas' principle of information hiding as an important lean software design practice. They, together with Coplien and Bjornvig, argue that building architecture with a few method-rich objects are central lean ideas, by referring to Parna's work and a method called DCI, respectively.

➔ **Programming**

On a lower implementation level, Middleton and Sutton discuss the lean principles implications for choice of programming language. They conclude that for high integrity software it is more 'lean' to select a language with proven ability to be statically verified, such as SPARC ADA, than a 'cooler' more powerful language like C++. This is in line with Liker's principles to choose proven techniques that serve the purpose of optimizing the whole process flow. By using the 'Design-by-contract' method and enabling local verification of input and output data, irrespective of the code being run, is in alignment with L5 and the lean jidoka practice. In this practice, the machine is designed to detect errors and stop the item or code from being passed on to the next step or process.

➔ **Testing and verification**

Poppendiecks are in favor of a test-first strategy such as Test Driven Development from the agile method Extreme Programming. They also specify that though testing is not a part of a customer-value add activity, it is essential to maintaining high quality. Testing is therefore, termed as a necessary waste. Middleton and Sutton points out that much of the testing (and rework) effort can be reduced much by selecting an easy-to-verify requirement and architecture.

➔ **Integration**

Building software differs significantly from the manufacturing or assembly-line process. For Lean principles to work, the process or project must be decomposed into smaller pieces of comparable size. This means that the system must be built incrementally. To maximize the flow, there must be simultaneously development with the source code. To be able to detect and stop defects as early as possible is vital to have continuous integration. Both the Poppendiecks and Middleton and Sutton refers to Microsoft's Synch and Stabilize process as an example of a process with regular integration points and superior quality demands.

➔ **Project management**

Management of flow is key to lean project management. Andersson's software Kanban method emphasizes that limiting work in progress is important for a small team. Another important feature of Kanban is visual control along with the Agile Scrum Method. The role for a product and project manager changes in a lean organization. Poppendiecks talks about a 'product champion' whose role is to align all project members, work and flow towards producing customer value. In this system, leadership is given more importance than traditional management practices. It is akin to Liker's principles.

11.5 Agile Unified Process

AUP is a simplified version of the Rational Unified Process (RUP). It illustrates a simple and straightforward approach to developing business application software using agile techniques and concepts. Though this is a simplified method, it conforms to the RUP.

11.5.1 Philosophies of the AUP

The Agile Unified Process (AUP) is based on the following principles:

➔ Your staff knows what they are doing: People prefer not read tedious documentation of a process; they respond better to trainings and high-level guidance. The AUP process provides links to several details which gives the people the option of exploring and learning.

➔ **Simplicity**: The description of content is concise, precise, and applicable.

➔ **Agility**: The Agile UP adheres to the guidelines and principles of the Agile Alliance.

➔ **Focus on high-value activities**: The value-add activities are the focal points in the project.

➔ **Tool independence**: Though any toolset can be used with the Agile UP, it is recommended that simple tools or open source tools be used, as they suit the job best.

➔ **Tailor this product to meet your own needs**: the AUP product can be tailored to suit specific requirements, using the available HTML editing tool.

It is not necessary to obtain a special toolset or take a special course to alter the AUP (refer to figure 11.9).

The Model discipline covers the RUP's Business Modeling, Requirements, and Analysis and Design disciplines. Though it is an important part of AUP, it does not control all aspects of the process. The primary goal, at the stage, is to produce models and documents which meet the bare-minimum requirements. Secondly, the Configuration and Change Management discipline has been re-labeled the Configuration Management discipline. In agile development, as part of the Model discipline, change management is a pre-requisite of the requirements management efforts.

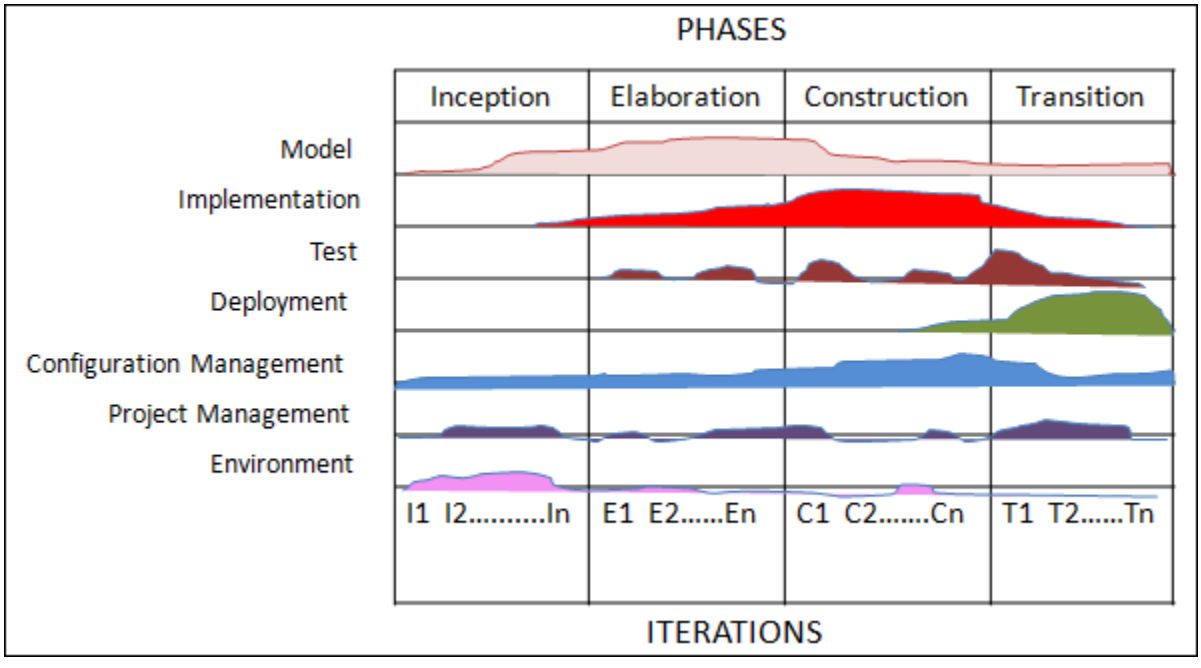


Figure 11.9: Agile Unified Process

➔ **AUP: Process – Phases**

The overall development cycle consists of four phases:

1. **Inception**: The aim, in the first stage, is to outline the scope of the project, identify potential architecture, obtain initial funding, and gain stakeholder acceptance.
2. **Elaboration**: The goal is to prove the system architecture.
3. **Construction**: The main objective is to develop working software in a frequent, incremental manner. This should meet the most critical requirements of project stakeholders.
4. **Transition**: To validate and deploy the system into the production environment is the main aim of this function.

➔ AUP: Process – Iterations and Disciplines

Each phase can be further broken down into iterations:

- Iteration is a complete development loop. As a result, there is a release of an executable increment to the system.
- Each iteration consists of seven work areas. These work areas, also known as disciplines, are performed during the iteration.
- Within each discipline, AUP defines sets of,
  - Artifacts or work-products
  - Activities or units of work performed on each artifact
- Roles and responsibilities of each team member.

➔ AUP: Process – Disciplines

Following are the seven disciplines performed during each iteration:

1. **Model**: The goal is to understand the business, the problem domain being addressed by the project and identify a feasible solution to address the problem domain.
2. **Implementation**: The main aim is to transform the models into codes, which can undergo a basic level of testing, especially unit testing.
3. **Test**: The purpose is to perform a comprehensive evaluation of the project – this is done to ensure the standards for quality are met, to confirm if the system works as it was designed to and to check if all the requirements were met.
4. **Deployment**: The goal is to schedule the delivery plan to the user and to see it through the completion – make the system available to the end-user.
5. **Configuration Management**: Management of artifacts – tracking the artifact versions, the changes, and updates over time, in addition to controlling and managing the changes, is the main purpose.
6. **Project Management**: The objective is to guide and direct all the activities that are part of the project – managing risks, directing people, and co-coordinating with the external teams to ensure delivery of the project on time and within the allotted budget.
7. **Environment**: The intention is to create and sustain an environment, by providing guidance (guidelines and standards), tools (hardware and software) plus a proper process, as and when it is necessary.

➔ AUP: Process – Inception Phase

Following are the tasks in the Inception phase:

1. Defining the scope of the project

2. Estimating the budget – cost and schedule
3. Defining the risks
4. Determining the feasibility of the project
5. Preparing the project environment – allocation of workspace, headcount/resources required, requesting necessary software and hardware, and tailoring the AUP.

➔ **AUP: Process – Elaboration Phase**

Tasks that are part of the Elaboration phase are as follows:

1. Creation of an architectural prototype for the system.
2. Develop the requirements model.
3. Draft an estimate project plan for the construction phase.
4. It is essential that critical tools, processes, standards, and guidelines are in place for the construction phase.
5. Understand and eliminate high-priority risk of the project.

➔ **AUP: Process – Construction Phase**

In the Construction phase, the following tasks are important:

1. Prioritize and understand the requirements.
2. Model-storm a solution.
3. Code and test the software.
4. Arrange early releases to gather early feedback.

➔ **AUP: Process – Transition Phase**

The main tasks in the Transition phase are as follows:

1. Testing and validation of the complete system.
2. Integration of the new system with the existing systems.
3. Conversion legacy databases and systems to support the new release.
4. Training the new users on the system.
5. Deployment of the new system into production.

➔ **AUP: Process – Model Discipline**

This encompasses RUP's Business Modeling, Requirements, and Analysis and Design disciplines. Agility is observed by creating models which meet the bare minimum requirements. It is performed

through model storming sessions which involve a minimum number of people who discuss an issue, with the help of a sketch board or paper. This discussion lasts a maximum of 15 minutes. On occasion, it can stretch to 30 minutes. The team then gathers around a medium such as a whiteboard, which acts as a shared modeling tool, and explore the issue until they have a firm grasp on it and then move on with the process (often coding).

11.5.2 AUP: Advantages and Disadvantages

The advantages of the AUP are as follows:

- Iterative-incremental process
- Based on system architecture
- Based on structural, functional, and behavioral modeling – both logical and physical, of the problem domain and the system
- Based on system functionality, described in use-cases
- Can be traced to requirements through use cases. This can be done throughout the process which serves as the basis for tasks and tests
- Design-based development
- Iterative development engine governed by planning/reviewing
- Though limited, seamlessness can be observed due to use-case based activities and design-based development
- Risk-based process
- Facility to add formal features through UML/OCL
- Addresses configurability and flexibility

The disadvantages of the AUP are as follows:

- It is possible that modeling can endanger agility if the limits are not observed precisely. The list of AUP models that are produced, as a minimum, is extensive.
- Tackling model inconsistencies is not addressed, explicitly.

11.6 User Stories

Kent Beck coined the term and concept of User Story in 1996, which was further developed by Ron Jeffries, Bill Wake, Alistair Cockburn, Mike Cohn and Chet Hendrickson. It was popularized during its addition to the first Extreme Programming Project. Though User Stories are not a requirement for Scrum, they are often included to represent Product Backlog items.

➔ Description

User Stories can be described as easy and simple articles which describe the wish list of a new feature, from the viewpoint of a user. This can be represented in the following template:

As a <type of user>, I want <some goal> so that <some reason>

User stories are not formally documented, but stored in small containers, such as shoe-boxes. They are written up in post-its or index cards and assembled on walls or on tables, to assist and encourage planning and dialogue within teams. This method puts the emphasis on discussions. This way, it shifts the importance from writing features down to open dialogue which yields better results.

➔ Components of User Stories

User Stories are made of three components. They are 'cards', 'conversations', and 'confirmations'. Let us explore them further.

➔ Cards

This is typically a sample of the story, usually written in a sentence or so. It acts as a reminder for the team to keep discussions alive. It consists of a title and a short description of the story and is in the form of a card or a line. It is best to keep the number of words to a minimum. The main function of this card is to prompt conversations.

➔ Conversations

Conversations mainly deal with the details of the story. The outcome of 'conversations' is test confirmation(s).

➔ Confirmations

Acceptance tests can be automated. The tests are important as they facilitate the implementation of results of the first two steps – cards and conversations.

➔ World's Smallest Online Bookstore

This example highlights the salient points of User Stories.

➔ Product Vision

Building a site called Smallestonlinebookstore.com.

The perspective of Smallestonlinebookstore.com differs from that of Amazon. According to them, an unending supply of books is not something the customer looks for. They wish to find the right book – this cannot be accomplished by giving the customer a huge selection to browse for hours.

Each book in the collection has been read and reviewed by our team. The reviews on the site are submitted by the public, as opposed to other sites which circulate reviews given by publishers and authors. The customer support is live and assists readers with any problems they might face with the site. Customer service is their ambition and they accomplish this guiding them to the right selection of books, saving them time and money.

➔ Users

Users are defined as anyone who leverages the system for business value, such as end-users and stake-holders. Some examples include first-time, casual or frequent book buyers, publishers, site team, and warehouse staff.

"The Card": Initial User Stories at Smallestonlinebookstore.com

The cards are the result of the release planning sessions made by the team for the initial product backlog.

Some examples of the cards are as follows:

- I am a first-time book buyer and I want to buy my first book.
- I am a first-time book buyer, looking for a book to help pass time on my next flight.
- I am a regular book-buyer and I want to buy a book in the most efficient way possible.
- I am a first-time book buyer looking for another book by Dan Airely, without having to squander my time on titles I do not like.
- I am a shipping clerk and I want shipping labels printed on self-adhesive labels, which can be attached to the final packages.
- I am a system administrator and I would like to attend to customer requests such as resetting passwords, as and when required, with minimal fuss.

➔ Conversations

Though conversations are the second component of User Stories, they can be held at any time in the process and at any frequency. The nature of the conversations depends on the teams. The central theme for conversations is acceptance tests. The team holds discussions on the story and produce test confirmations with the acceptance criteria in mind.

➔ Participants in the Conversations

Participants in conversations cannot be fixed as they vary as per the nature of the teams involved. While users, programmers, stakeholders and members of the Scrum team can be engaged in conversations, it is usually the Product Owner who has the final word on product vision and requirements. It is recommended to solicit the opinions of stakeholders and end-users as their contributions prove to be very valuable.

➔ Timeline for Conversations

Product Backlog Refinement is a process where it is most important to have conversations. It is to be noted that conversations can happen at any time, without any fixed schedule. Product Backlog Refinement is a continuous process, in a Scrum project. Approximately 10% of the Sprint is assigned to preparation for the upcoming Sprint. The preparation time consists of Product Backlog Refinement, where adding, modifying, deleting, and separating Product Backlog Items is done.

➔ Probable Outcomes

Whenever the user story conversations happen, they should result in shared Team understanding along with test confirmations from Team and Product Owner. An effort must be made for the entire team to be on the same page when it comes to a major portion of the story details and test confirmations, as small changes and conversations are an on-going part of the Sprint. Do note that the Acceptance Criteria has to be shared with the team before the Sprint begins.

➔ Conversations at Smallestonlinebookstore.com

Continuing along the line of the previous example, conversations occur in the following manner in User Stories:

- A meeting is held once a week to filter the stories that will form a part of the upcoming Sprint. The Product and Team owners meet to perform the Product Backlog Refinement.
- As conversations happen in an arbitrary manner, any updates to a story provided by stakeholders, end-users, and team members must be conveyed to the rest of the team.

➔ Confirmations

There are two steps involved in a test confirmation. They are as follows:

1. Sharing of Acceptance Criteria.
2. Conversion of Acceptance Criteria into automated tests, which can verify that each of the acceptance criteria has been met.

➔ Acceptance Criteria

The main objectives of Acceptance Criteria are as follows:

- **Explain the requirements** – what should be built, in code and automated tests, at the beginning of the project.

- To check and confirm understanding of the problem is consistent within the team.
- To communicate the completion of a story with the team.
- To validate the story with the help of automated tests.

➔ When Acceptance Criteria are Created

Normally created during the Product Backlog Refinement and User Story conversations, acceptance criteria need to be established well before the implementation of User stories. Past experience demonstrates that determination of acceptance criteria before the beginning of a Sprint meeting helps produce efficient and industrious results. It also highlights the team's real drive and capacity. Failure to settle on the acceptance criteria before the Sprint meeting leads to over-commitment by the team. It is also reminiscent of the waterfall approach behavior.

➔ Is Documentation Necessary for Acceptance Criteria

Minimal documentation and more conversation is the mantra of the User Story Practitioners. They must not fall victim to detailed and intricate documentation to explain the Acceptance Criteria. The purpose of User Stories is efficiency in communication. Elaborate documentation can lead misuse of time and effort.

Even though documentation is not given utmost importance in User Stories, a light manual can be created to record Acceptance Criteria.

➔ Salient Points of User Stories

- User Stories can be created with the effort of a few days, from one or two team members.
- The main focus of the User Story is the customer's wants.
- The partnership between the Product Owner and the Scrum team begins with a User Story.
- It is advisable to structure the User Story in simple terms to ensure that the users and stakeholders, who have knowledge of the domain, understand it clearly.
- Documentation of User Stories can be trimmed down once they are implemented with the help of Automated Acceptance tests.
- Though there are several ways of describing the User Story, some teams use the following template:

  As <a user> I want to <do something> so that <value statement>

  The 'I-want' statement sums up the value the customer is trying to draw out from the said functionality.
- There are several other components of the Product Backlog, other than the User Story.
- The labor of the Scrum team should be evident at all times, however, all the effort does not necessarily have to centre on a specific User Story or Product Backlog Item.

➔ **Invest**

The following factors help to judge whether a User Story is of high quality:

- **Independent**: It is left to the discretion of the Product Owner, to solicit stories in any sequence of their choice. Therefore, it would be wise to reduce dependencies between stories, as it reduces the flexibility of the development team and the Product Owner. Certain stories are independent, as they are built as per the requirements of the Product Owner. For example, a request is made by a Canadian reader, to have the books delivered to his/her Canadian address followed by a request from an American reader, to have his/her books shipped to his/her American address. In this instance, the team would indicate to the Product Owner that the implementation of the first request is costlier than the second one, as the infrastructure must be built for it. The second request automatically becomes cheaper, as the infrastructure will be in place.
- **Negotiable**: One of the best features of a User Story is that it allows the team and the Product Owner to fill in the finer details at a later time. They can defer the avoidable decisions till the time implementation starts. The team can come across multiple and novel options until the decision needs to be made.
- **Valuable**: Each story that is developed must be able to convey some value to the customer, independently. This compels the team to divide the work as per the value and not according to the technological aspects. Scrum and XP teams believe that infrastructure needs to be put in place only to resolve problems. It should not become the focal point of a User Story.
- **Estimable**: The team's inexperience or disparity in shared understanding should not result in an inaccurate estimate of a story. Experiments to test these difficulties or Spikes, should be run to expand their knowledge base and help estimate the difficulty of the story.
- **Sized Appropriately**: If Backlogs are portrayed in a pyramid structure, the top layer, the stories must be small. The team should be able to create five to ten complete stories at the end of each Sprint, by approximately three Sprints. The middle layer must have stories of medium size, between four and ten Sprints, with the team turning out one to two stories at the conclusion of each Sprint. The bottom layer can consist of large stories.
- **Testable**: It is obvious how the story will be tested.

11.7 Agile Best Practices

Agile lists several best practices under it. Some of the practices have been discussed in details are as follows:

➔ **Strategic Planning**

- The Agile process is more than just following the Agile methods. The process has more to do with the way a team thinks and adopts the philosophy, than just following through the steps listed under the method. This is often referred to as 'being Agile' as opposed to just using it.

- Steady migration to Agile methodologies is recommended. Do not force the Agile methodology on to the team. Integrate the existing process gradually with the Agile methodology. Carry out tests and set up the requisite infrastructure.
- Study and keep in touch with other organizations that are using Agile. It would be prudent to communicate with other organizations that work using Agile, to get a better understanding of how the process works. It could include the private sector, academic segments, and federal practitioners.
- Establish a structure which adheres to change disciplines and build a change vision. Launch a strategy which clarifies the vision of change of the organization, to the teams. This way the teams know what to expect and can contribute towards the new vision. It also helps to develop new strategies for communication.
- Equip the organization to deal with problems. This approach reiterates that Agile is not a problem-free process. Users may find themselves back at square one, with deep-rooted software methods.
- Enlist Agile guidance and adoption strategies. This method proposes to have the guidance and adoption strategies ready before beginning the project work. These modules could also be borrowed from external sources.

➔ **Organizational Commitment and Collaboration**

- The components of Agile projects and the organization's Agile approach should be aligned. Promote Agile processes and encourage all the teams to add value to the approach as well as the projects. Soliciting the input of external sources such as sponsors adds value to this approach.
- Identify an Agile champion within the ranks of senior management, who would act as a promoter of the approach and have the power to resolve problems.
- Each functioning team must include an Agile coach. If not an Agile coach, any team member who has experience working with Agile methodology, should be able to assist the team through rough patches. It is vital for the team to have hands-on experience with Agile, to learn for themselves, in addition to training.
- Grow the number of Agile experts in the team. A small team, with a span of about 18 members (and a minimum of seven) can decide on the product and the technology required to create it. This should be process-dependant and not person-dependant. The entire knowledge base should not be concentrated only on one person.

➔ **Preparation**

- It is essential to train the entire organization on the Agile process. Since every organization adapts the practices to suit the nature and type of its' projects, it would make sense to train Agile practitioners on your Agile methods. For example, the manager must be familiar with the overall aspects of the project, while the team should have in-depth knowledge of the

steps involved in an iteration, to be able to operate correctly and effectively.

- Aside from the Agile practitioners, coach the organization's subject matter experts and team members on Agile. As the team is involved in the day-to-day tasks using this approach, the team members have to become experts in the process. Especially, in cases of iterations that are fast-paced and consists of processes that call for automation. This helps reduce delays in production. For example, a team member dealing with financial customers is required to possess extensive knowledge of the customer base and their requirements.
- Introduction of Agile jargon and examples in place of normal business terms will help with the integration of Agile processes with those of the organization. Expressions such as User Stories, instead of requirements and Agile Center of Excellence in place of Project Management Office will help steer the mindset of the team-members towards Agile. Examples can include the creation of User Stories by small teams and the scope involved.
- Alter the physical environment to encourage team-work and collaboration. A widespread method to create this kind of atmosphere is place the team in a single room, where they can constantly mingle and work together. Tools such as conference calls and bridge calls can be used to connect with teams remotely. Restructuring the office space can also help create an environment conducive of team-work.
- Focus more on the result and not the output of a project, using Agile. One way to accomplish this is by creating a vision statement, which is qualitative in nature. For example, a vision statement can read reduction of processing time, by 'n'% by fixed deadline. This approach steers away from vision statements that are quantitative, and focus on the number of codes produced, for instance.
- Confer with the customers to oversee the requirements as an ongoing process. The teams can produce regular, usable demonstrations which help gain the confidence of reviews and investors. This reduces the requirement for formal supervision and oversight documents by a significant amount.
- Make sure that the team is clear on how the user stories are defined. They need to be comprehensive – including the details of documentation, testing, and packaging. They should also be verified independently. The results should be measureable against subjective judgment.
- Contracts must be designed a provision to accommodate Agile methodologies. In case contracts are drawn up with more traditional approaches, such as the waterfall approach, in mind, they will not be accommodating of numerous changes and verifications done under Agile. This could also hinder the opportunity to adopt the Agile methodology.

➔ **Execution**

- The length of each iteration should be the same. For instance, determine that each iteration will be spaced at two weeks, within a release, thus, establishing a timeline.
- Leverage different methodologies such as Scrum and XP, if applicable. A combination of

these different approaches can be used for your projects. For example, the technical practices from XP in combination with Scrum project management practices could work well for your project.

- Involve the customer in the development process and use test-driven developments for designing. Test driven developments refer to the software codes written to pass a test. By engaging the customer in these tests, helps obtain their inputs while developing software.
- Ensure security and progress monitoring systems are in place with respect to backlogs. The evaluation and updates regarding backlogs should form a part of the reviews, as they reveal the time and costs involved. Their security and progress reports should be tracked simultaneously with the delivery of iterations, and not after.
- Every defect in an iteration should be catalogued, preferably in a tool, where they can be tracked and addressed for later iterations. In this manner, the defects created at the end of an iteration, usually during a demonstration are queued and dealt with before the next iteration is started.
- Use the automated tools to accelerate the delivery process. For example, some tools help track the changes made to a software. Sandboxes, which are compliant development sites, help customers build a model of a software in an environment where they can customize the architectural components and the security standards.
- Testing should be done at regular intervals throughout the life-cycle of the process, instead of testing at the very end of the project. This helps reduce the cost, alterations, and risks significantly.

➔ **Evaluation**

- Stakeholder and customer feedback is an essential input and should be solicited on a regular basis. This should also be tracked and implemented as and when possible. The practice of requesting and using feedback is connected to better customer satisfaction, increased level of motivation of the technical team and a reduction in risks.
- Agile should be embraced at a micro and macro level. This means that the Agile practices should be adopted at the project level and at the organizational level. The key emphasis is on constant improvement. Automated tests and vision control tools are examples of improvements. Any concerns that arise can be followed up on, with the help of backlogs on a project level and an organizational level.
- Problem areas should be identified and addressed at all times. Brainstorm with the team and promote discussions to tackle problems at the project and organizational levels.
- Project value can be determined in several ways such as the budget or delivery schedule that is established at the beginning of a project. If cost or schedule criteria are used to determine the value, any changes that occur will provide an inaccurate picture. Under the Agile methodology it is gauged through customer feedback and return on investment.

- Also, evaluating the cost of a solution against the value of the solution offers a good measure of the project value.
- At the end of each iteration, give a presentation to show customers, that the important components were taken care of in the initial iterations and also, how their feedback was tracked and leveraged to improve the project.
- Leverage tools and data to track the progress made on a project. Success indicators can be tracked through customer satisfaction, capacity utilization and efficiency, decrease in overtime, and reduction in stress. Metrics will help determine the progress in numbers.
- Progress reports should be circulated daily within the team and within the organization, if possible. Putting up a daily progress chart, which reflects constant feedback, helps the team make course corrections, while doing revisions.

11.8 Check Your Progress

1. There are _____ processes under FDD.

(A)	Three	(C)	Seven
(B)	Five	(D)	Twenty-two

2. Under UML in color, Pink refers to _____.

(A)	Serial Numbers on a bill
(B)	The role of a person
(C)	Item description
(D)	A point in time/time interval that can be traced
(E)	A system administrator

3. The main aim of Lean Software Development is to _____.

(A)	Reduce errors and wasted time, maximize efficiency
(B)	Reporting and documentation to be done in a timely manner
(C)	Collate all the best practices from 1980 till date and publish them
(D)	Assignment of ownership of features

4. LSD is a concept that was derived from the _____ Industries.

(A)	IT and Manufacturing	(C)	Banking
(B)	Programming	(D)	Education

5. AUP has _____ and _____.

(A)	5 phases, 4 disciplines	(C)	4 phases, 7 disciplines
(B)	22 phases, 5 disciplines	(D)	3 phases, 8 disciplines

6. User stories are often written up in _____.

(A)	Sticky notes or post-its	(C)	Team journals
(B)	Hand-books or manuals	(D)	Guide-books

7. User stories have three components – cards, confirmations, and _____.

(A)	Conversations	(C)	Cancellations
(B)	Conclusions	(D)	Criteria

8. Iteration defects and customer feedback can be tracked using _____.

(A)	Automated Tests	(C)	Risk Charts
(B)	Queues	(D)	Tools such as Backlogs

11.8.1 Answers

1.	B
2.	D
3.	A
4.	A
5.	C
6.	A
7.	A
8.	D



Summary

- FDD is highly adaptive software that is iterative, pays attention to quality and provides working results at each step.
- There are five processes under FDD that are Develop an overall model, Build feature list, Plan by feature, Design by feature, and Build by feature.
- LSD is dedicated to reducing errors and wasted time while maximizing education and efficiency.
- There are seven principles under Lean Software Development namely, Waste elimination, Encouraging learning, Late decision-making, Fast delivery, Team empowerment, Built-in integrity, and Vision of the big picture.
- AUP is a simple and straightforward approach to developing business application software using agile techniques and concepts.
- There are four phases under AUP namely, Inception, Elaboration, Construction, and Transition.
- Some of the best practices under Agile are namely, Strategic planning, Organizational commitment and collaboration, Preparation, Execution, and Evaluation.



Onlinevarsity

your e-way to learning

Session - 12
Comparing Agile with Traditional Models

Welcome to the Session, **Comparing Agile with Traditional Models**.

This session explains the parameters to be considered for selecting a right SDLC model. The session also provides the differences between traditional methods, such as Waterfall model when compared with Agile along with a case study. Finally, the session compares the features of the different SDLC methods.

In this Session, you will learn to:

- List the parameters for selecting a right SDLC model
- Compare Agile with traditional Waterfall SDLC model
- Describe a case study for Waterfall and Agile approach
- Compare features of different SDLC models



12.1 Introduction

One of the most critical factors in ensuring success of your project is the SDLC methodology used in it. The methodology used assists in software development and its implementation. It is crucial to determine which process fits your project best. There are several methodologies to choose from traditional Waterfall model to widely used Agile methodologies.

By choosing an inappropriate SDLC, you can expose your project to various risks. Some of the risks are mentioned as follows:

- ➔ Damage to the quality of software product
- ➔ Inability to maintain the budget of the project
- ➔ Difficulty in meeting deadlines
- ➔ Impair the initial goals
- ➔ Hamper the software project management best practices

12.1.1 Parameters to Choose Right Model

The features of a SDLC methodology can prove to be beneficial or otherwise, based on the type of the software project. All SDLC methodologies have their advantages and disadvantages. As the characteristics of more than one SDLC methodologies may suit a project, a mix of two or more methodologies based on the project requirements, is often chosen for the best fit.

Consider an example that illustrates this point best is that of a screwdriver toolkit with different sizes and shapes of screws. Different screws are chosen depending on the requirement, whereas the driver remains the same. Similarly, a base requirements of a project needs to be determined by the team. Then, the right SDLC process model can be choose from the existing SDLC methodologies. The best practices of the selected SDLC methodology can be customized as per your project requirements. This will pave the way for top-quality software development.

Thus, selecting appropriate SDLC is extremely important for the success of the project.

Some of the factors that need to be considered while selecting the SDLC methodology are as follows:

- ➔ Nature of the project
- ➔ Nature of the product – if it is a new product or a new feature
- ➔ Complexity of the functionalities in the project
- ➔ Size and resource availability in your project team

- Team composition and location – if the entire team is in-house or if it is spread-out geographically
- Identifying your organization's business nature and strategies, as software development requirements differ from industry to industry
- Work atmosphere and culture – if the members of the teams are used to collaborative work culture, as SDLC methodologies such as Agile, emphasize on teamwork
- The potential and competence of your team members, in terms of engineering

The SDLC process models and practices adopted should align with the business objectives of your organization. This will increase the team productivity and will also help in delivering better product to customers.

One of the most commonly used process model is the Waterfall model. It is a process that is of a sequential design. This process is thought to follow a steady downward trend, going through phases such as conception, analysis, design, testing, production, and maintenance. This process has been adopted from the construction and manufacturing industries.

Similarly, one of the modern SDLC processes are the Agile methods that proposes alternatives to traditional project management. Agile approaches are typically used in software development to help businesses respond to unpredictability.

12.2 Comparing Agile with Waterfall Model

Table 12.1 shows comparison between Agile model and Waterfall model.

Agile	Waterfall
Idea and Design	
With Agile, the important components are designed first. This gives the customer the chance to alter their requirements, without affecting the progress of the project. Agile also gives a provision for adding new functionalities to existing design.	As a first step, a detailed plan is drawn up, accounting for every single detail, from start to finish of the project. Problems can occur if inaccurate and inadequate information was provided, while the plan was being drafted.
Coordinated Actions	
The customer's team is involved in the development process and works hand-in-hand with the development team. This helps to avoid any problems that may arise. Also, detection of incorrect documentation can be done sooner.	The development team works independently without barely any inputs or involvement from the customer. This also means that the entire responsibility of the project lies solely with the development team.
Project Execution	
The development process works in iterations and each iteration is executed in two to four weeks.	The development process is divided into phases.

Agile	Waterfall
Project Execution	
As the time interval between iterations is minimal, new versions can be upgraded with important features. There is minimal planning involved in this process.	Each phase requires thorough planning and considerable time, as compared to iterations. When one phase is completed, the team is required to prepare a detailed plan factoring in the requirements of the next phase.
Testing	
Testing is incorporated into each iteration as a phase which allows regression testing before deployment of new functions and releases.	Testing is a phase that follows the development phase. This can be done post completion of all modules, as each module cannot be tested independently. The entire software has to be assembled before verification can be carried out.
Deliverables	
Given that the important features are completed first, they can be delivered to the client, each time an iteration ends. The client can use the new features upon delivery.	The features are developed in the implementation phase and are delivered all at once to the client. This also makes it difficult for the client to suggest any alterations, as this would require the entire implementation phase to be done again.
Documentation	
Software development is given precedence in this process. The documentation is more of a handbook which is easy to understand. A formal document is not maintained.	Documentation is a vital part of this process. All kinds of documents are drawn up – user guides, instruction manuals, and other written material. This is used for training purposes and can also help other teams upgrade their software.
Design Time	
The development time for small projects is estimated and implemented quickly, however, it is a challenge to calculate the development time for larger projects.	The schedule for delivery of the software is worked into a detailed plan and the release date can be calculated for all the projects.

Table 12.1: Comparison between Agile and Waterfall Model

Figure 12.1 provides a snapshot of success and failure percentage for Agile and Waterfall models based on a study by the Standish Group.

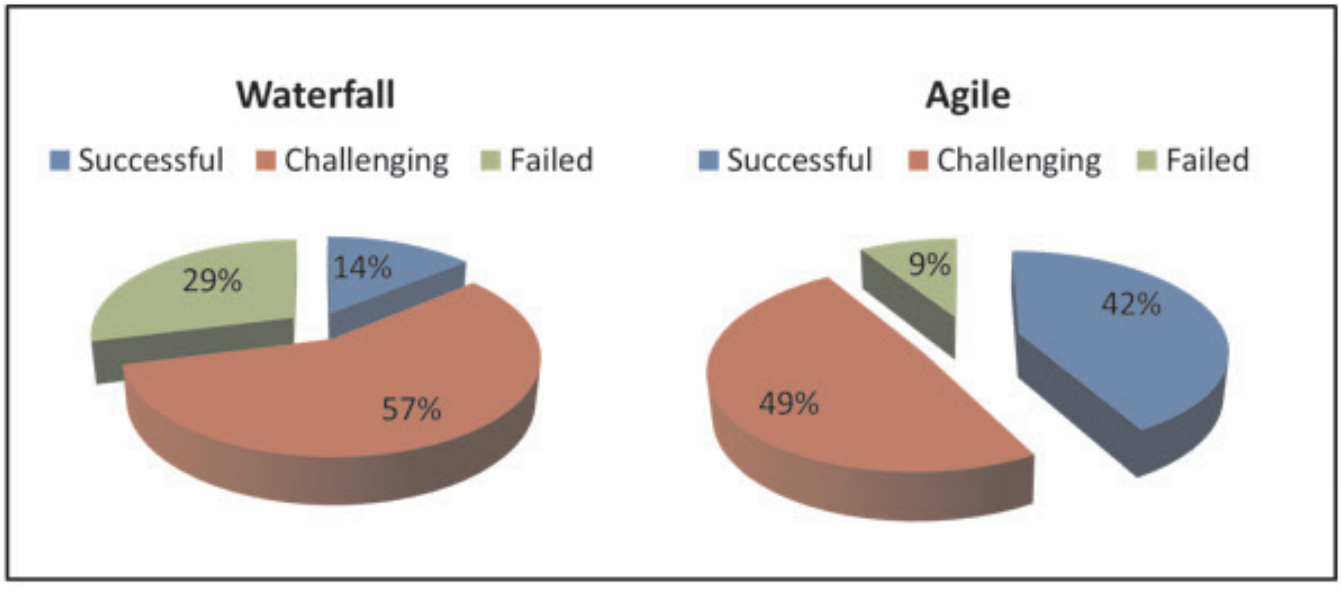


Figure 12.1: Success and Failure Percentage

12.3 Case study for Agile and Waterfall Models

Consider a case study of Bill Holst, the president and consulting software engineer for Prescient Software Engineering. It outsourced the development work of two projects based on distribution design systems to two teams of software engineers. The first team handled an electric project. The project is based on the Waterfall approach. Similarly, the second team handled a gas project which is based on the consideration of Time and Materials pricing.

➔ **Efforts of First Team**

In the Waterfall approach, the first task was to define the scope and list the requirements of the project.

- **Initial Design**
  - The project requirements were taken from the customer and the team started phase-by-phase work on the project.
  - After the analysis and design tasks were completed by the team, the coding phase began. Here, the customer involvement was not done to verify its initial requirements. The only time when customer was involved is during the User Acceptance Test, that is, at the end of the project.

Concepts

- **Result**

  Some of the shortfalls of the Waterfall approach observed in the electric project are listed as follows:

  - There was a huge gap between the estimated delivery date and the actual delivery date.
  - Some of the functionalities in the developed and delivered software did not match with the customer's requirements.
  - More cost involvement to rework on the project functionalities.

➔ **Efforts of Second Team**

In the Agile process, user-collaboration is continuous which acted as a key to the success of the project.

- **Agile Process – Initial Transition**

  In spite of mild disagreements from the team, an Agile training team was engaged to coach the team through the transition. The initial reaction of the team to Agile was that, it was very 'hands-on' and stressed on commitment. Even with the new process, the project hit several rough spots. The logic was complex and elaborate and did not match the test cases. The velocity and burn-down charts showed a downward trajectory when it came to progress of the project. The team made the inevitable decision to stop and regroup.

- **Turning Point**

  The 7th iteration was chosen for in-depth study and as a basis to reconsider the requirements. The developers were given two weeks' time off. The field engineers reworked the requirements, based on the pseudo-codes that were used to define the original requirements. They reduced the 800 lines pseudo-code to 200 lines, by using their new found understanding of the system and its functionalities, in a rational manner. This redesign proved to be a turning point in the project.

  The Agile methodology is a flexible model that provides for changes and redesign. With any other approach, this move would have cost the business extremely. The provision to make changes mid-way and a flexible pricing model turned out to be crucial for the success of this project.

- **Results**

  The redesign of the requirements turned the project around. The team adopted the Agile process entirely, clearing the backlog and delivering the project, before the deadline and within the estimated budget. This was accomplished despite a two-week delay to regroup.

  This project was deemed a success in all aspects. Especially when compared to the earlier electric project, the gas project could be graded higher in terms of quality, test cases and other parameters. The best feature of the project were its budget and was delivered time.

Some of the factors of the Agile approach observed in the gas project are listed as follows:

- The Agile training team and the decision to regroup were credited with the success of the gas project.
- The flexibility that comes with Agile was a key factor, as it allowed the team to regroup half-way through the project.

12.4 Comparison between Different SDLC Models

The comparisons between different SDLC models in relation to their features, is given in table 12.2.

Model/ Features	Waterfall	V-Model	Prototype	Incremental	Spiral	RAD
Requirement Specifications	Beginning	Beginning	Changed frequently	Beginning	Beginning	Time-boxed
Understanding Requirements	Well understood	Easily understood	Not well understood	Well understood	Well understood	Easily understood
Cost	Low	Expensive	High	Low	Expensive	Low
Guarantee of Success	Low	High	Good	High	High	Good
Resource Control	Yes	Yes	No	Yes	Yes	Yes
Cost Control	Yes	Yes	No	No	Yes	Yes
Simplicity	Simple	Intermed-iate	Simple	Intermediate	Intermed-iate	Very simple
Risk Involvement	High	Low	Low	Easy to manage	Low	Very low
Expertise Required	High	Medium	Medium	High	High	Medium
Changes Incorporated	Difficult	Difficult	Easy	Easy	Easy	Easy
Risk Analysis	Only at beginning	Yes	No risk analysis	No risk analysis	Yes	Low
User Involvement	Only at beginning	At beginning	High	Intermediate	High	Only at the beginning
Overlapping Phases	No overlapping	No	Yes	No	Yes	No
Flexibility	Rigid	Little flexible	Highly flexible	Less flexible	Flexible	Highly flexible
Maintenance	Least	Least	Routine maintenance	Promotes maintainability	Typical	Easily maintained
Integrity and Security	Vital	Limited	Weak	Robust	High	Vital

Model/ Features	Waterfall	V-Model	Prototype	Incremental	Spiral	RAD
Reusability	Limited	To some extent	Weak	Yes	Yes	To some extent
Interface	Minimal	Minimal	Crucial	Crucial	Crucial	Minimal
Documentation and Training Required	Vital	Yes	Weak	Yes	Yes	Limited
Time Frame	Long	Acc to project size	Short	Very long	Long	Short

Table 12.2: Comparing Different SDLC Models

12.5 Check Your Progress

1. Which of the following factors are to be considered while selecting an SDLC model?

(A)	Nature of the project
(B)	Simplicity of the data in the project
(C)	Size and resource availability in your project team
(D)	Team composition and location – if the entire team is in-house or if it is spread-out geographically

(A)	A, B	(C)	B, C, and D
(B)	A, B, and C	(D)	A, C, and D

2. When compared with the Waterfall model, Agile focuses on _____.

(A)	Detailed documentation	(C)	Customer collaboration at every stage
(B)	Elaborate planning	(D)	Delivery on an exact time-table to be communicated at the beginning

3. The Waterfall model is a process that is of a _____.

(A)	Automated Tests	(C)	Sequential Design
(B)	Risk Charts	(D)	Elaborate Planning

4. Which of the following are the features of Waterfall model?

(A)	Iterative and incremental process
(B)	Structure and sequential process
(C)	Documentation
(D)	Big team size and projects

(A)	A, B	(C)	C, D
(B)	B, C	(D)	A, D

5. In case of Agile, precedence is given to _____.

(A)	Delivery of swift business value	(C)	Big team size and projects
(B)	Planning and high assurance	(D)	Centralized command structure

12.5.1 Answers

1.	D
2.	C
3.	C
4.	B
5.	A



Summary

- Choosing the correct SDLC approach can determine the success or failure of your project.
- There are several parameters that help you make this crucial decision, such as, complexity and nature of the project, nature and size of the organization, the structure of the team, work culture, engineering capacity, and industry.
- Agile, an SDLC methodology can be compared with traditional approaches, such as the Waterfall model, depending on the nature of the project and organization.
- Agile focuses less on documentation and more on active participation and discussion amongst the team-members.
- The SDLC methodology and practices adopted should align with the business objectives of the organization.
- All SDLC methodologies have their advantages and disadvantages.
- A mix of two or more methodologies, based on the project requirements, is often chosen for the best fit.